



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Unit 3

Session - 3.1

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

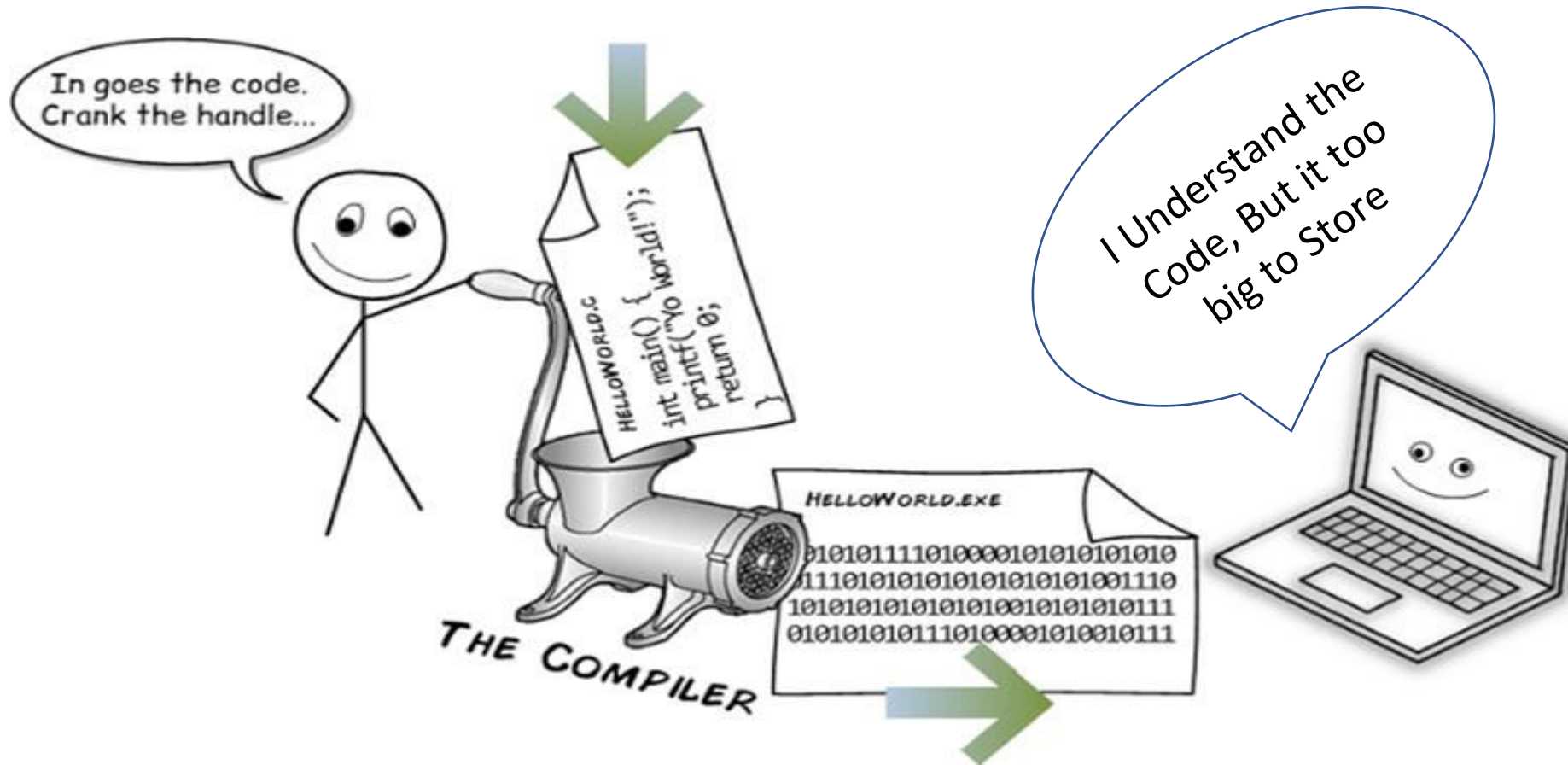
Introduction to Memory

Unit 3 : Basics of Cache and Cache Optimization

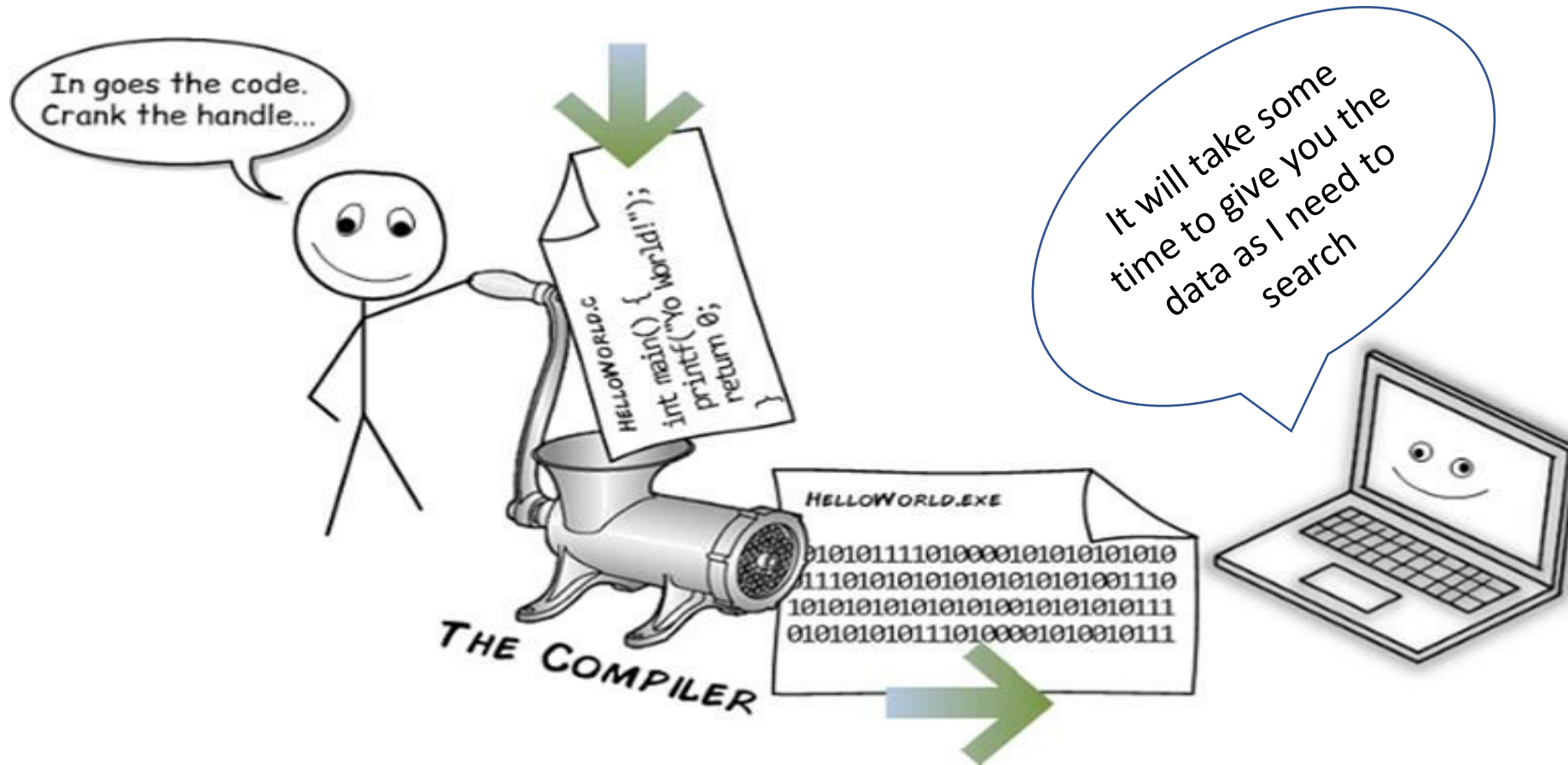
Basics of Caches - Fully Associative, Direct Mapped and Set Associativity, Cache Performance, Basic Cache Optimization- Reduce in Miss Rate. Basic Cache Optimization- Reduce Miss Penalty, Reduce Hit Time.

Computer Organization and Design”,
Patterson, Hennessey, 5th Edition,
Morgan Kaufmann, 2014

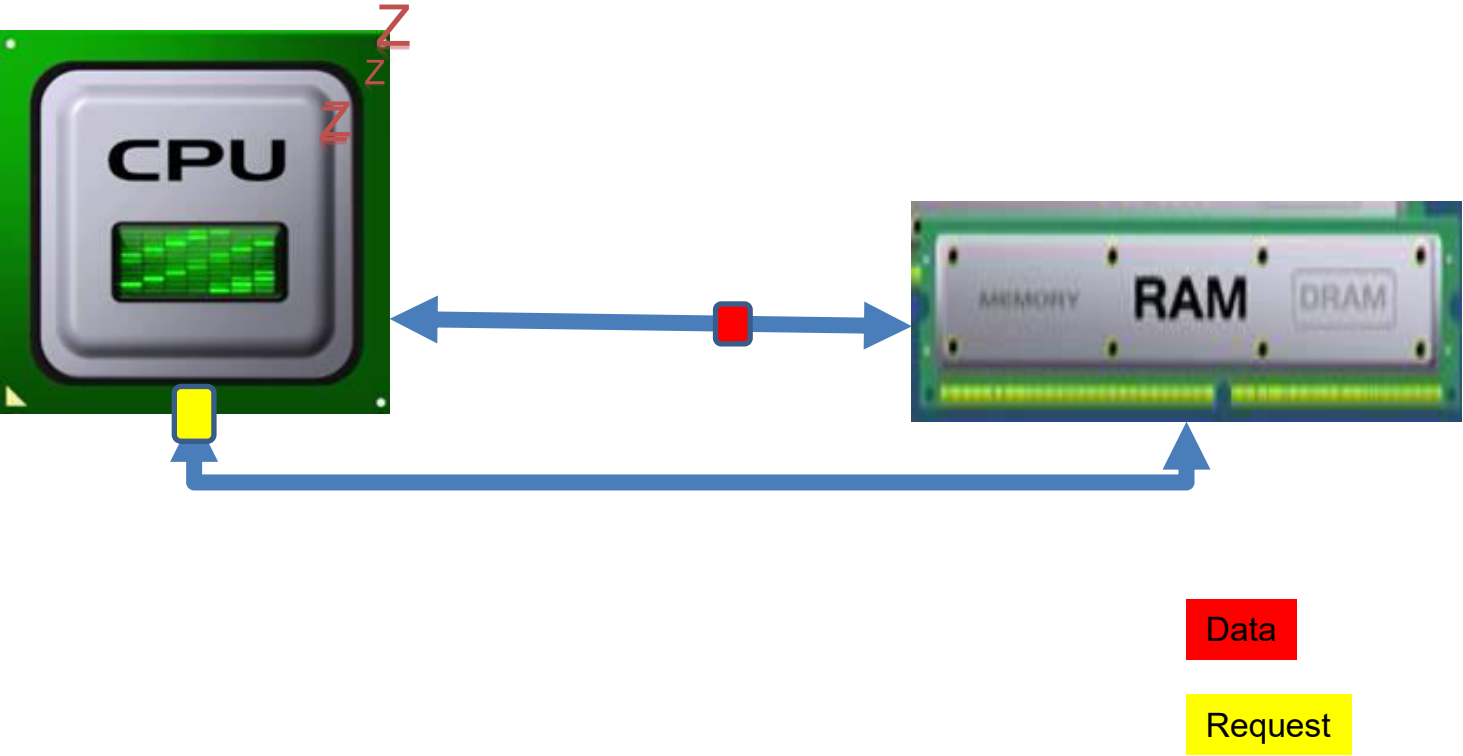
Introduction to Memory



Introduction to Memory



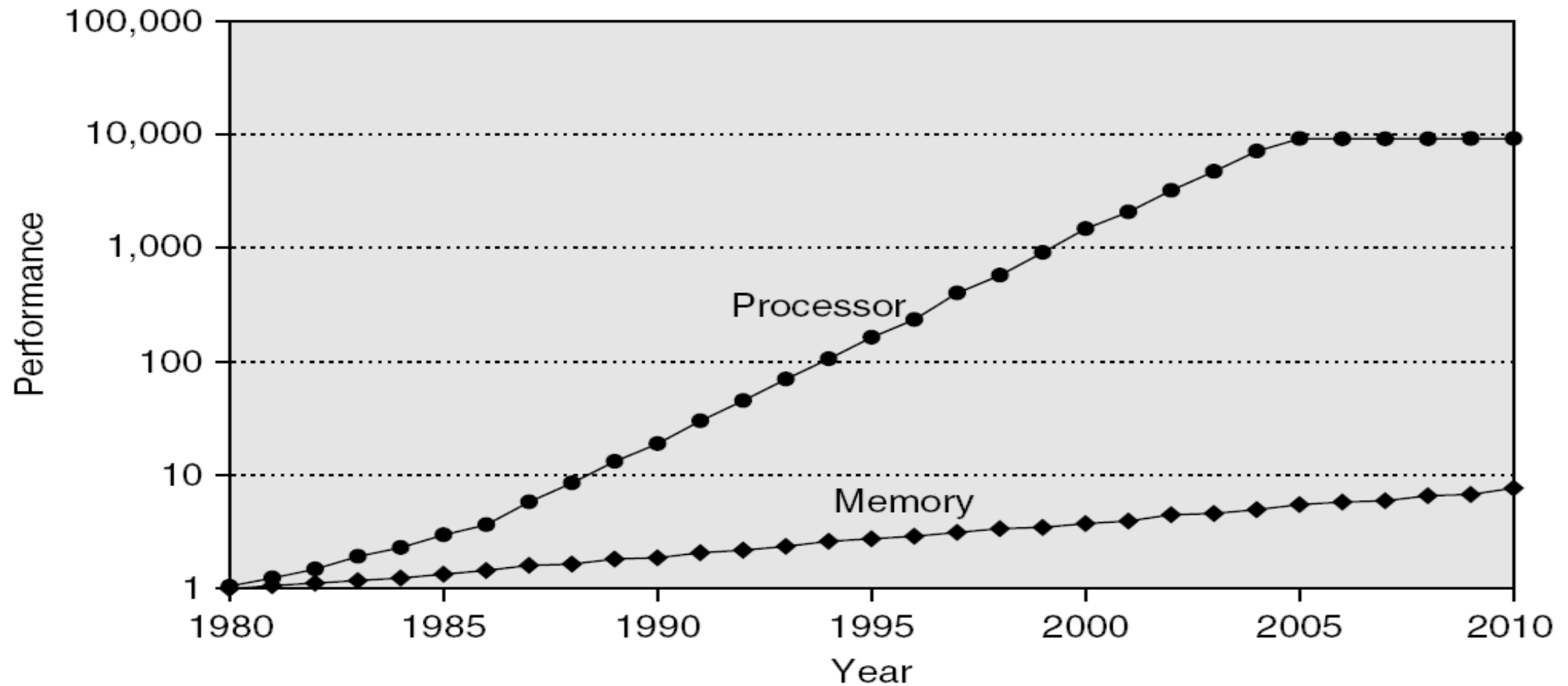
Processor Vs Memory



Processor Vs Memory

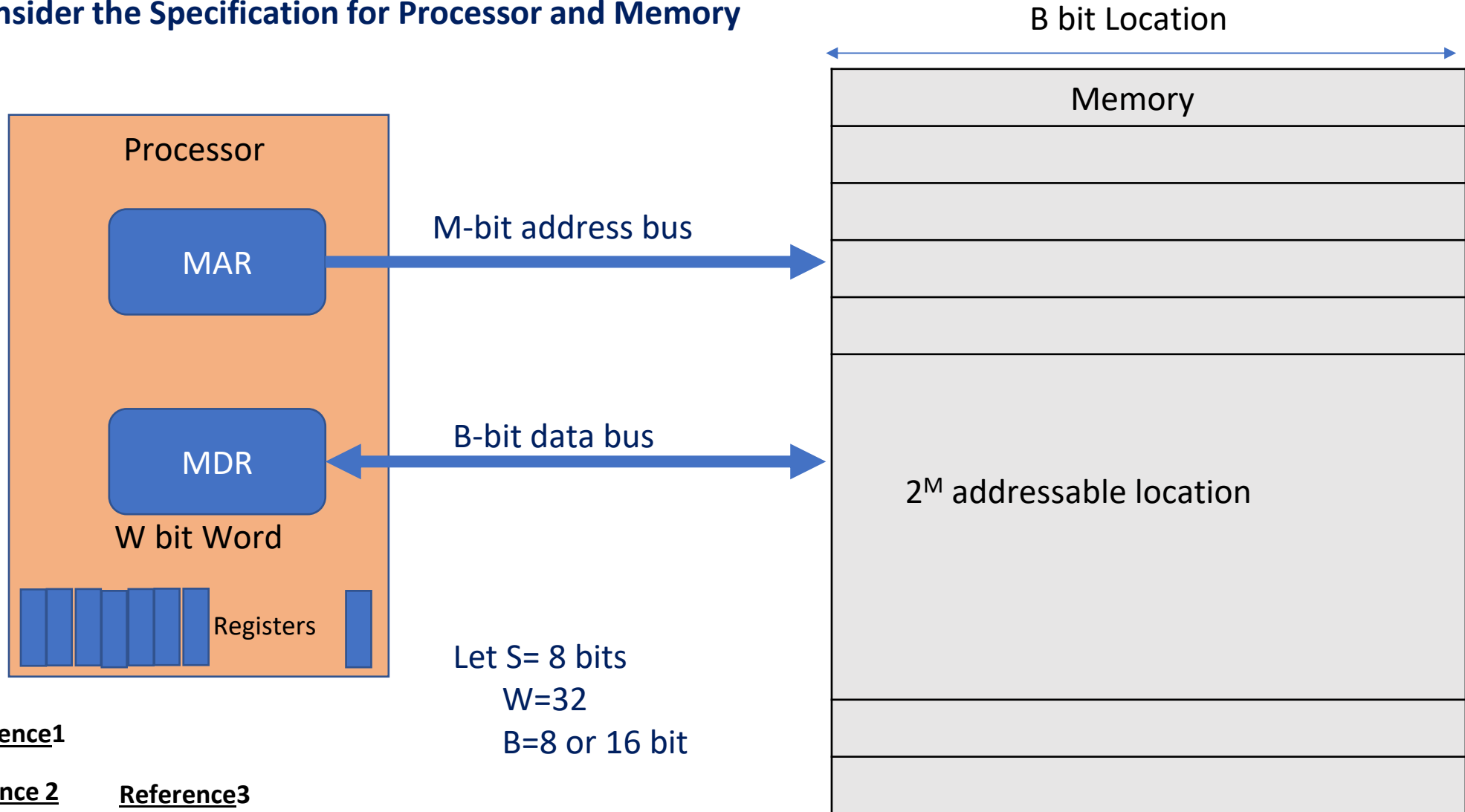
- Program is stored in Memory.
- Data required by Program stored in Memory.
- Executing Program on Processor is relatively faster than Accessing, Instruction and Data from Memory.
- CPU performance improves by 55% per year
- Memory performance improves by 7% per year
- Gap between CPU performance and memory performance increases year on year
- Fast memory technology is more expensive per bit than slower memory

Memory Performance Gap



One Reason!!!!

Consider the Specification for Processor and Memory

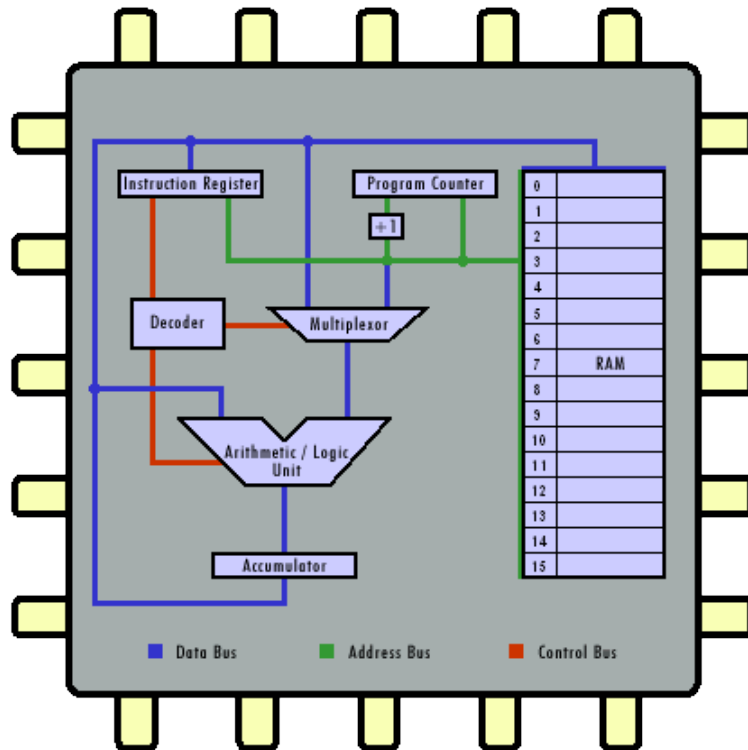


Reference1

Reference 2

Reference3

Components of CPU



- RAM
- Registers-IR,PC,accumulator
- ALU
- Control unit
- Buses-data address and control

The read operation transfers the data(bits) from word to Memory Data Register. The write operation transfers the data(bits) from Memory Data Register to word.

Data communication between CPU and memory

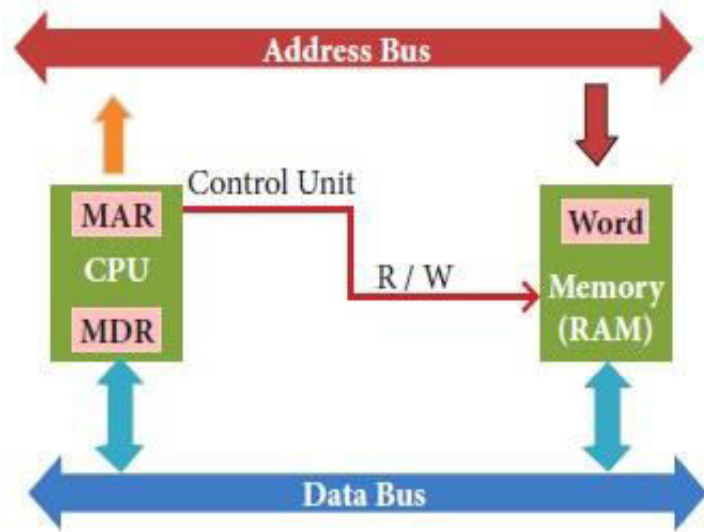


Figure 3.3 Bus connectivity between CPU and Memory

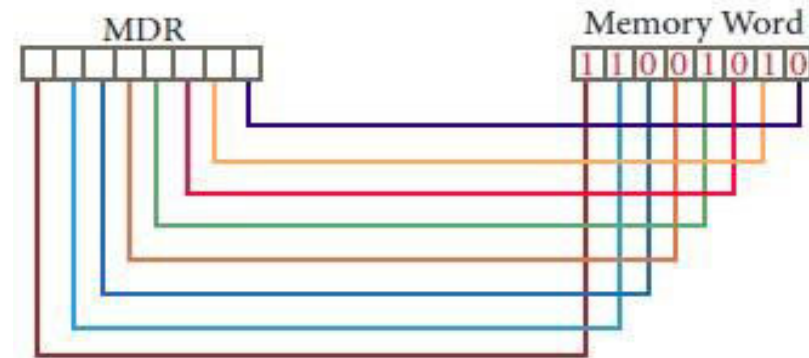


Figure 3.4 Before the read operation

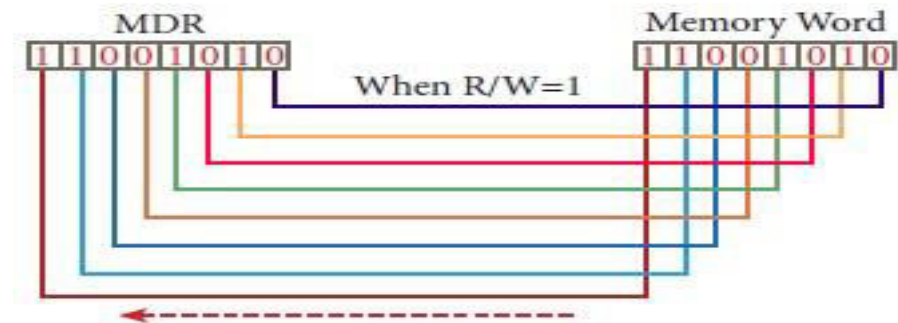
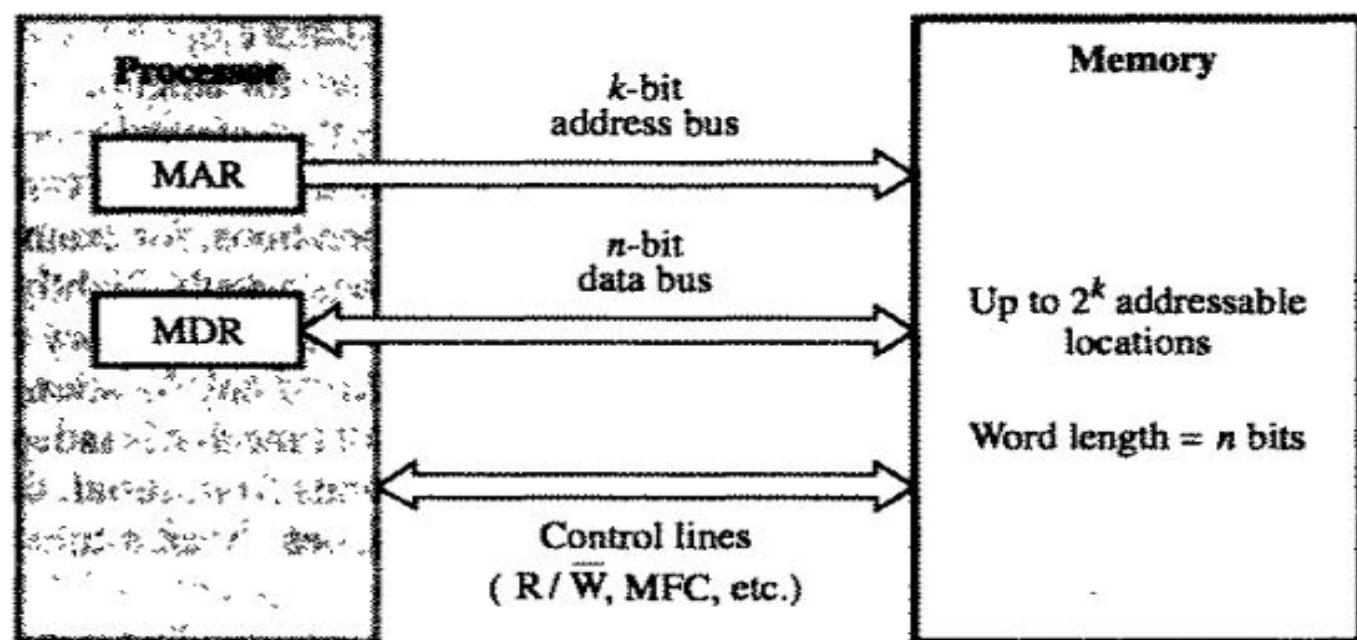


Figure 3.5 After the read operation



• **Figure 5.1** Connection of the memory to the processor.

Memory Latency is the time it takes to transfer a word of data to or from memory

Memory Bandwidth is the number of bits or bytes that can be transferred in one second.

Speed vs Size vs Cost

Requirement of Programmers

Programmers want unlimited amounts of memory with low latency

Big challenge

If Size Increases, then Speed Decreases

Funny is

If Size Decreases, then Cost Increases

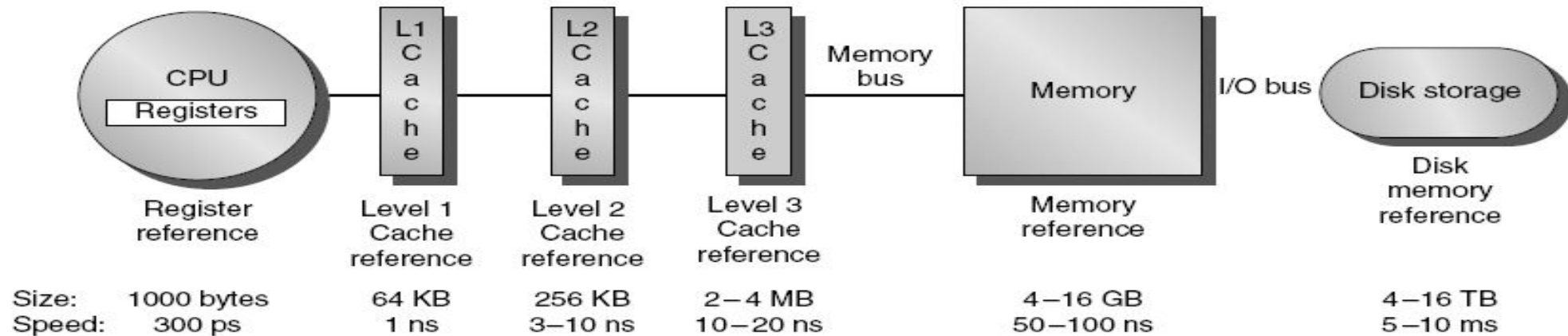
Expected by Designers

To provide a sufficiently large memory, with a reasonable speed at an affordable cost.

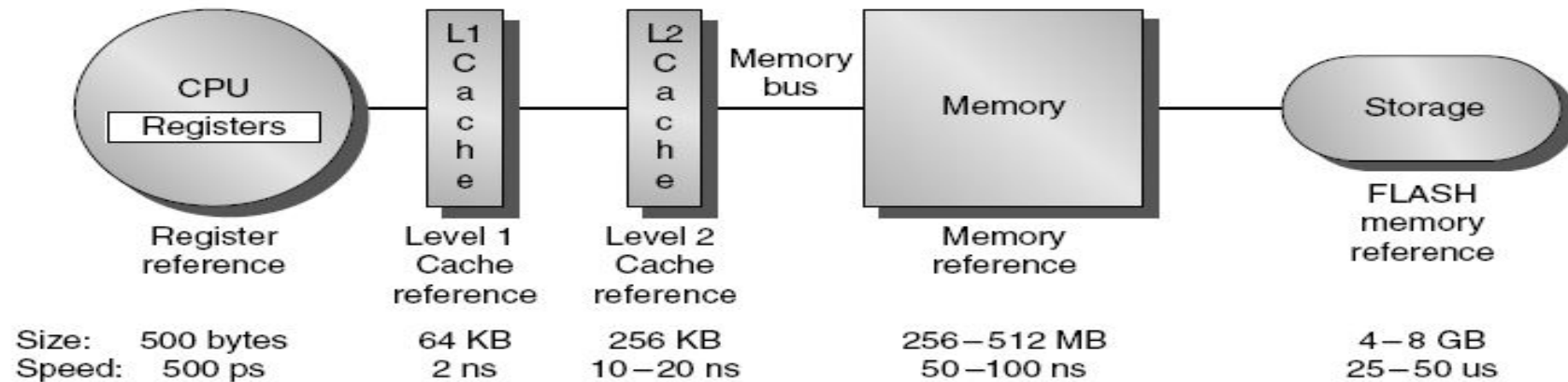
Memory Hierarchy

- Organize memory system into a hierarchy
 - Entire addressable memory space available in largest, slowest memory
 - Incrementally smaller and faster memories, each containing a subset of the memory below it, proceed in steps up toward the processor

Memory Hierarchy



(a) Memory hierarchy for server



(b) Memory hierarchy for a personal mobile device

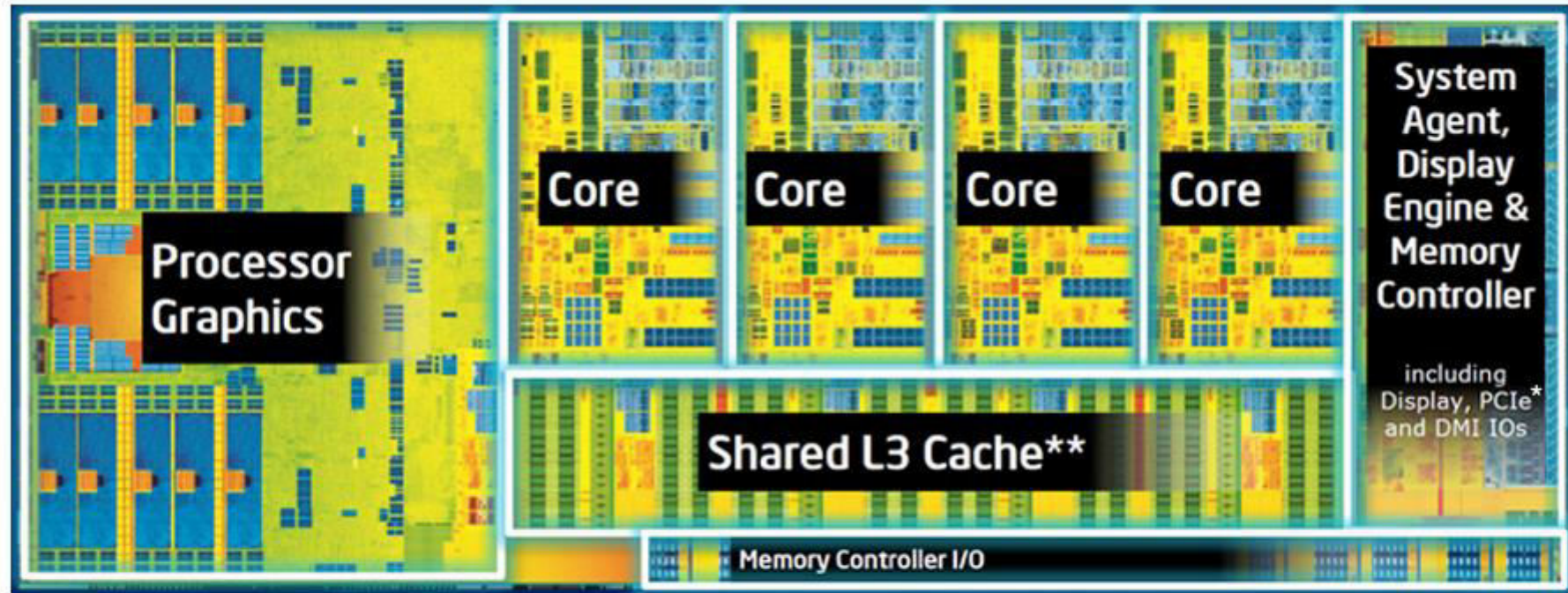
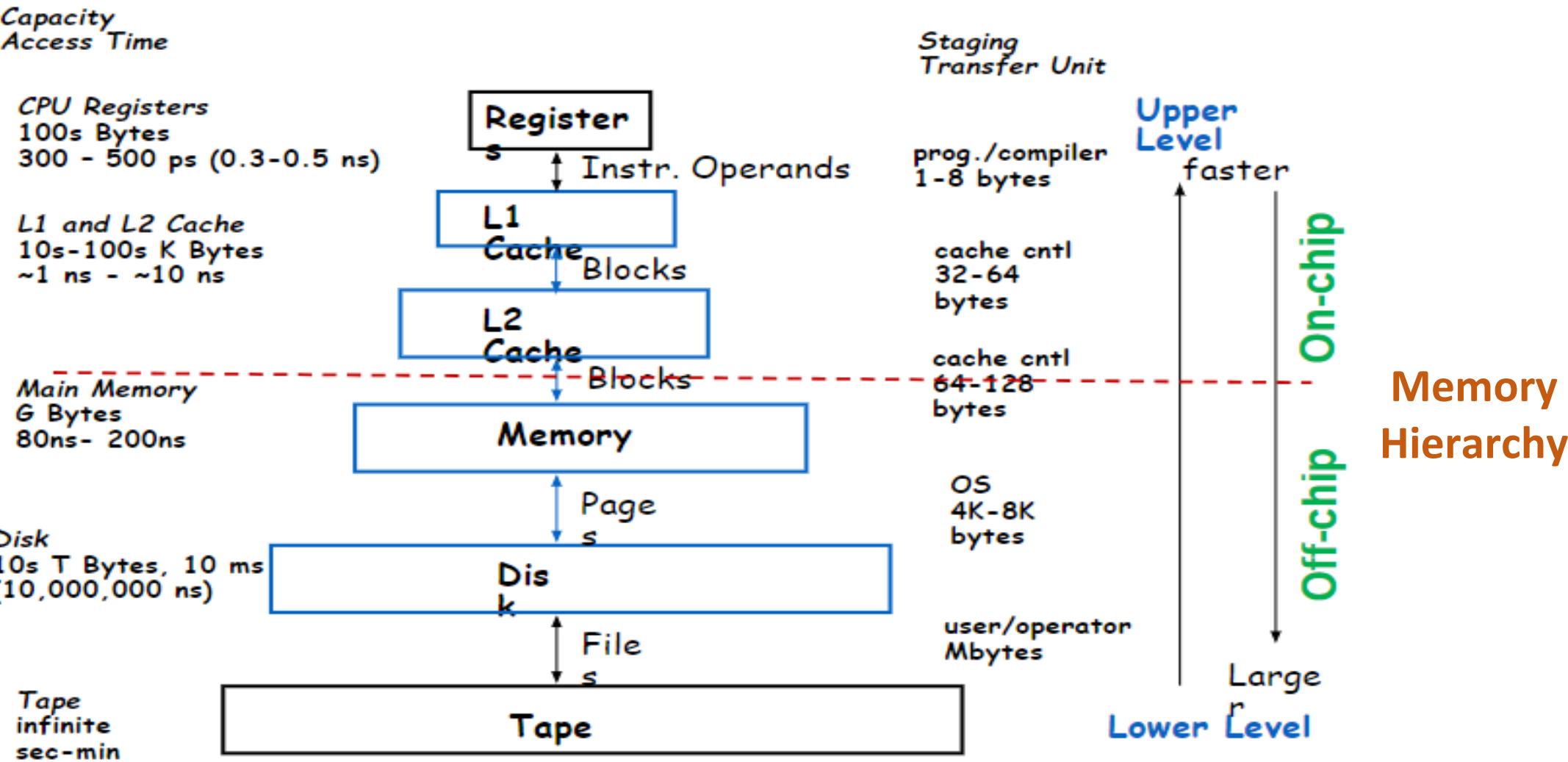


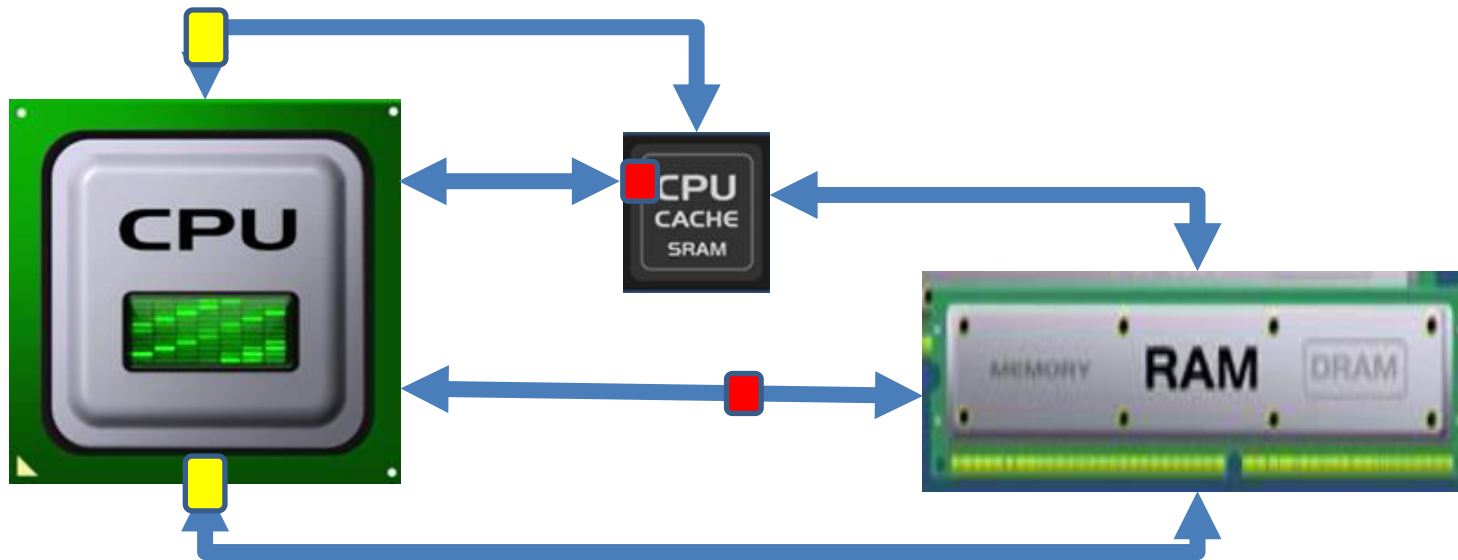
Figure 3: Intel® Core™ i7 processor internal die photograph

Memory Hierarchy



Cache

- Cache memory is an architectural arrangement which makes the main memory appear faster to the processor than it really is.

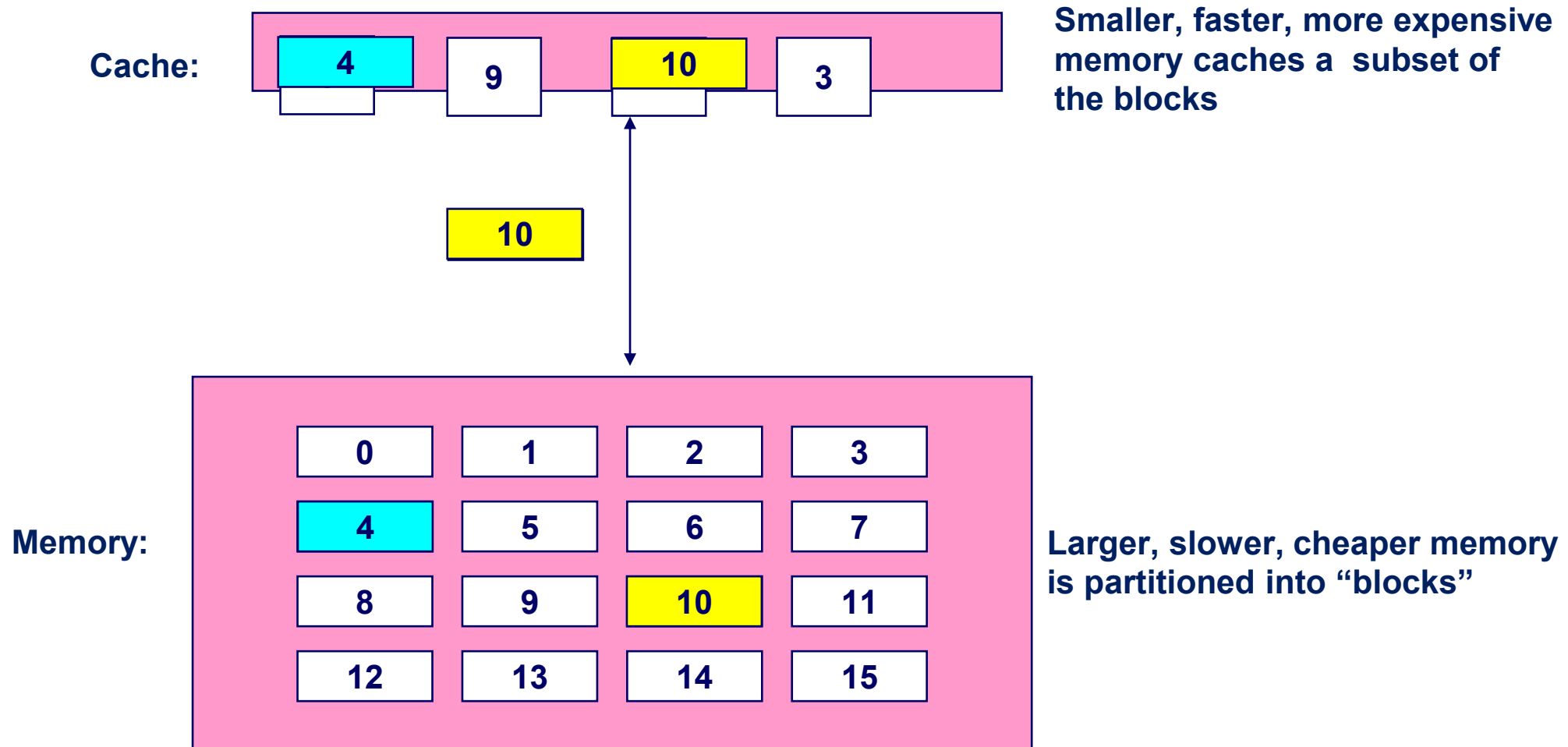


Reference

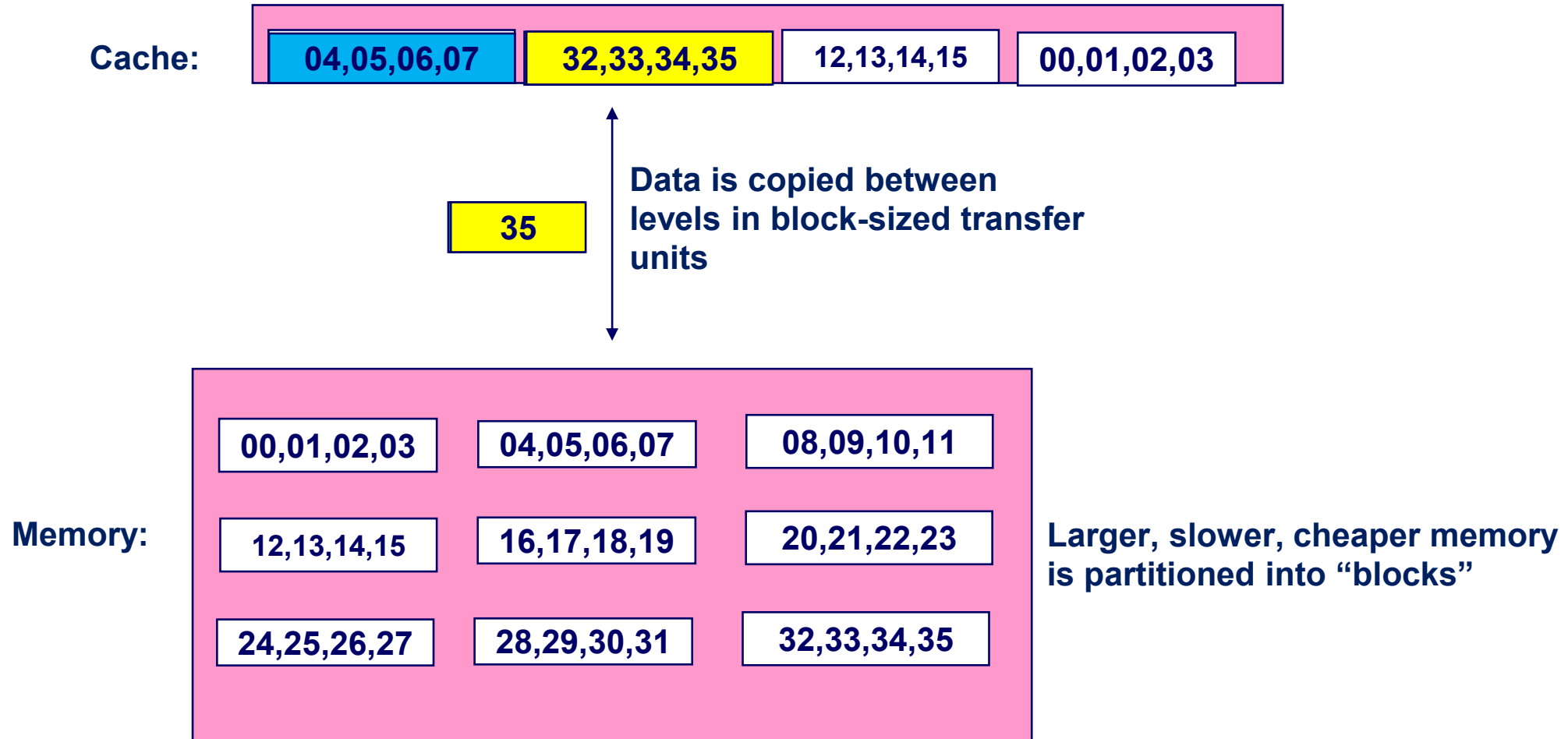
Data

Request

General Cache Mechanics



General Cache Requirement



Why is SRAM Used for Cache Memory?

- It **does not require refreshing**, making it **faster** than DRAM.
- Since cache memory is closer to the CPU, it reduces **memory latency**.
- It is **expensive**, so it is only used in small amounts (L1, L2, L3 cache)
- SRAM (Static Random Access Memory) is significantly more expensive than DRAM (Dynamic Random Access Memory) because of its complex design, higher speed, and lower power consumption

Why SRAM is Expensive

- 1. Uses 6 transistors per bit** (vs. 1 transistor + 1 capacitor in DRAM).
- 2. No refresh circuit needed**, but more **complex design**.
- 3. Faster speed (~1-10ns)** requires **higher manufacturing precision**.
- 4. Larger chip area per bit** → Lower storage density.
- 5. Low power consumption needs high-quality transistors.**

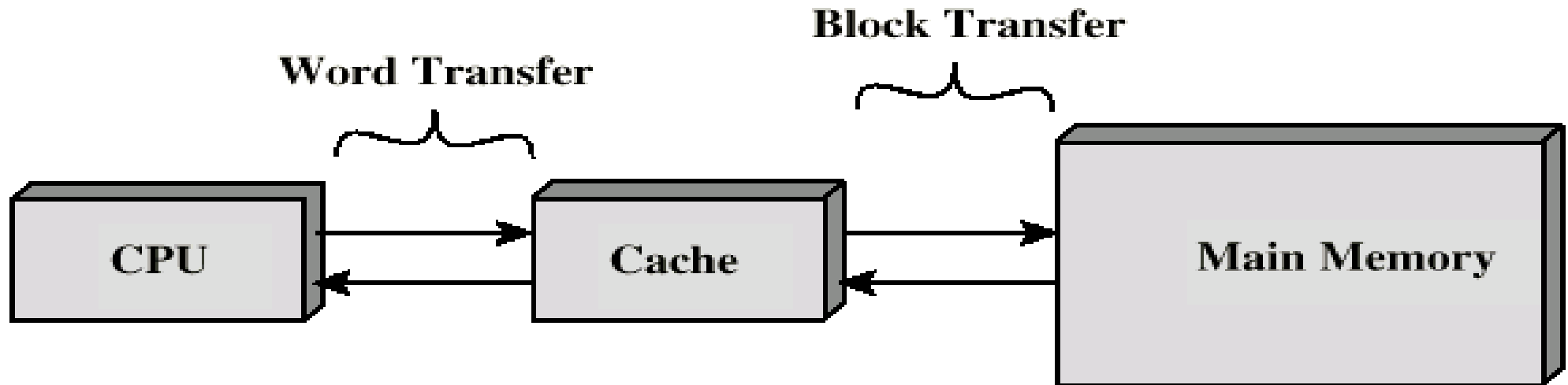
Static and Dynamic memory

Dynamic Random Access Memory (DRAM) is the primary type of volatile memory used as main memory (RAM) in computers. It stores data that the CPU needs to access quickly while executing programs.

How Does DRAM Work?

- DRAM stores data using **capacitors** and **transistors** in a **memory cell**.
- Each **bit** of data is stored in a **capacitor**, which can either hold a charge (**1**) or be empty (**0**).
- Since capacitors **lose charge over time**, the **memory controller must refresh them periodically** (every few milliseconds).
- The refresh process makes DRAM **slower than SRAM** but **more cost-effective for large memory storage**.

- Cache memory is an architectural arrangement which makes the main memory appear faster to the processor than it really is.



Fast memory technology is more expensive per bit than slower memory

Solution:

90/10 rule comes from empirical observation:
"A program spends 90% of its time in 10% of its code"

i.e The data or code accessed recently, may get accessed soon

Locality of Reference or Principle of Locality

Memory Hierarchy

- **Processor is much faster than the main memory.**
 - As a result, the processor has to spend much of its time waiting while instructions and data are being fetched from the main memory.
 - Major obstacle towards achieving good performance.
- **Speed of the main memory cannot be increased beyond a certain point.**
- Cache memory is an architectural arrangement which makes the main memory appear faster to the processor than it really is.
- Cache memory is based on the property of computer programs known as **“locality of reference”**.

PRINCIPLE OF LOCALITY

- Temporal locality (locality in time): if an item is referenced, it will tend to be referenced again soon.
- Spatial locality (locality in space): if an item is referenced, items whose addresses are close by will tend to be referenced soon.

Observation: temporal locality means that we don't put all program into memory whereas spatial locality means that we don't put all data into memory, hence we have "Memory Hierarchy"

PRINCIPLE OF LOCALITY

- Temporal locality (locality in time): if an item is referenced, it will tend to be referenced again soon.
- Spatial locality (locality in space): if an item is referenced, items whose addresses are close by will tend to be referenced soon.

```
// Multiply the two matrices together
for ( ty = 0 ; ty < BLOCK_SIZE ; ty++ ){
    for ( tx = 0 ; tx < BLOCK_SIZE ; tx++ ){
        Csub = 0.0 ;
        for (k = 0; k < BLOCK_SIZE; ++k ){
            Asub = As[ty][k ] ;
            Bsub = Bs[k ][tx] ;
            Csub += Asub * Bsub ;
        }
        c = wB * BLOCK_SIZE * by + BLOCK_SIZE * bx;
        C[c + wB * ty + tx] += Csub;
    } // for tx ;
} // for ty
```

→ for-loop is temporal locality

→ array is spatial locality

Observation: temporal locality means that we don't put all program into memory whereas spatial locality means that we don't put all data into memory, hence we have "Memory Hierarchy"

Memory Hierarchy – Locality of Reference

- Analysis of programs indicates that many instructions in localized areas of a program are executed repeatedly during some period of time, while the others are accessed relatively less frequently.
 - These instructions may be the ones in a loop, nested loop or few procedures calling each other repeatedly.
 - This is called “locality of reference”.
- Temporal locality of reference:
 - Recently executed instruction is likely to be executed again very soon.
 - Whenever an instruction or data is needed for the first time, it should be brought into a cache. It will hopefully be used again repeatedly.

Memory Hierarchy – Locality of Reference

- Spatial locality of reference:

- Instructions with addresses close to a recently instruction are likely to be executed soon.
- Instead of fetching just one item from the main memory to the cache at a time, several items that have addresses adjacent to the item being fetched may be useful.
- The term “block” refers to a set of contiguous addresses locations of some size.

1. Which of the following statements correctly distinguishes **memory latency** from **memory bandwidth**?
 - A. Memory latency is the amount of data transferred per second, while memory bandwidth is the delay before data transfer begins.
 - B. Memory latency is the time taken to transfer a word of data to/from memory, while memory bandwidth is the amount of data transferred per second.
 - C. Memory latency and memory bandwidth both measure the speed of the CPU.
 - D. Memory latency measures storage capacity, while memory bandwidth measures memory speed.

2. Which principle states that if an item is referenced, it will tend to be referenced again soon?
 - A) Spatial locality
 - B) Block locality
 - C) Temporal locality
 - D) Cache locality

1. Which of the following statements correctly distinguishes **memory latency** from **memory bandwidth**?
- A. Memory latency is the amount of data transferred per second, while memory bandwidth is the delay before data transfer begins.
 - B. Memory latency is the time taken to transfer a word of data to/from memory, while memory bandwidth is the amount of data transferred per second.
 - C. Memory latency and memory bandwidth both measure the speed of the CPU.
 - D. Memory latency measures storage capacity, while memory bandwidth measures memory speed.

Answer: B

2. Which principle states that if an item is referenced, it will tend to be referenced again soon?
- A) Spatial locality
 - B) Block locality
 - C) Temporal locality
 - D) Cache locality

Answer: C

3. What is the primary purpose of cache memory?

- A) To permanently store the operating system.
- B) To make the main memory appear faster to the processor than it really is.
- C) To increase the overall physical size of the main memory.
- D) To replace the disk storage in a computer system.

4. In a typical memory hierarchy, which type of memory provides the fastest access time but has the smallest capacity?

- A) Disk storage
- B) CPU Registers
- C) L1 Cache
- D) Main Memory

3. What is the primary purpose of cache memory?

- A) To permanently store the operating system.
- B) To make the main memory appear faster to the processor than it really is.
- C) To increase the overall physical size of the main memory.
- D) To replace the disk storage in a computer system.

Answer: B

4. In a typical memory hierarchy, which type of memory provides the fastest access time but has the smallest capacity?

- A) Disk storage
- B) CPU Registers
- C) L1 Cache
- D) Main Memory

Answer: B



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Unit 3

Session - 3.2

Microprocessor & Computer Architecture (μ pCA)

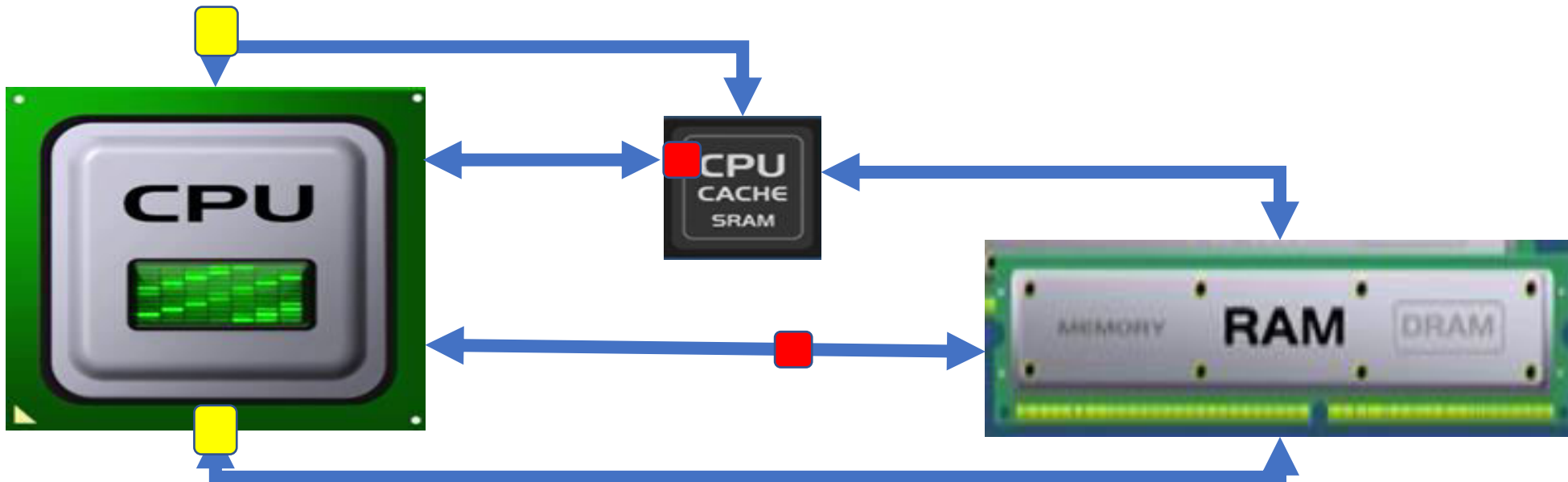


UE24CS251B

Cache Memory -Terminology

Cache Memory

- Cache memory is an architectural arrangement which makes the main memory appear faster to the processor than it really is.



Reference

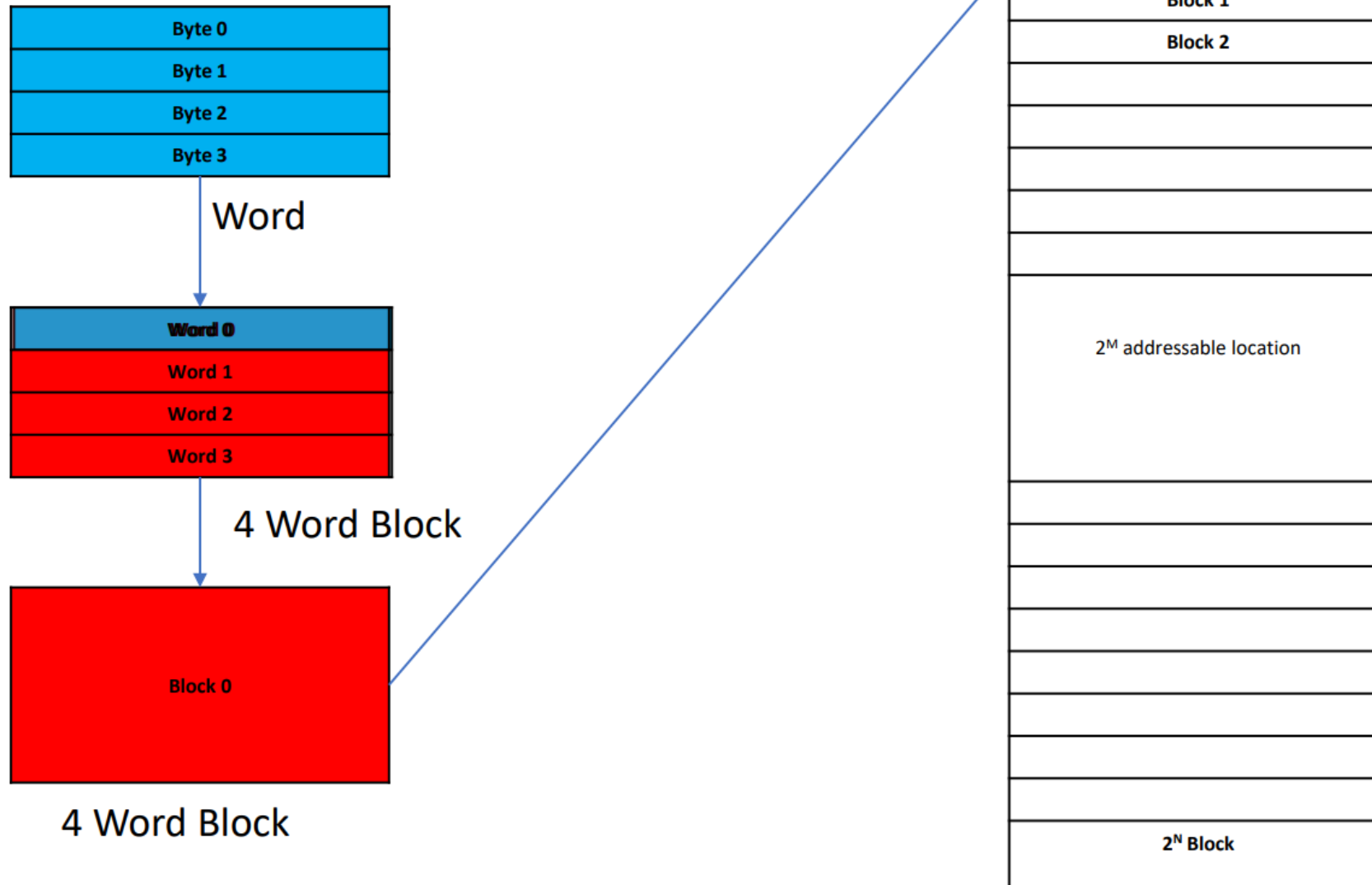
Data

Request

- Cache Lines vs Blocks
- Block Size
- Line Size
- # Cache lines $\ll$ # Blocks
- Block Transfer

- Cache Hit
 - Read Hit
 - Write Hit
- Cache Miss
 - Read Miss
 - Write Miss

The term “block” refers to a set of contiguous addresses locations of some size.



Cache Terminology – An Example

Main memory

- 16-bit address.
- 65536 (64 k) words.
- 4096 (4 K) Blocks
- Each block of 16 words.

Cache Memory

- consisting of 128 Lines of 16 words each.
- Total size of cache is 2048 (2K) words.
- How many mappings?
- $4096/128 = 32$ possible Blocks for one Line



128 Block Cache



Four Questions in Cache Design

Q1: Where can a block be placed in the cache?

- Block Placement

Q2: How is a block found if it is in the cache?

- Block Identification.

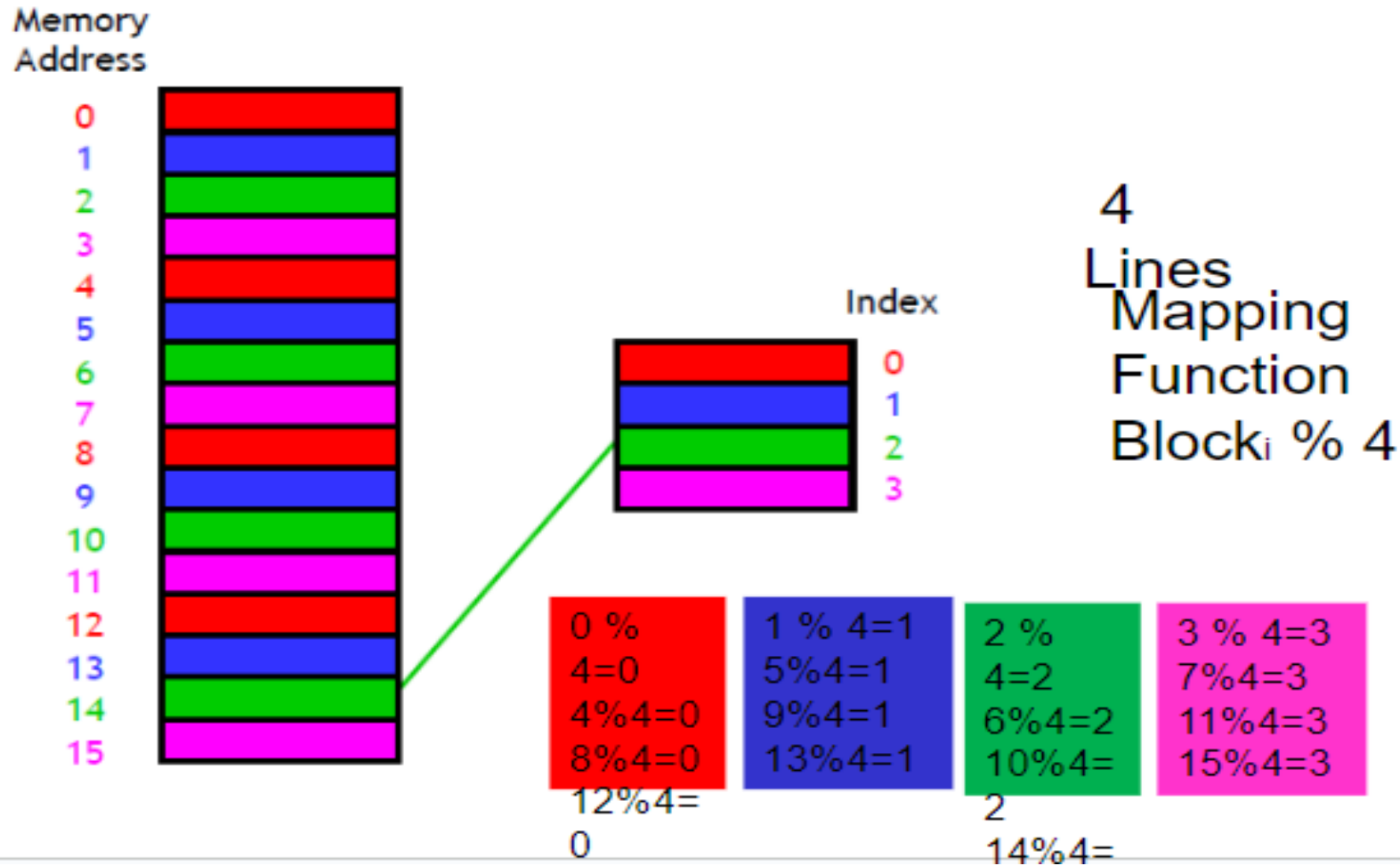
Q3: Which block should be replaced on a miss?

- Block Replacement.

Q4: What happens on a write ?

- Write Strategy.

Q1: Where can a block be placed in the cache?



Block no.	0	1	2	3	4	5	6	7
Cache								

Block no.

0	1	2	3	4	5	6	7

Block no.	0	1	2	3	4	5	6	7
Set 0								
Set 1								
Set 2								
Set 3								

[illegible]

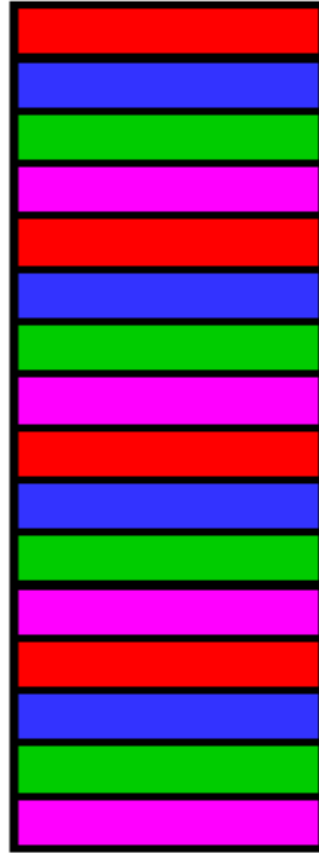
Figure B.2 This example cache has eight block frames and memory has 32 blocks. The three options for caches are shown left to right. In fully associative, block 12 from the lower level can go into any of the eight block frames of the cache. With direct mapped, block 12 can only be placed into block frame 4 ($12 \bmod 8$). Set associative, which has some of both features, allows the block to be placed anywhere in set 0 ($12 \bmod 4$). With two blocks per set, this means block 12 can be placed either in block 0 or in block 1 of the cache. Real caches contain thousands of block frames, and real memories contain millions of blocks. The set associative organization has four sets with two blocks per set, called *two-way set associative*. Assume that there is nothing in the cache and that the block address in question identifies lower-level block 12.

Q2: How is a block found if it is in the cache?

0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Memory
Address

0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111



Block address (tag and index) and block offset-
selects desired data from block

Index



0000
0100
1000
1100

0001
0101
1001
1101

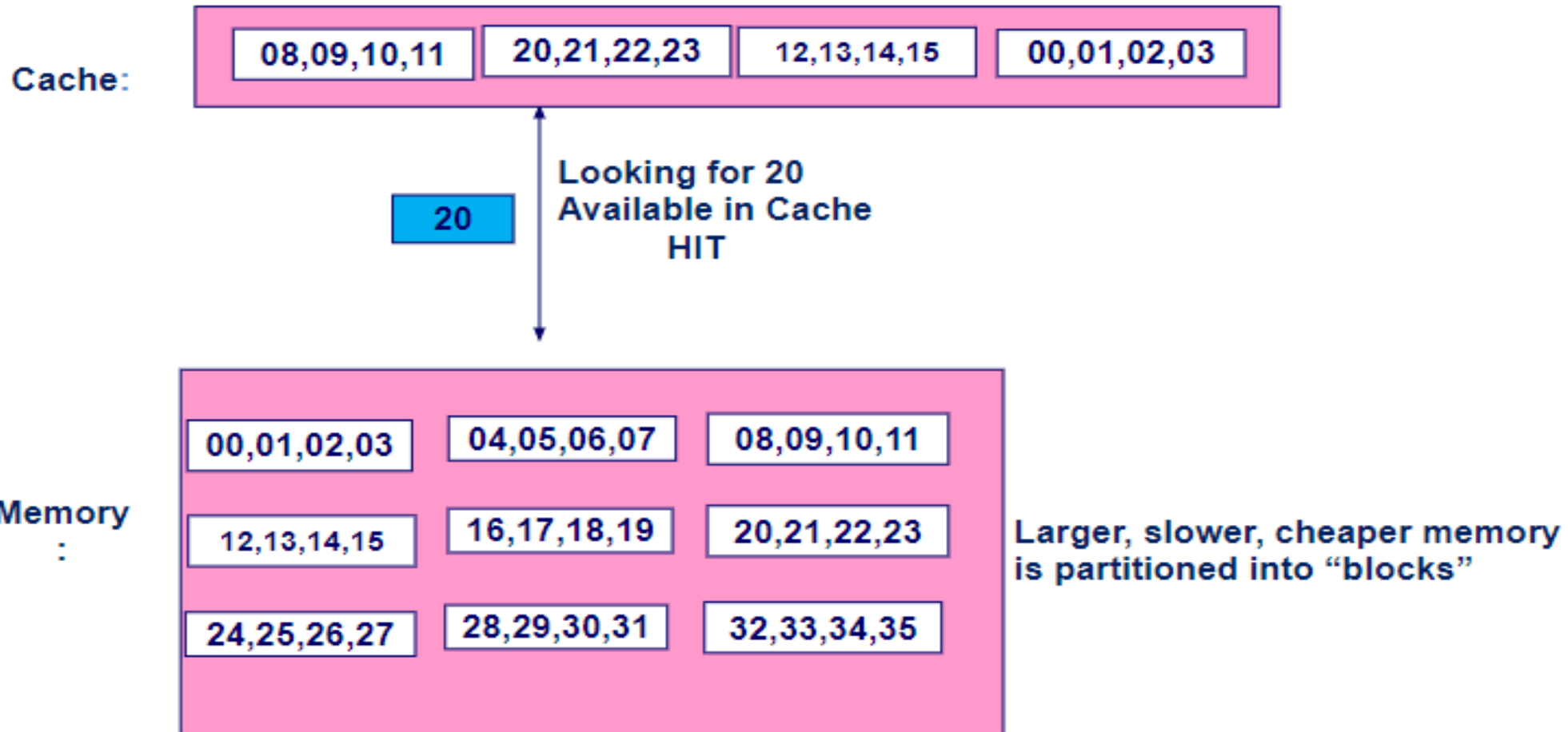
0010
0110
1010
1110

0011
0111
1011
1111

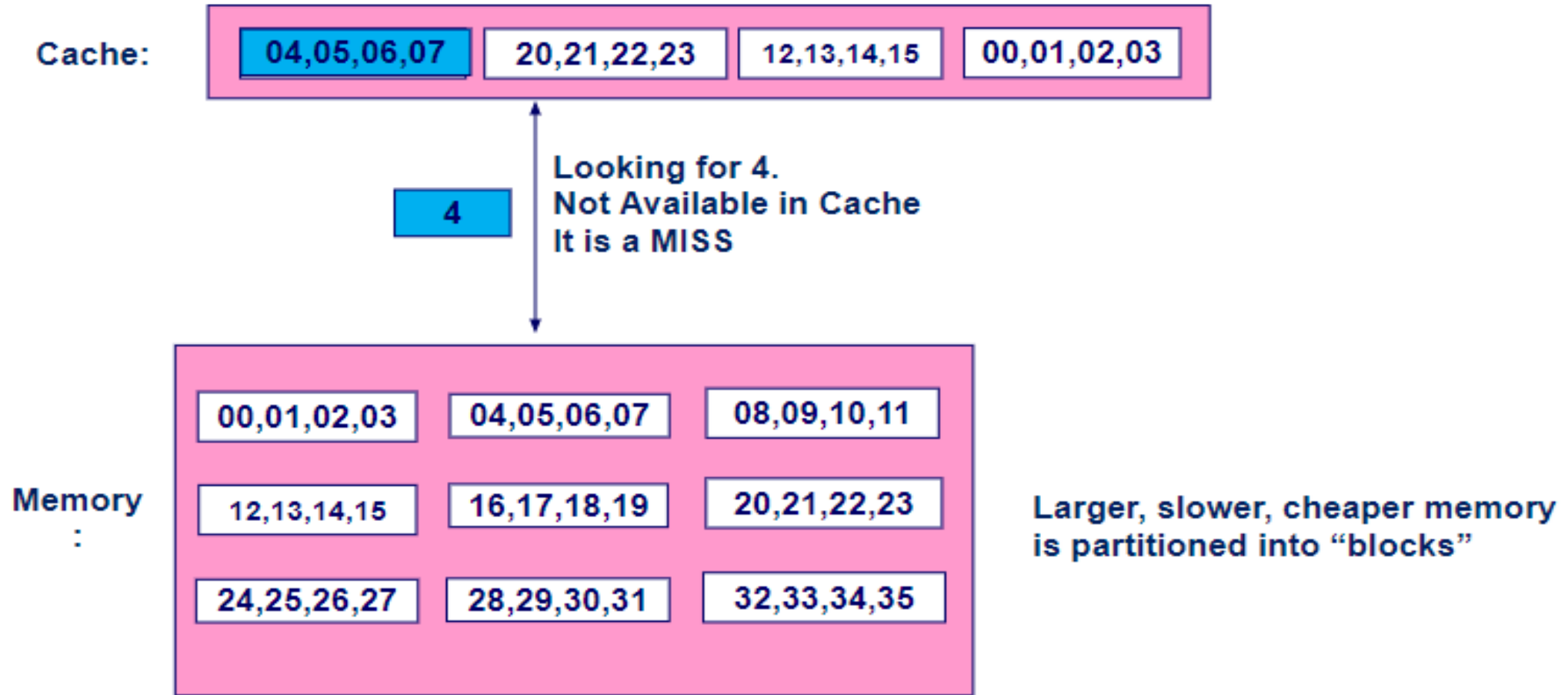
Q2: How is a block found if it is in the cache?



Read or Write HIT



Rear or Write MISS



HIT or MISS

0000	
0001	
0010	
0011	
0100	
0101	
0110	
0111	
1000	
1001	
1010	
1011	
1100	
1101	
1110	
1111	

Tag	Data
00	0 TH ELEMENT
11	13 TH ELEMENT
01	6 TH ELEMENT
01	7 TH ELEMENT

If Processor is looking for 9th Element

Address Generated is: 1001

Index the cache with value 01

Compare with the tag: 10

Tag is not matched: it is MISS

HIT or MISS

0000	
0001	
0010	
0011	
0100	
0101	
0110	
0111	
1000	
1001	
1010	
1011	
1100	
1101	
1110	
1111	

Tag	Data
00	0 TH ELEMENT
11	13 TH ELEMENT
01	6 TH ELEMENT
01	7 TH ELEMENT

If Processor is looking for 13th Element
 Address Generated is: 1101
 Index the cache with value 01
 Compare with the tag: 11
 Tag is matched: it is HIT

1. Which of the following is NOT one of the four main questions in cache design?

- A) Where can a block be placed in the cache?
- B) How is a block found if it is in the cache?
- C) Which block should be replaced on a miss?
- D) How is the clock speed of the cache increased?

2. When comparing cache lines to main memory blocks, what is the typical relationship?

- A) The number of cache lines is much greater than the number of memory blocks.
- B) The number of cache lines is equal to the number of memory blocks.
- C) The number of cache lines is much less than the number of memory blocks.
- D) Cache lines and memory blocks are entirely unrelated.

1. Which of the following is NOT one of the four main questions in cache design?

- A) Where can a block be placed in the cache?
- B) How is a block found if it is in the cache?
- C) Which block should be replaced on a miss?
- D) How is the clock speed of the cache increased?

Answer: D

2. When comparing cache lines to main memory blocks, what is the typical relationship?

- A) The number of cache lines is much greater than the number of memory blocks.
- B) The number of cache lines is equal to the number of memory blocks.
- C) The number of cache lines is much less than the number of memory blocks.
- D) Cache lines and memory blocks are entirely unrelated.

Answer: C

3. How does the cache controller determine if a specific memory block is present in the cache (a Cache Hit)?

- A) By comparing the tag field of the generated address with the tag in the cache index.
- B) By reading the entire main memory to find a match.
- C) By checking the dirty bit of the cache line.
- D) By looking at the size of the block offset.

4. In the context of cache mapping, what happens when the processor looks for an element and the tag does NOT match the tag in the cache?

- A) A Write Hit
- B) A Read Hit
- C) A Cache Miss
- D) A Block Transfer

3. How does the cache controller determine if a specific memory block is present in the cache (a Cache Hit)?

- A) By comparing the tag field of the generated address with the tag in the cache index.
- B) By reading the entire main memory to find a match.
- C) By checking the dirty bit of the cache line.
- D) By looking at the size of the block offset.

Answer: A

4. In the context of cache mapping, what happens when the processor looks for an element and the tag does NOT match the tag in the cache?

- A) A Write Hit
- B) A Read Hit
- C) A Cache Miss
- D) A Block Transfer

Answer: C



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Unit 3

Session - 3.3

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

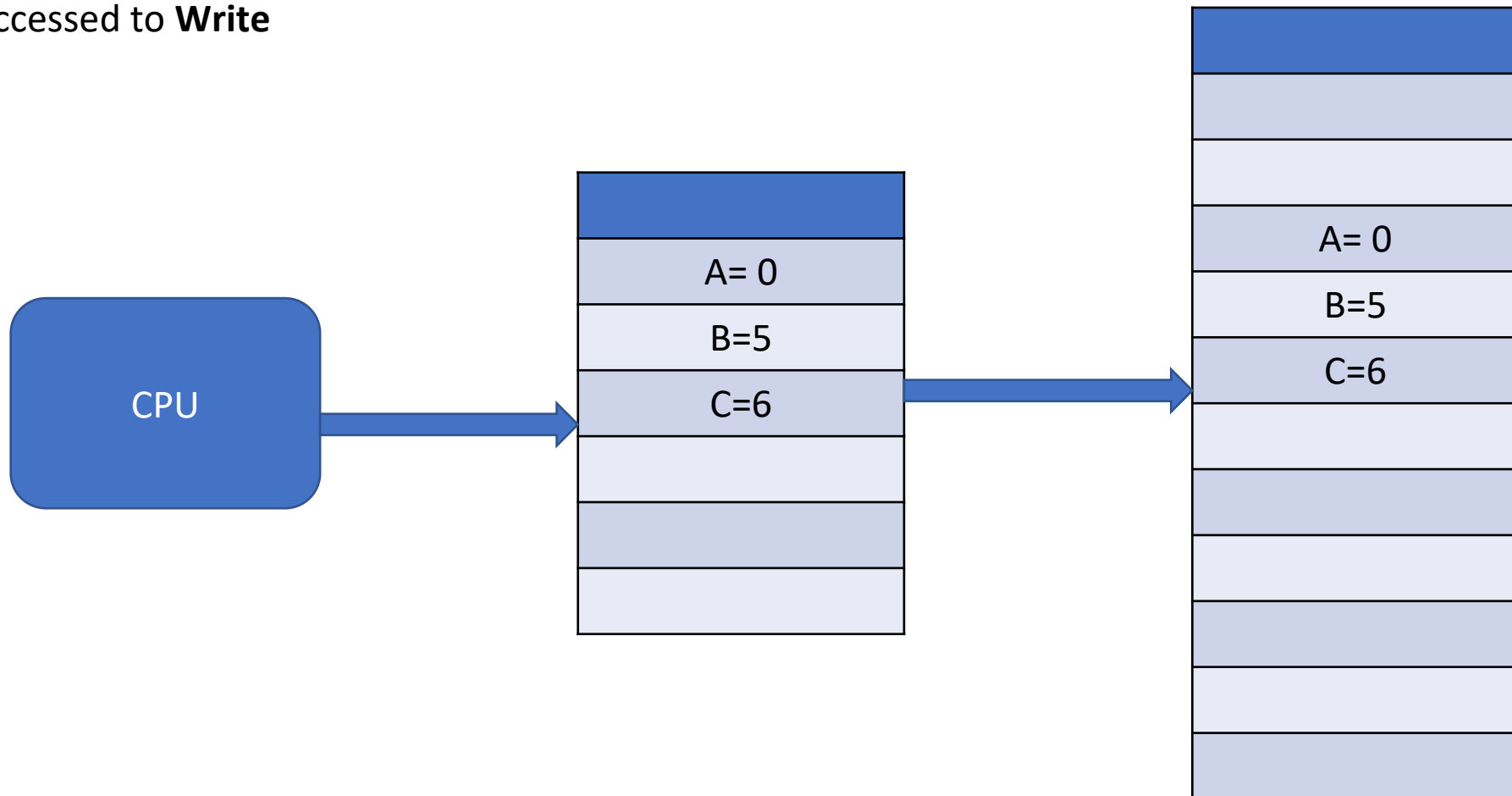
Cache Memory - Read and Write Policy

Read vs Write

Consider the statement $A=B+C$

B and C are accessed to compute the result A: Only **Read**

A is accessed to **Write**



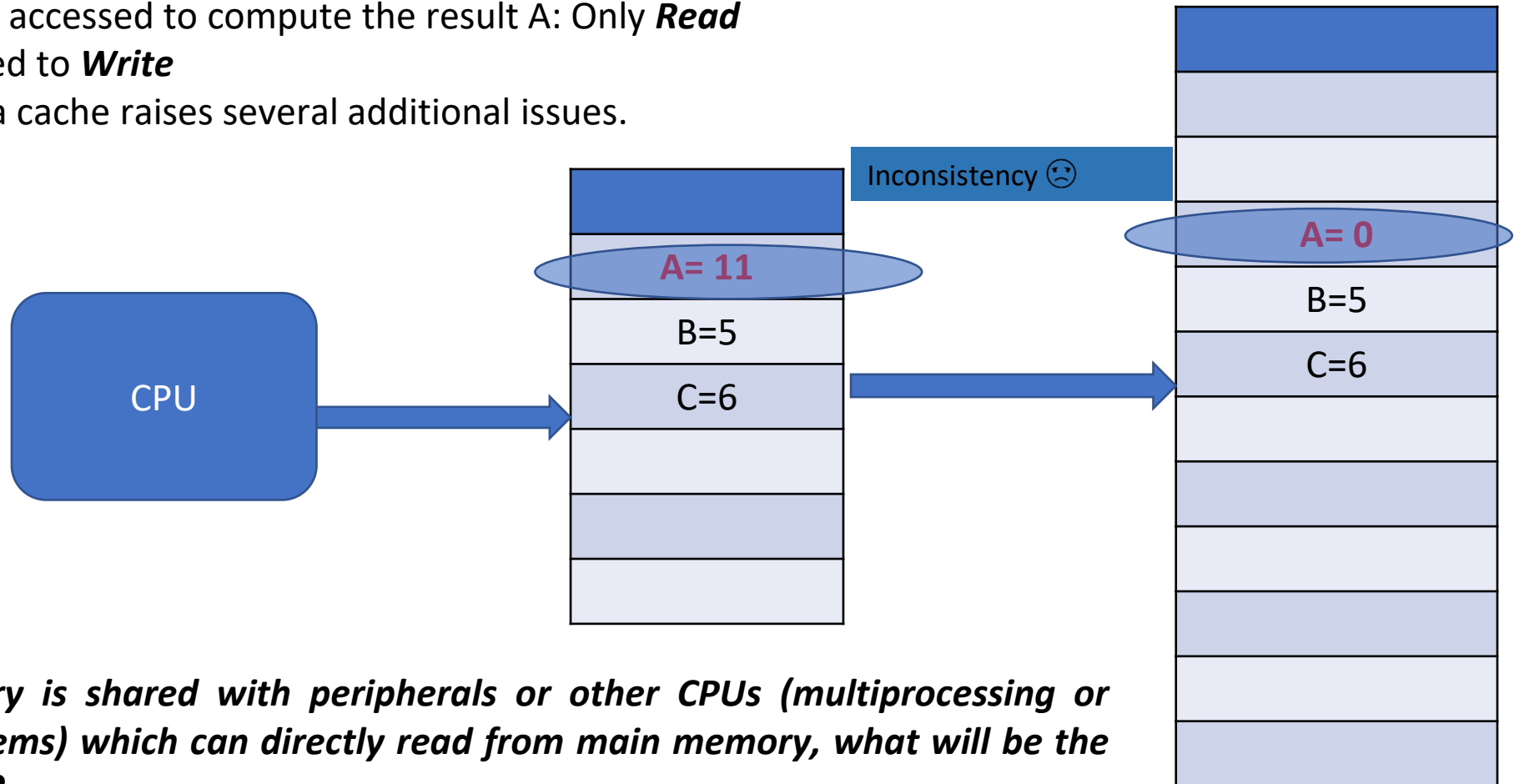
Read vs Write

Consider the statement $A=B+C$

B and **C** are accessed to compute the result **A**: Only **Read**

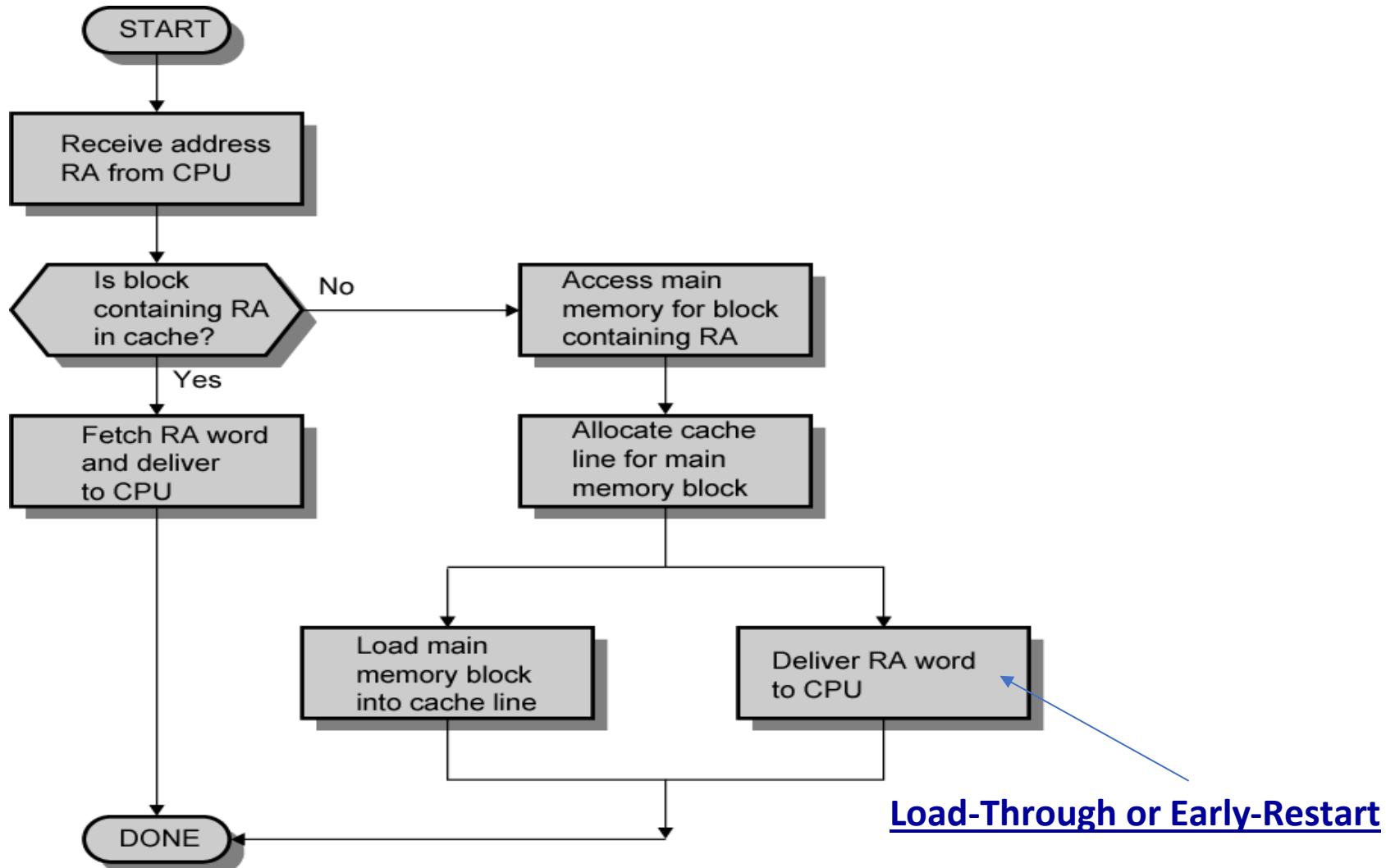
A is accessed to **Write**

Writing to a cache raises several additional issues.



If main memory is shared with peripherals or other CPUs (multiprocessing or multi-core systems) which can directly read from main memory, what will be the value returned?

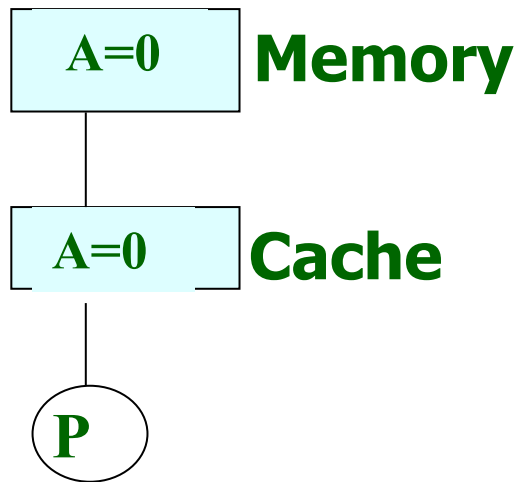
Read: Hit or Miss



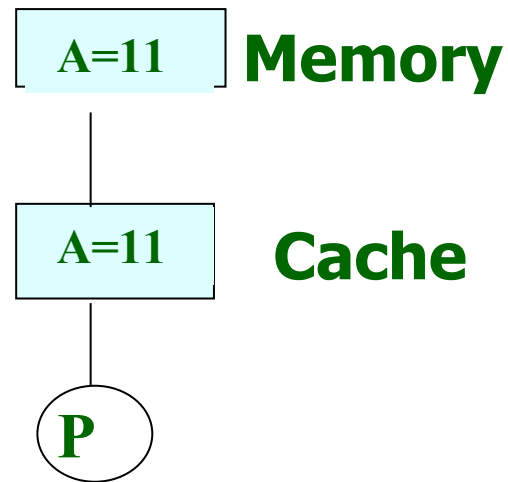
Load-through or early-restart

- Block of words containing this requested word is transferred from the memory.
- After the block is transferred, the desired word is forwarded to the processor.
- The desired word may also be forwarded to the processor as soon as it is transferred
without waiting for the entire block to be transferred.
- This is called **load-through or early-restart.**

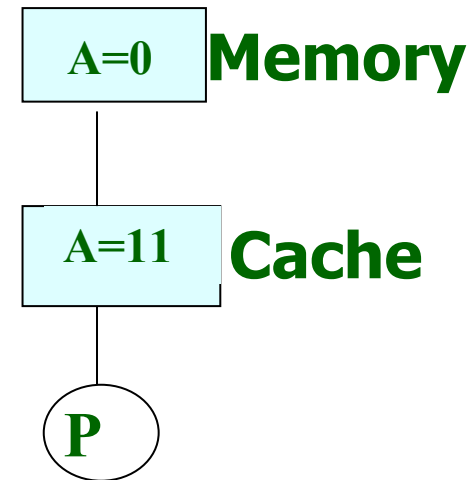
Write On Hit



Before



Write through



Write back

Write Through on Hit

Valid Bit	TAG	Data
1	10100	A=0
1	10100	B=5
1	10100	C=6

- Each Line in the cache has Valid Bit.
- **Valid Bit** indicates that no other program has changed the Data.

10100.....1010..0000	A= 0
10100.....1010..0001	B=5
10100.....1010..0010	C=6
00010.....1010..0000	P=100

Updated Simultaneously

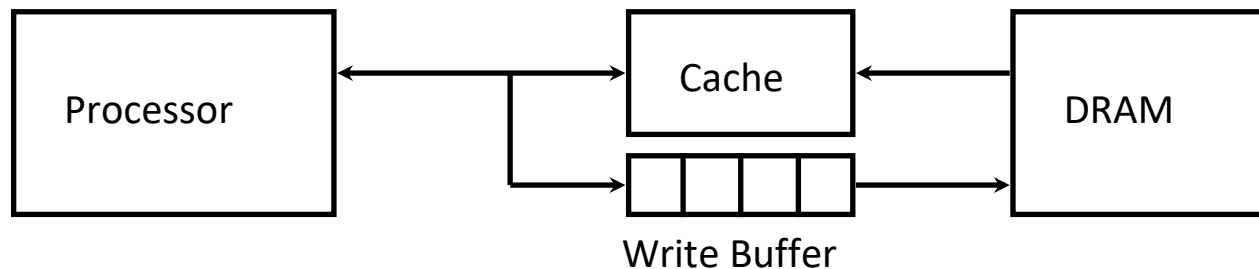
Valid Bit	TAG	Data
1	10100	A=11
1	10100	B=5
1	10100	C=6

10100.....1010..0000	A= 11
10100.....1010..0001	B=5
10100.....1010..0010	C=6
00010.....1010..0000	P=100

- Avoid Inconsistency.
- Purpose of having Cache is dishonored.
- Some design, buffer the update and continue executing other instruction
- **Cache Coherence Problem**

Write Buffer for Write Through on Hit

- Some design, update the buffer and continue executing other instruction



When the processor must wait for writes to complete during write-through, the processor is said to **write stall**. A common optimization to reduce write stalls is a write buffer, which allows the processor to continue as soon as the data are written to the buffer, thereby overlapping processor execution with memory updating.

Write Back on Hit: With Validation

Before Write A

Valid Bit	Dirty Bit	TAG	Data
1	0	10100	A=0
1	0	10100	B=5
1	0	10100	C=6

- Each Line in the cache has Valid Bit and Dirty Bit
- **Valid Bit** indicates that no other program has changed the Data
- **Dirty Bit** indicates that Data is written or not written

10100.....1010..0000	A= 0
10100.....1010..0001	B=5
10100.....1010..0010	C=6
00010.....1010..0000	P=100

Write Back on Hit: With Validation

After Write A

Valid Bit	Dirty Bit	TAG	DAta
1	1	10100	A=11
1	0	10100	B=5
1	0	10100	C=6

10100.....1010..0000	A= 0
10100.....1010..0001	B=5
10100.....1010..0010	C=6
00010.....1010..0000	P=100

Write Back on Hit: With Validation

To Replace A by P

Valid Bit	Dirty Bit	TAG	DAta
1	0	00010	P=100

- Step 1: Update A in memory to 11
- Step 2: Replace A with P
- Step3: Set Dirty Bit to 0

10100.....1010..0000	A= 11
10100.....1010..0001	B=5
10100.....1010..0010	C=6
00010.....1010..0000	P=100

Difference between

1.Purpose:

1. **Valid Bit:** Indicates whether the data in a cache entry is valid or meaningful. It helps the CPU determine if the data in the cache can be used for a memory access or if a cache miss needs to occur.
2. **Dirty Bit:** Indicates whether the data in a cache entry has been modified (is "dirty") and needs to be written back to the main memory before the cache entry can be replaced. It is particularly relevant in write-back caching strategies.

2.Function:

1. **Valid Bit:** Primarily used to prevent using stale or invalid data from the cache. When the valid bit is not set, **it signals that the data in the cache entry is not reliable**, and a cache miss is typically initiated.
2. **Dirty Bit:** Primarily used in write-back caching to track modifications made to cached data. When a write occurs to a location already in the cache, the dirty bit is set, **indicating that the corresponding data in the main memory is outdated and needs to be updated.**

3.Impact on Cache Operations:

1. **Valid Bit:** Affects cache hits and misses. A cache hit occurs only if the valid bit is set, ensuring that only valid data is used from the cache.
2. **Dirty Bit:** Affects cache replacement policies. When a cache line with a set dirty bit is replaced, it triggers a write-back operation to update the main memory with the modified data.

1.Initial State:

1. Cache Entry 1: V = 0 (invalid), D = 0 (clean)
2. Cache Entry 2: V = 0 (invalid), D = 0 (clean)

2.Memory Access:

1. The CPU attempts to read data from memory location X.
2. The cache is checked:
 1. Cache Entry 1 (V = 0): Cache miss, data for location X needs to be fetched from main memory.
 2. Cache Entry 2 (V = 0): Cache miss, data for location X needs to be fetched from main memory.

3.Data Fetch:

1. Data for location X is fetched from main memory and stored in Cache Entry 1.
2. Cache Entry 1 is updated: V = 1 (valid), D = 0 (clean).

4.Memory Write:

1. The CPU writes new data to location X.
2. Cache Entry 1 is updated: V = 1 (valid), D = 1 (dirty).

5.Cache Replacement:

1. A new memory access for location Y results in Cache Entry 2 being selected for replacement.
2. Cache Entry 2 is replaced, and its contents (if modified) need to be written back to main memory.
3. Since D = 0 (clean) for Cache Entry 2, no write-back is necessary.

In this example:

•**The valid bit (V) determined whether the data in a cache entry was valid or if a cache miss was needed.**

•**The dirty bit (D) tracked whether the data in a cache entry had been modified and needed to be written back to main memory during cache replacement.**

Advantages of write through

Easier to implement

Cache is always clean

Simplifies coherency for multiprocessor

Write back cache- speed- since need not update Main memory

Saves power for small embedded applications

Write back- reduces traffic

Write through-keeps cache and main memory consistent

Write Back on Miss: With Validation

On Miss : Two Possibilities

1: Bring the Block on the Line (**Write Allocate**)

2: Update the Block in the Memory Directly (**Write No Allocate**)

Write Allocate On Miss

P need to be written?

Valid Bit	Dirty Bit	TAG	Data
1	1	10100	A=11
1	0	10100	B=5
1	0	10100	C=6

Replace P by A

10100.....1010..0000	A= 11
10100.....1010..0001	B=5
10100.....1010..0010	C=6
00010.....1010..0000	P=100

Write Allocate On Miss

Update P to 200
P need to be written?

Valid Bit	Dirty Bit	TAG	DAta
1	1	00010	P=200
1	0	10100	B=5
1	0	10100	C=6

Replace P by A

10100.....1010..0000	A= 11
10100.....1010..0001	B=5
10100.....1010..0010	C=6
00010.....1010..0000	P=100

Write No Allocate On Miss

P need to be written?

Valid Bit	Dirty Bit	TAG	DAta
1	1	10100	A=11
1	0	10100	B=5
1	0	10100	C=6

Update P directly in Memory

10100.....1010..0000	A= 11
10100.....1010..0001	B=5
10100.....1010..0010	C=6
00010.....1010..0000	P=200

Assume a fully associative write-back cache with many cache entries that starts empty. Below is a sequence of five memory operations (the address is in square brackets):

Write Mem[100];

Write Mem[100];

Read Mem[200];

Write Mem[200];

Write Mem[100].

What are the number of hits and misses when using no-write allocate versus write allocate?

Answer For no-write allocate, the address 100 is not in the cache, and there is no allocation on write, so the first two writes will result in misses. Address 200 is also not in the cache, so the read is also a miss. The subsequent write to address 200 is a hit. The last write to 100 is still a miss. The result for no-write allocate is four misses and one hit.

For write allocate, the first accesses to 100 and 200 are misses, and the rest are hits since 100 and 200 are both found in the cache. Thus, the result for write allocate is two misses and three hits.

Write No Allocate On Miss

This is good when data is written but not immediately used again, in which case there's no point to load it into the cache yet.

```
for(i=0;i<10;i++)  
    a[i]=i*2;
```

Where ith Location is referred to write only once.

Think About it!!!

Writing Cache Friendly Code

- Two major rules:
- Repeated references to data are good (**temporal locality**)
- Stride-1 reference patterns are good (**spatial locality**)
- Example: **4-byte words, 4-word cache blocks**

```
int sum_array_rows(int a[M][N])
{
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = **1/4 = 25%**

```
int sum_array_cols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = **100%**

1. What is "load-through" or "early-restart" in cache read operations?

- A) Writing data to memory and cache simultaneously.
- B) Forwarding the desired word to the processor as soon as it is transferred without waiting for the entire block.
- C) Restarting the CPU whenever a cache miss occurs.
- D) Flushing the cache completely before loading new data.

2. In a write-back cache, what is the purpose of the "Dirty Bit"?

- A) To indicate that no other program has changed the data.
- B) To show that the cache line is empty.
- C) To indicate whether the data in the cache has been written to (modified) or not.
- D) To determine if the block is a read hit or read miss.

1. What is "load-through" or "early-restart" in cache read operations?

- A) Writing data to memory and cache simultaneously.
- B) Forwarding the desired word to the processor as soon as it is transferred without waiting for the entire block.
- C) Restarting the CPU whenever a cache miss occurs.
- D) Flushing the cache completely before loading new data.

Answer: B

2. In a write-back cache, what is the purpose of the "Dirty Bit"?

- A) To indicate that no other program has changed the data.
- B) To show that the cache line is empty.
- C) To indicate whether the data in the cache has been written to (modified) or not.
- D) To determine if the block is a read hit or read miss.

Answer: C

3. Which write policy updates the cache block but does NOT immediately update the main memory?

- A) Write Through
- B) Write Back
- C) Write Allocate
- D) Write No Allocate

4. When a write miss occurs, which strategy brings the block into the cache line before updating it?

- A) Write No Allocate
- B) Write Through
- C) Early-Restart
- D) Write Allocate

3. Which write policy updates the cache block but does NOT immediately update the main memory?

- A) Write Through
- B) Write Back
- C) Write Allocate
- D) Write No Allocate

Answer: B

4. When a write miss occurs, which strategy brings the block into the cache line before updating it?

- A) Write No Allocate
- B) Write Through
- C) Early-Restart
- D) Write Allocate

Answer: D



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Unit 3

Session - 3.4

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Cache Memory – Mapping Techniques

Cache Terminology – An Example

Main memory

- 16-bit address.
- 65536 (64 k) words.
- 4096 (4 K) Blocks
- Each block of 16 words.

Cache Memory

- consisting of 128 Lines of 16 words each.
- Total size of cache is 2048 (2K) words.
- $4096/128 = 32$ possible Blocks for one Line

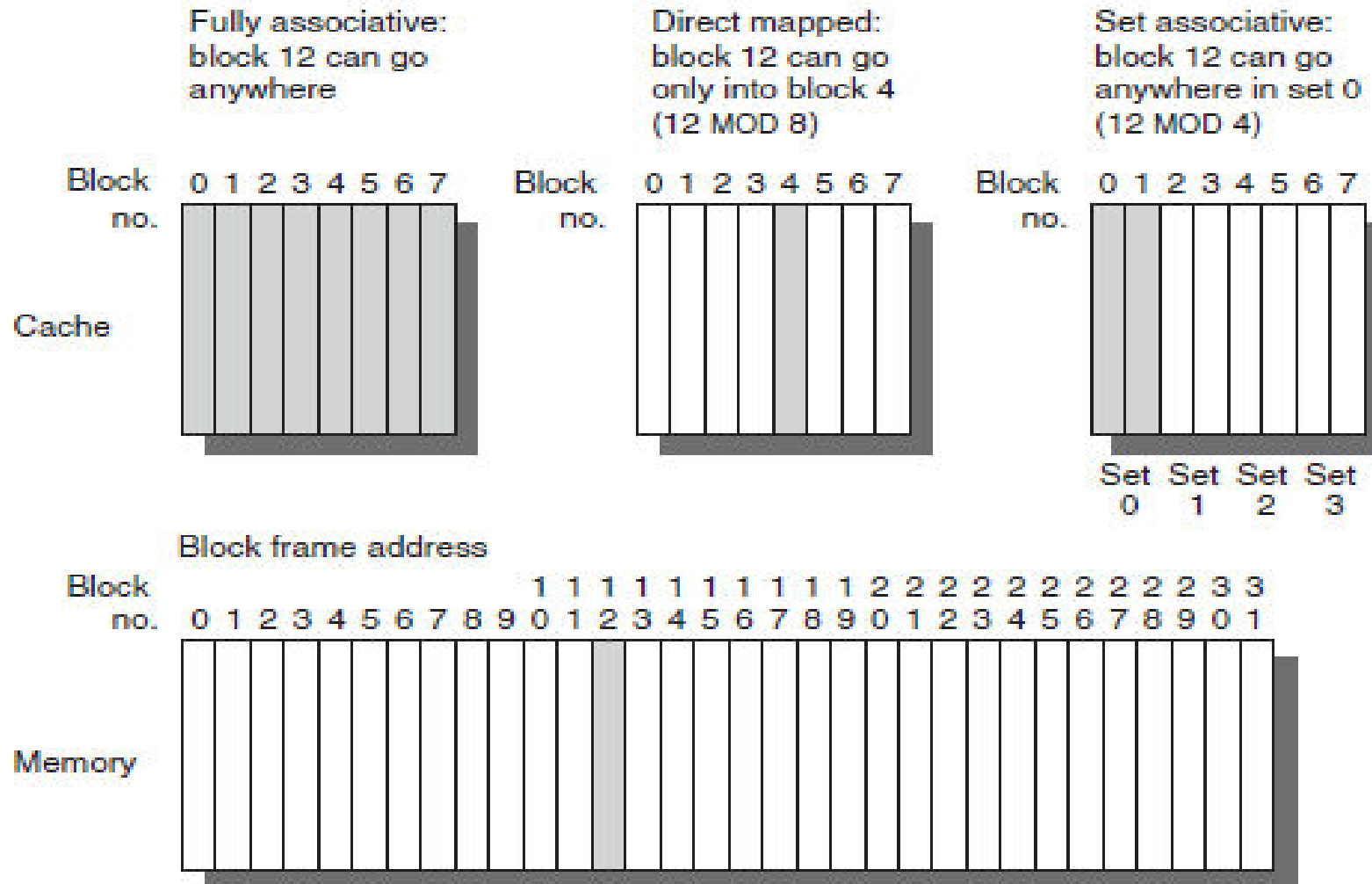


Cache Memory – Mapping Functions

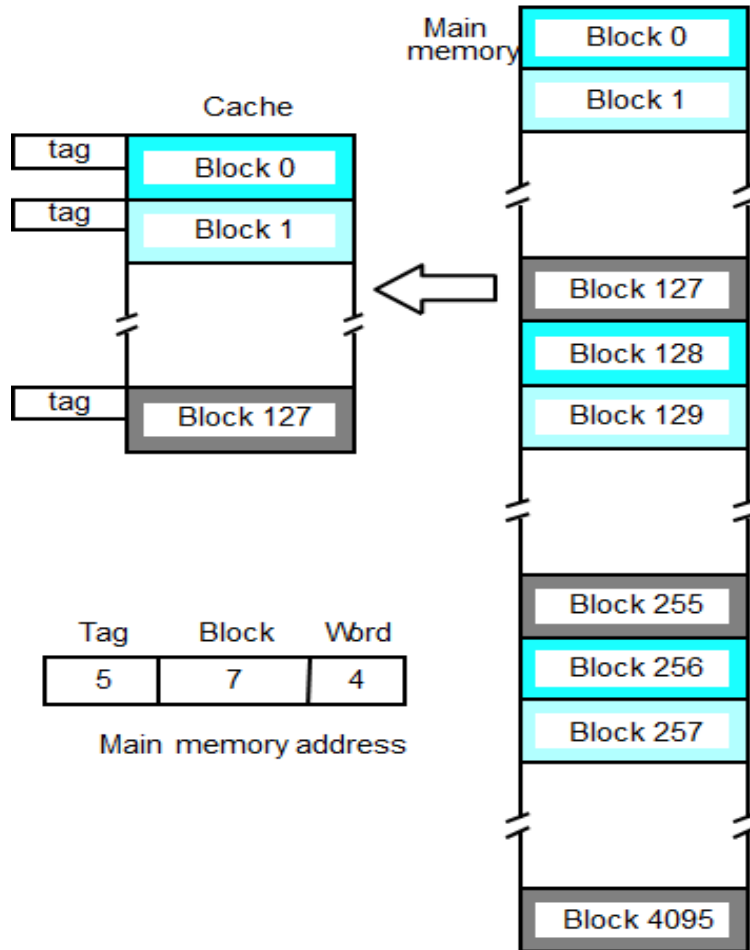
Three mapping functions:

- **Direct mapping**
- **Associative mapping**
- **Set-associative mapping.**

Mapping Functions

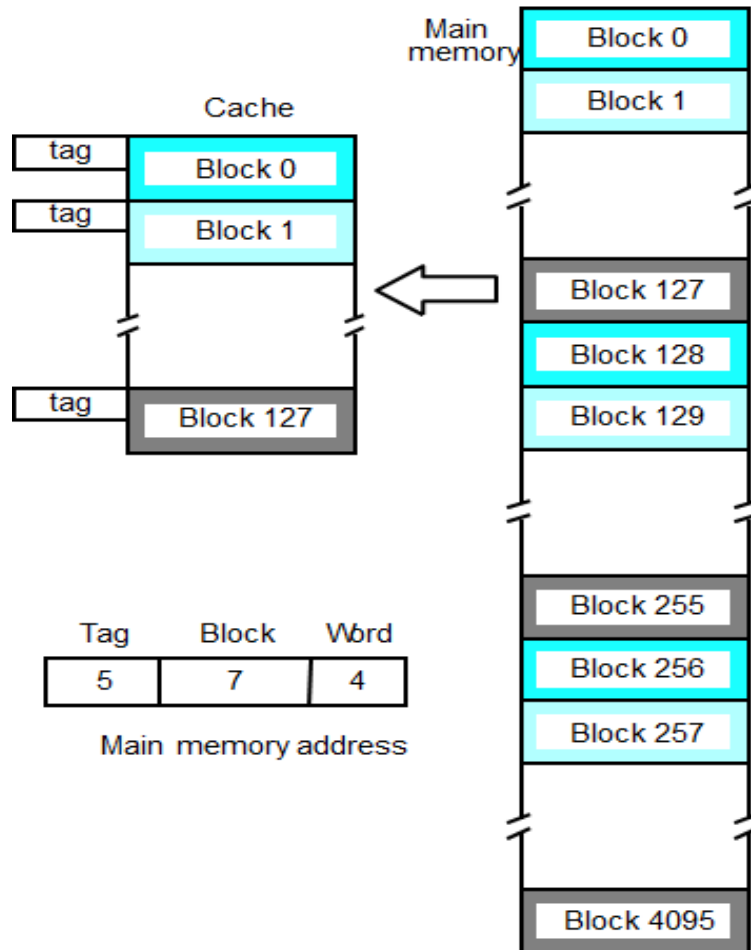


Direct Mapping



- Block j of the main memory maps to j modulo 128 of the cache. 0 maps to 0, 129 maps to 1.
- More than one memory block is mapped onto the same position in the cache.
- May lead to contention for cache blocks even if the cache is not full.
- Resolve the contention by allowing new block to replace the old block, leading to a trivial replacement algorithm.

Direct Mapping



Memory address is divided into three fields:

- Low order 4 bits determine one of the 16 words in a block.
- When a new block is brought into the cache, the next 7 bits determine which cache block this new block is placed in.
- High order 5 bits determine which of the possible 32 blocks is currently present in the cache. These are tag bits.
- Simple to implement but not very flexible.

Direct Mapping: Block Placement

Direct Mapping: Block Identification HIT

16 bit Address: 00111 0101010 1010 = 15018
 7th Block of 32 Blocks 42nd Line 10th Word

16 bit Address: 00101 0101010 1010 = 10922
 5th Block of 32 Blocks 42nd Line 10th Word

Cache Hit or Miss

If Processor is looking for data in the address:
0011101010101010

Step 1: Look at Line Number 42 i.e 0101010

Step 2: Compare the TAG field.

2.1 If it Matches, it is a *HIT* Fetch the 10th Word

2.2 If not Matched it is a *MISS*.

Replace current Line with the block which contain the word which the Processor is looking for.

Line No	Tag	Word	Data
Line 0			
Line 1			
Line 42 0101010 0		0000	
		0001	
		0010	
		0011	
		0100	
		0101	
		0110	
		0111	
		1000	
		1001	
	00111	1010	
		1011	
		1100	
		1101	
		1110	
		1111	
Line 126			
Line 127			

Direct Mapping: Block Identification MISS

16 bit Address: **00111** **0101010** **1010** = 15018
 7th Block of 32 Blocks 42nd Line 10th Word

16 bit Address: **00101** **0101010** **1010** = 10922
 5th Block of 32 Blocks 42nd Line 10th Word

Cache Hit or Miss

If Processor is looking for data in the address:
0011101010101010

Step 1: Look at Line Number **42** i.e **0101010**

Step 2: Compare the TAG field.

2.1 If it Matches, it is a *HIT* Fetch the 10th Word

2.2 If not Matched it is a *MISS*.

Replace current Line with the block which contain the word which is Processor is looking for.

Line No	Tag	Word	Data
Line 0			
Line 1			
Line 42 0101010		0000	
		0001	
		0010	
		0011	
		0100	
		0101	
		0110	
		0111	
		1000	
		1001	
	00101	1010	
		1011	
		1100	
		1101	
		1110	
		1111	
Line 126			
Line 127			

cache memory system working similarly:

- **Library shelves** → **Cache Lines**
- **Books** → **Blocks of Data**
- **Book labels** → **Tag Field**
- **Page number in a book** → **Word in a Block**

How Direct Mapping Works?

1. Suppose you want to **read a specific page** of a book.
2. You are given a **book number (Block)** and a **page number (Word in the block)**.
3. The librarian **maps the book to a specific shelf slot (Cache Line)** using a simple rule.
4. If the **correct book is already in that slot**, you can read the page immediately (**Cache Hit**).
5. If a **different book is in that slot**, you need to **replace it with the correct book** before you can read the page (**Cache Miss**).

Address = **15018**

Step 1: Find Main Memory Block

Block size = **16 words**

$$15018/16 = 938$$

So the data is in **Main Memory Block 938**.

Block Number=Address/Block Size

Step 2: Map Block to Cache Line

Cache has **128 lines**

$$938 \bmod 128 = 42$$

So **Block 938 maps to Cache Line 42**.

This is the **mapping from main memory → cache**.

Cache Line=Block Number mod Cache Lines

Processor requests address **15018**

Cache checks:

1.Line 42

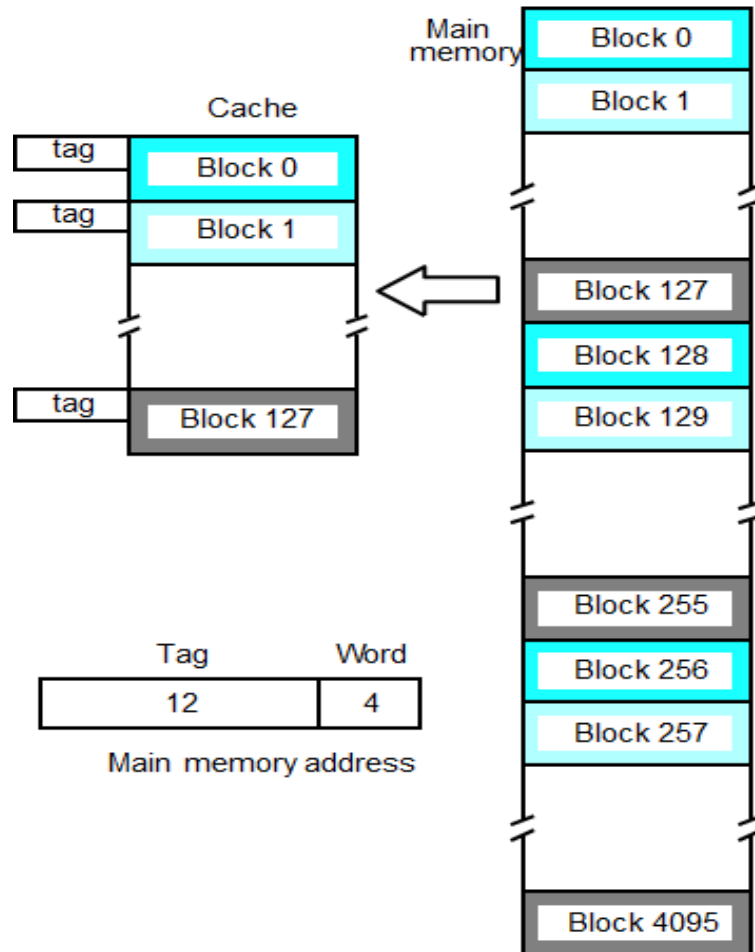
2.Compare Tag

If tag matches → **Cache HIT**

If tag does not match → **Cache MISS** and block **938** is loaded into **line 42**.

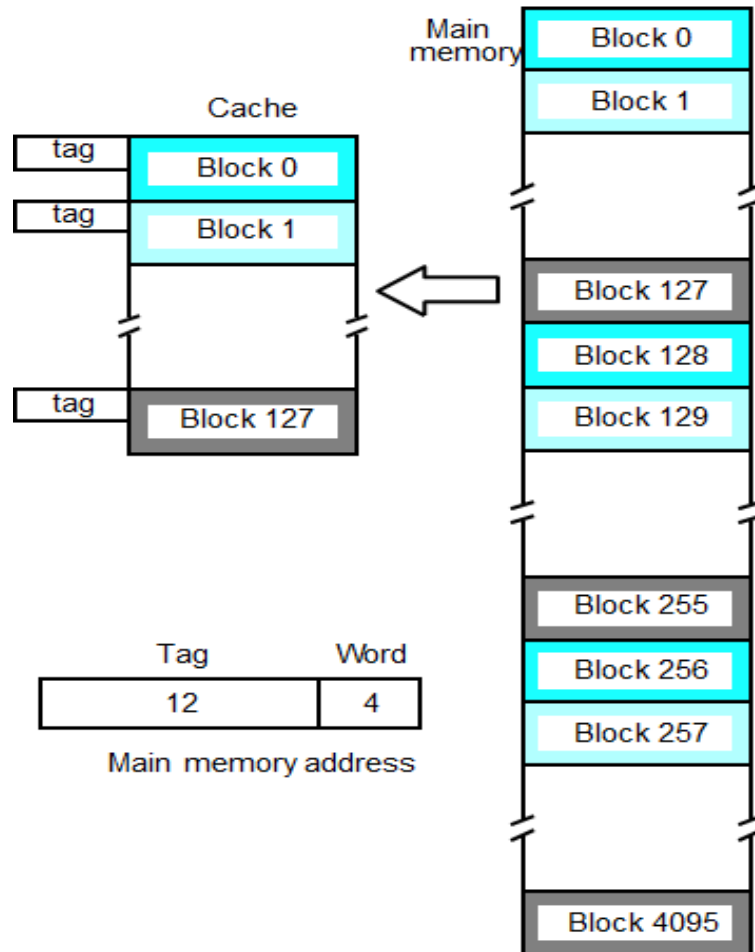
Main Memory Blocks	Cache Lines
-----	-----
Block 0	Line 0
Block 1	Line 1
...	
Block 128	Line 0
Block 129	Line 1
...	
Block 938	-----> Line 42

Associative Mapping



- Main memory block can be placed into any cache position.
- Memory address is divided into two fields:
 - Low order 4 bits identify the word within a block.
 - High order 12 bits or tag bits identify a memory block when it is resident in the cache.
- Flexible, and uses cache space efficiently.

Associative Mapping



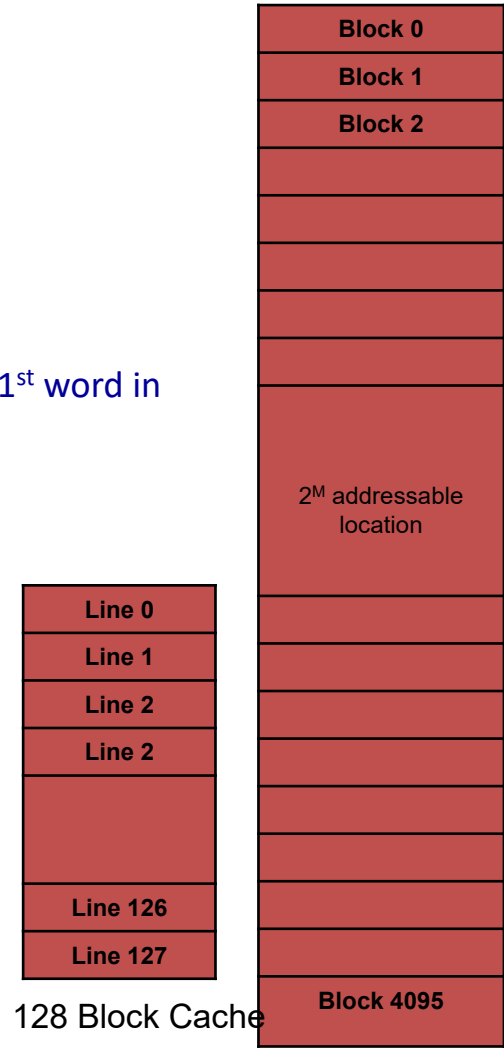
- Replacement algorithms can be used to replace an existing block in the cache when the cache is full.
- Cost is higher than direct-mapped cache because of the need to search all 128 patterns to determine whether a given block is in the cache.

Full Associative Mapping

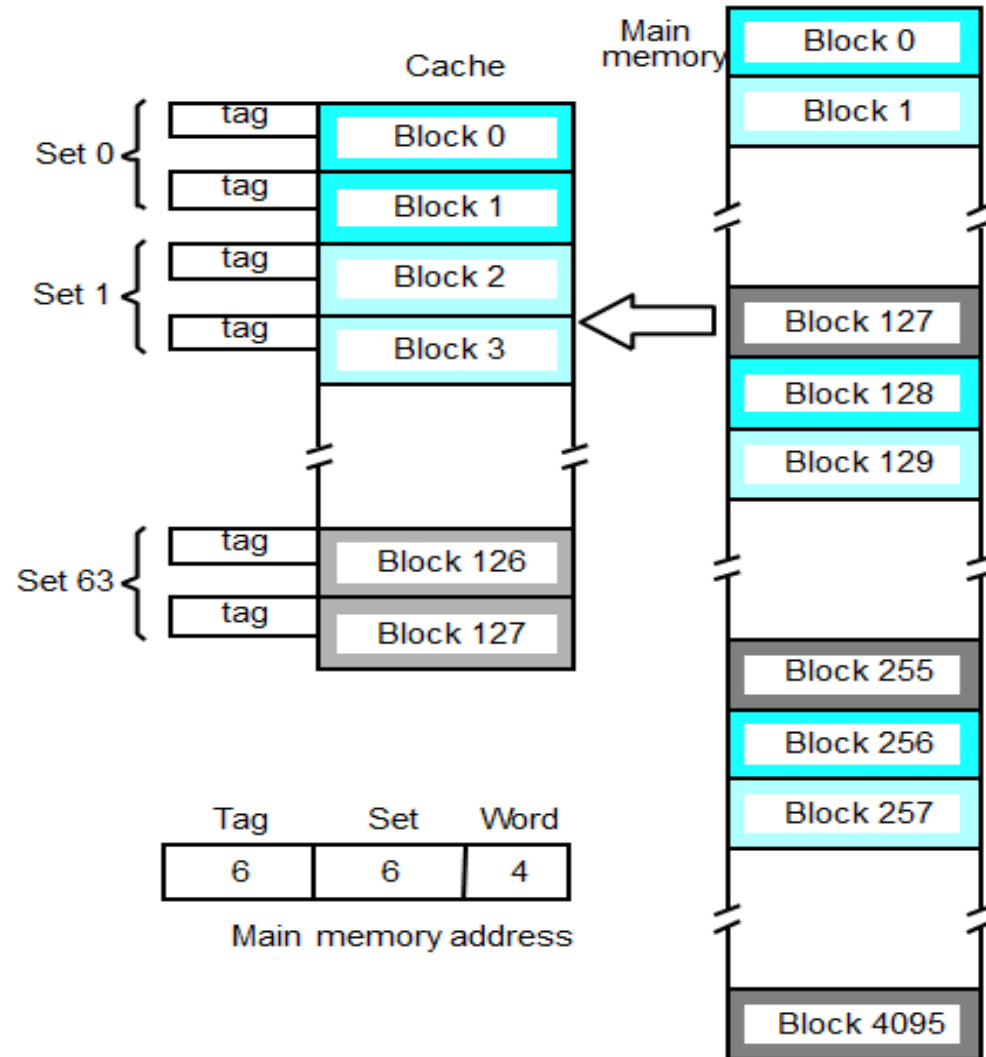
- Block Placement:**
 - Main Memory block can be placed into any cache line
- Block Identification:**
 - Memory Address is divided into Word, and Tag part
 - word is used to identify placement
 - Tag is used to identify validity of the word
 - Ex: How to differentiate between 1st word in Block 0 and 1st word in Block 256
 - Need to search all 128 lines for suitable match
- Block Replacement:**
 - Replacement algorithms are used to replace an existing block in the cache when cache is full
 - Flexible and use cache space efficiently
 - Does avoid thrashing*



$2^4 = 16$ words $2^{12} = 4096$ blocks can be on any Line

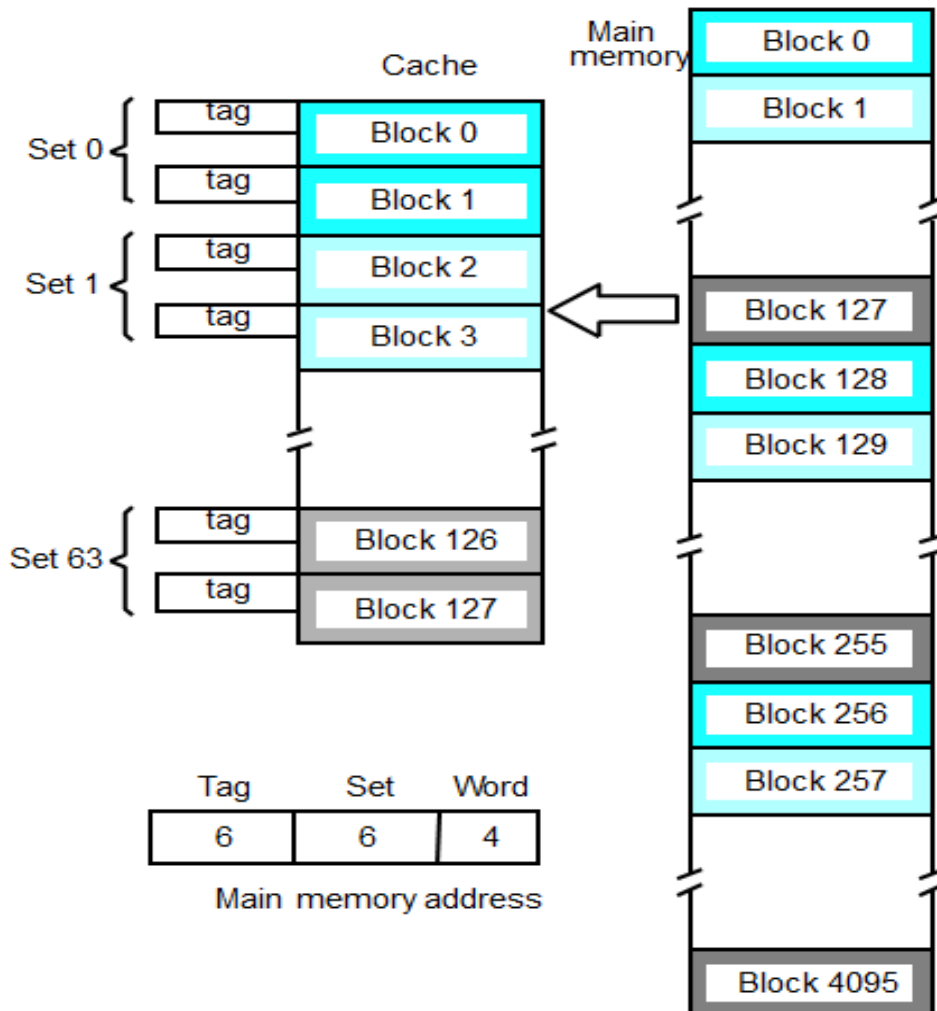


Set Associative Mapping



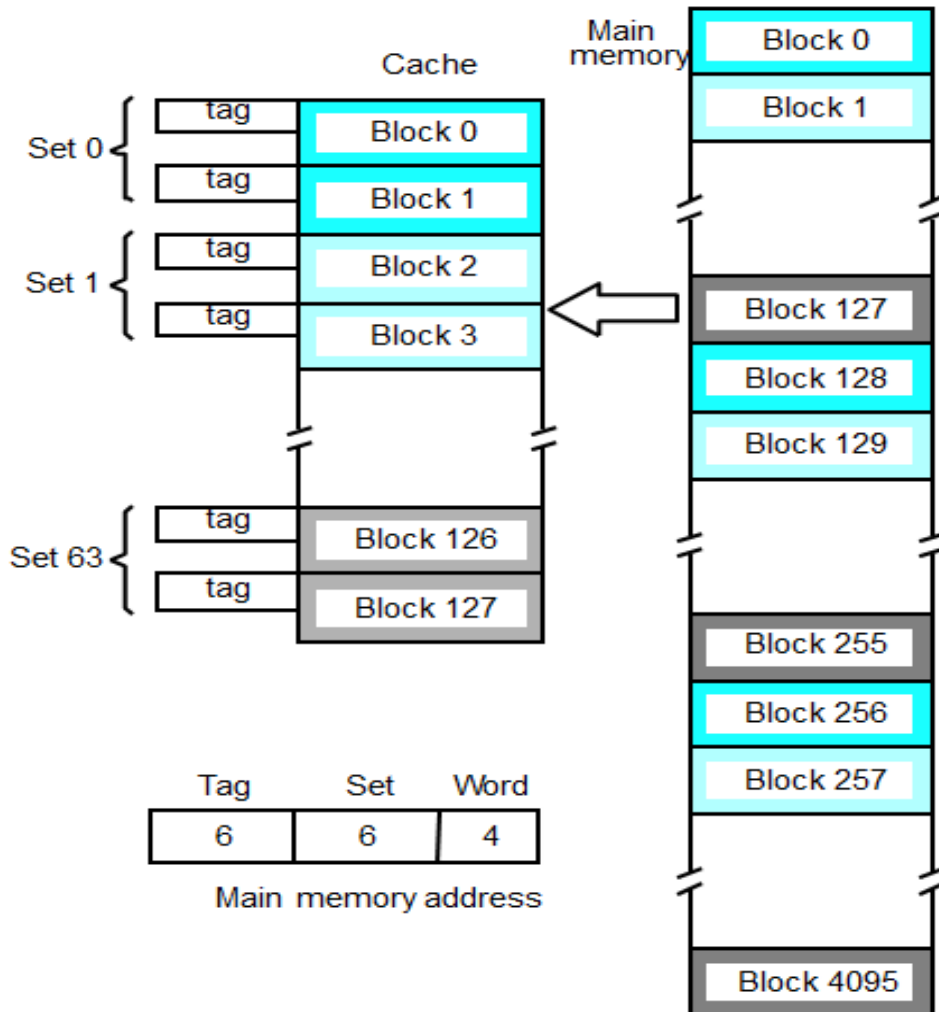
- Blocks of cache are grouped into sets.
- Mapping function allows a block of the main memory to reside in any block of a specific set.
- Divide the cache into 64 sets, with two blocks per set. Memory block 0, 64, 128 etc. map to block 0, and they can occupy either of the two positions.

Set Associative Mapping



- Memory address is divided into three fields:
 - 6 bit field determines the set number.
 - High order 6 bit fields are compared to the tag fields of the two blocks in a set.
- Set-associative mapping combination of direct and associative mapping.

Set Associative Mapping



- Number of blocks per set is a design parameter.
 - One extreme is to have all the blocks in one set, requiring no set bits (fully associative mapping).
 - Other extreme is to have one block per set, is the same as direct mapping.

2 Way Set Associative Mapping: Block Placement

- Memory address is divided into three fields:
 - 6 bit field determines the set number.
 - High order 6 bit fields are compared to the tag fields of the two blocks in a set.
- Set-associative mapping combination of direct and associative mapping.

16 bit Address

Line 0	Set 0
Line 1	
Line 2	Set 1
Line 3	
Line 126	Set 63
Line 127	

Tag 6 bits	Set 6 bits	Word 4 bits
---------------	---------------	----------------

$2^4 = 16$ words

$2^6 = 64$ Sets

$2^6 =$ Which block out of 64 Blocks



Problems on direct mapping

<https://www.gatevidyalay.com/direct-mapping-cache-practice-problems/>

Problems on fully associative mapping

<https://www.gatevidyalay.com/fully-associative-cache-practice-problems/>

Problems on set associative mapping

<https://www.gatevidyalay.com/set-associative-mapping-practice-problems/>

Exercise 1

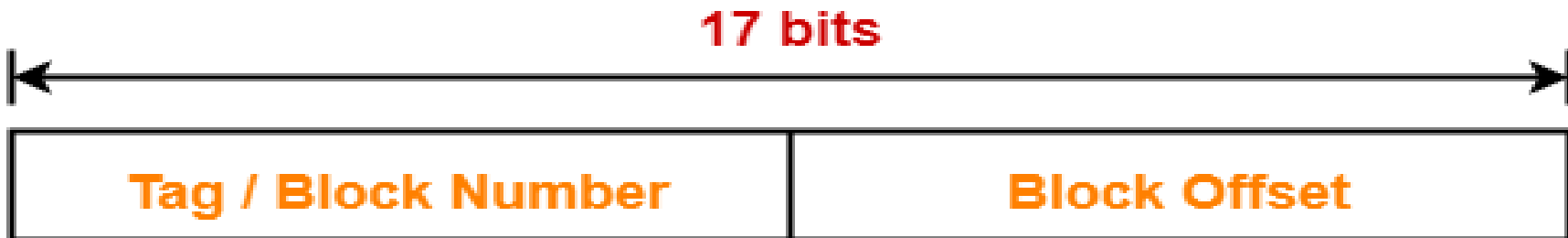
Consider a fully associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find number of tag bits.

Exercise 1

Consider a fully associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find number of tag bits.

Given-

- Cache memory size = 16 KB
 - Block size = Set size = Line size = 256 bytes
 - Main memory size = 128 KB = 2^{17} bytes
- Thus, Number of bits in physical address = 17 bits



Block size

= 256 bytes

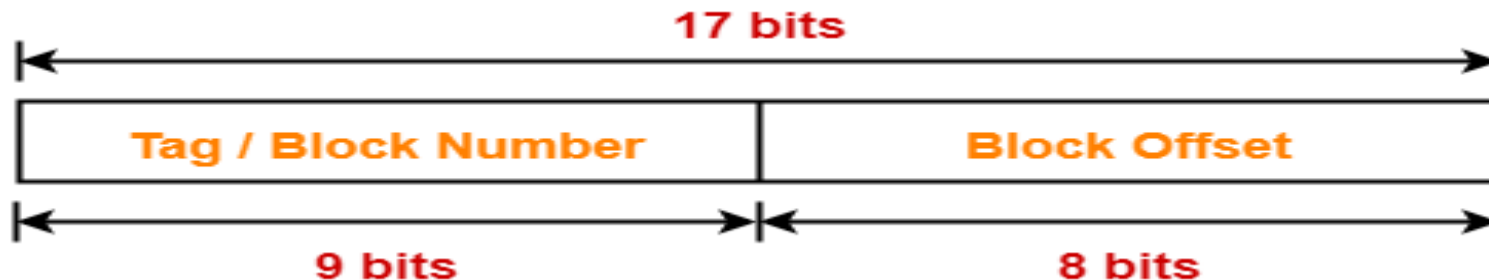
= 2^8 bytes

Thus, Number of bits in block offset = 8 bits

Number of bits in tag = Num of bits in PA – Num of bits in block offset

= 17 bits – 8 bits = 9 bits

Thus, Number of bits in tag = 9 bits



Exercise 2

Consider a 2-way set associative cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find Number of bits in tag

Exercise 2

Consider a 2-way set associative cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find Number of bits in tag

Given-

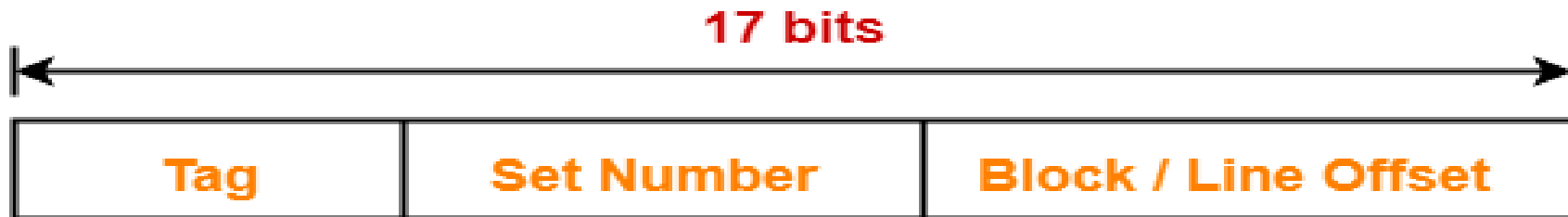
Cache memory size = 16 KB

of blocks /set or line = 2

of bytes per block = 256 bytes.

Main memory size = 128 KB = 2^{17}

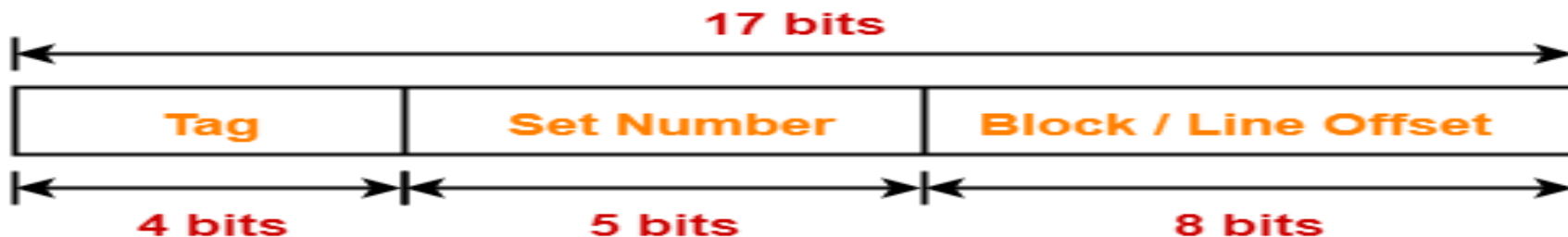
Size of Memory Address: 17 bits



Exercise 2



TAG	Set Number	Byte offset
$17 - (8 + 5) = 4 \text{ bits}$	Total number of blocks in cache memory $= \text{Cache size} / \text{Block size}$ $= 16\text{kb} / 256$ $= 2^{14} \text{ bytes} / 2^8 \text{ bytes}$ $= 2^6 \text{ Blocks or Lines}$ Number of Sets $= 2^6 \text{ Blocks or Lines} / 2$ $= 2^5$ i.e, 5 bits.	Block Size = 256 $= 2^8$ Block Offset = 8 bits



Consider a 8-way set associative mapped cache of size 512 KB with block size 1 KB. There are 7 bits in the tag. Find-

1. Size of main memory

Given-

- Set size = 8
- Cache memory size = 512 KB
- Block size = Frame size = Line size = 1 KB
- Number of bits in tag = 7 bits

We have,

Block size

= 1 KB

= 2^{10} bytes

Thus, Number of bits in block offset = 10 bits

Number of Lines in Cache-

Total number of lines in cache

= Cache size / Line size

= 512 KB / 1 KB

= 512 lines

Thus, Number of lines in cache = 512 lines

Number of Sets in Cache-

Total number of sets in cache

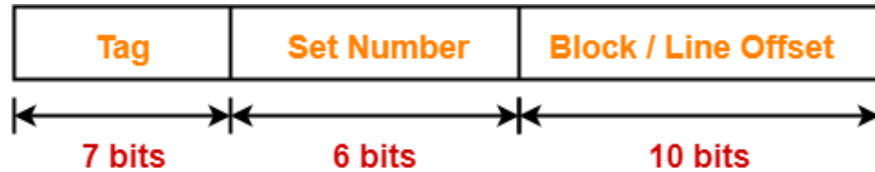
= Total number of lines in cache / Set size

= 512 / 8

= 64 sets

= 2^6 sets

Thus, Number of bits in set number = 6 bits



Number of Bits in Physical Address-

Number of bits in physical address
= Number of bits in tag + Number of bits in set number + Number of bits in block offset
= 7 bits + 6 bits + 10 bits
= 23 bits
Thus, Number of bits in physical address = 23 bits

Size of Main Memory-

We have,
Number of bits in physical address = 23 bits
Thus, Size of main memory
= 2^{23} bytes
= 8 MB

Exercise 3

A computer system uses 16-bit memory addresses. It has a 2K-byte cache organized in a 2-way set associative manner with 64 bytes per cache block. Assume that the size of each memory word is 1 byte.

- (a) Calculate the number of bits in each of the Tag, Block, and Word fields of the memory address.
- (b) When a program is executed, the processor reads data sequentially from the following word addresses: **128, 144, 2176, 2180, 128, 2176**

All the above addresses are shown in decimal values. Assume that the cache is initially empty.

For each of the above addresses, indicate whether the cache access will result in a hit or a miss.

Exercise 3 –Contd..

Block size = 64 bytes = 2^6 bytes

Therefore, **Number of bits in the *Word* field = 6**

Cache size = 2K-byte = 2^{11} bytes

Number of

cache blocks = Cache size / (Block size x Number of blocks per set)
 $= 2^{11} / (2^6 \times 2) = 2^4$

Therefore, **Number of bits in the *set* field = 4**

Total number of address bits = 16

Therefore, **Number of bits in the *Tag* field = $16 - (6+4) = 6$**

For a given 16-bit address, the 6 most significant bits, represent the *Tag*, the next 4 bits represent the *Block*, and the 6 least significant bits represent the *Word*.

Exercise 3 –Contd..

Address = (128)₁₀ = (0000000010000000)₂

Address = (144)₁₀ = (0000000100100000)₂

Address = (2176)₁₀ = (0000100010000000)₂

Address = (2180)₁₀ = (0000100010000100)₂

Exercise 3 –Contd..

Access # 1:

Address = (128)₁₀ = (0000000010000000)₂

For this address, *Tag* = 000000, *Set* = 0010, *Word* = 000000

Since the cache is empty before this access, this will be a cache **miss**

After this access, **Tag field for the first block in set 0010 is set to 000000**

Access # 2:

Address = (144)₁₀ = (0000000010010000)₂

For this address, *Tag* = 000000, *Set* = 0010, *Word* = 010000

The tag field for this address matches the tag field for the first block in set 0010.

Therefore, this access will be a cache **hit**.

Exercise 3 –Contd..

Access # 3:

Address = $(2176)_{10} = (0000100010000000)_2$

For this address, $Tag = 000010$, $Set = 0010$, $Word = 000000$

The tag field for this address does not match the tag field for the first block in set 0010.

The **second block in set 0010** is empty. Therefore, this access will be a cache **miss**.
After this access, **Tag field for the second block in set 0010 is set to 000010**

Access # 4:

Address = $(2180)_{10} = (0000100010000100)_2$

For this address, $Tag = 000010$, $Set = 0010$, $Word = 000100$

The tag field for this address matches the tag field for the second block in set **0010**.
Therefore, this access will be a cache **hit**.

Exercise 3 –Contd..

Access # 5:

Address = $(128)_{10} = (0000000010000000)_2$

For this address, $Tag = 000000$, $Set = 0010$, $Word = 000000$

The tag field for this address matches the tag field for the **first block in set 0010**.

Therefore, this access will be a cache **hit**.

Access # 6:

Address = $(2176)_{10} = (0000100010000000)_2$

For this address, $Tag = 000010$, $Set = 0010$, $Word = 000000$

The tag field for this address matches the tag field for the **second block in set 0010**.

Therefore, this access will be a cache **hit**.

Exercise 3 –Contd..

$$\begin{aligned}\text{Cache hit rate} &= \text{Number of hits} / \text{Number of accesses} \\ &= 4/6 \\ &= 0.666\end{aligned}$$

Compare with direct mapping

$$\begin{aligned}\text{Cache hit rate} &= \text{Number of hits} / \text{Number of accesses} \\ &= 2/6 \\ &= 0.333\end{aligned}$$



Exercise 6

A computer system uses 16-bit memory addresses. It has a 2K-byte cache organized in a direct-mapped manner with 64 bytes per cache block. Assume that the size of each memory word is 1 byte.

- (a) Calculate the number of bits in each of the Tag, Block, and Word fields of the memory address.
- (b) When a program is executed, the processor reads data sequentially from the following word addresses: **128, 144, 2176, 2180, 128, 2176**

All the above addresses are shown in decimal values. Assume that the cache is initially empty.

For each of the above addresses, indicate whether the cache access will result in a hit or a miss.



Exercise 6

Block size = 64 bytes = 2^6 bytes

Therefore, **Number of bits in the *Word* field = 6**

Cache size = 2K-byte = 2^{11} bytes

Number of cache blocks = Cache size / Block size = $2^{11}/2^6 = 2^5$

Therefore, **Number of bits in the *Block* field = 5**

Total number of address bits = 16

Therefore, **Number of bits in the *Tag* field = 16 - 6 - 5 = 5**

For a given 16-bit address, the 5 most significant bits, represent the *Tag*, the next 5 bits represent the *Block*, and the 6 least significant bits represent the *Word*.



Exercise 6

Address = $(128)_{10} = (0000000010000000)_2$

Address = $(144)_{10} = (0000000010010000)_2$

Address = $(2176)_{10} = (0000100010000000)_2$

Address = $(2180)_{10} = (0000100010000100)_2$



Exercise 6

Access # 1: The cache is initially empty. Therefore, all the cache blocks are invalid

Address = $(128)_{10} = (0000000010000000)_2$

For this address, $Tag = 00000$, $Block = 00010$, $Word = 000000$

Since the cache is empty before this access, this will be a cache **miss**

After this access, **Tag field for cache block 00010 is set to 00000**

Access # 2: Address = $(144)_{10} = (0000000010010000)_2$

For this address, $Tag = 00000$, $Block = 00010$, $Word = 010000$

Since tag field for cache block 00010 is 00000 before this access, this will be a cache **hit** (because address tag = block tag)



Exercise 6

Access # 3: Address = $(2176)_{10} = (0000100010000000)_2$

For this address, $Tag = 00001$, $Block = 00010$, $Word = 000000$

Since tag field for cache block 00010 is 00000 before this access, this will be a cache **miss** (address tag $\neq$ block tag)

After this access, **Tag field for cache block 00010 is set to 00001**

Access # 4:

Address = $(2180)_{10} = (0000100010000100)_2$

For this address, $Tag = 00001$, $Block = 00010$, $Word = 000100$

Since tag field for cache block 00010 is 00001 before this access, this will be a cache **hit** (address tag = block tag)



Exercise 6

$$\begin{aligned}\text{Cache hit rate} &= \text{Number of hits} / \text{Number of accesses} \\ &= 2/6 \\ &= 0.333\end{aligned}$$

Replacement Algorithms

- **Direct-mapped cache**, the position that each memory block occupies in the cache is fixed. As a result, the replacement strategy is **trivial**.
- Associative and set-associative mapping provide some flexibility in deciding which memory block occupies which cache block.
- When a new block is to be transferred to the cache, and all the positions it **may occupy are full, which block in the cache should be replaced?**

When a miss occurs, cache controller must select a block to be replaced with the desired data.

Replacement Algorithms

These are three primary strategies employed for selecting which block to replace:

- **Random**
- **Least Recently Used [LRU]**
- **First In First Out [FIFO]**
- **Least Frequently Used [LFU]**

Random:

To spread allocation uniformly, candidate blocks are randomly selected.

First In First Out:

Other replacement algorithms which include removing the “oldest” block, disregard the locality of reference principle and do not perform as well.

Least Frequently Used:

Replace the block that has had the fewest hits.

Replacement Algorithms

Least Recently Used (LRU):

- Locality of reference suggests that it may be okay to replace the block **that has gone the longest time without being referenced**.
- This block is called Least Recently Used (LRU) block, and the replacement strategy is called LRU replacement algorithm.
- LRU algorithm has been used extensively.
 - It provides poor performance in some cases.
 - Performance of the LRU algorithm may be improved by introducing a small amount of randomness into which block might be replaced.

**Note: Random is simple to build in the hardware.
LRU becomes expensive as the number of
blocks increases.**

Additional Examples

- ❑ Consider a direct mapped cache with 8 cache blocks (0-7). If the memory block requests are in the order 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7, which of the memory blocks will be present in the cache at the end of the sequence? Also, calculate the hit ratio and miss ratio.
- ❑ Consider a 2-way SA mapped cache with 8 cache blocks (0-7) with LRU replacement policy. If the memory block requests are in the order 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7, which of the memory blocks will be present in the cache at the end of the sequence? Also, calculate the hit ratio and miss ratio.

- 1. Which mapping function allows a main memory block to be placed into any position in the cache?** A) Direct mapping B) Set-associative mapping C) Fully associative mapping D) Modulo mapping **Answer: C**
- 2. In Direct Mapping, how is the memory address typically divided?** A) Set, Block, Word B) Tag, Set, Word C) Tag, Block, Word D) Block, Offset, Line **Answer: C**
- 3. Which of the following is a disadvantage of Associative Mapping?** A) It uses cache space inefficiently. B) The cost is higher because it requires searching all patterns to determine if a block is in the cache. C) It relies heavily on a trivial replacement algorithm. D) It only allows one block per set. **Answer: B**
- 4. Which cache replacement algorithm replaces the block that has gone the longest time without being referenced?** A) Least Frequently Used (LFU) B) First In First Out (FIFO) C) Random D) Least Recently Used (LRU) **Answer: D**

1. Which mapping function allows a main memory block to be placed into any position in the cache?

- A) Direct mapping
- B) Set-associative mapping
- C) Fully associative mapping
- D) Modulo mapping

2. In Direct Mapping, how is the memory address typically divided?

- A) Set, Block, Word
- B) Tag, Set, Word
- C) Tag, Block, Word
- D) Block, Offset, Line

1. Which mapping function allows a main memory block to be placed into any position in the cache?

- A) Direct mapping
- B) Set-associative mapping
- C) Fully associative mapping
- D) Modulo mapping

Answer: C

2. In Direct Mapping, how is the memory address typically divided?

- A) Set, Block, Word
- B) Tag, Set, Word
- C) Tag, Block, Word
- D) Block, Offset, Line

Answer: C

3. Which of the following is a disadvantage of Associative Mapping?

- A) It uses cache space inefficiently.
- B) The cost is higher because it requires searching all patterns to determine if a block is in the cache.
- C) It relies heavily on a trivial replacement algorithm.
- D) It only allows one block per set.

4. Which cache replacement algorithm replaces the block that has gone the longest time without being referenced?

- A) Least Frequently Used (LFU)
- B) First In First Out (FIFO)
- C) Random
- D) Least Recently Used (LRU)

3. Which of the following is a disadvantage of Associative Mapping?

- A) It uses cache space inefficiently.
- B) The cost is higher because it requires searching all patterns to determine if a block is in the cache.
- C) It relies heavily on a trivial replacement algorithm.
- D) It only allows one block per set.

Answer: B

4. Which cache replacement algorithm replaces the block that has gone the longest time without being referenced?

- A) Least Frequently Used (LFU)
- B) First In First Out (FIFO)
- C) Random
- D) Least Recently Used (LRU)

Answer: D

THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Unit 3

Session - 3.5

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Cache Memory – Mapping Techniques

REPLACEMENT ALGORITHMS

- When a miss occurs, cache controller must select a cache line to be replaced with the desired data.
- **First In First Out(FIFO):**
 - Evict the page that has been in the cache the longest
- **Least Recently Used(LRU):**
 - Replace the block which is never used or used long ago
- **Random:**
 - Choose a page at random to evict from the cache.

Replacement Algorithms : Direct mapping

- No choice
- Each Block only maps to one Line
- Replace that line

Exercise 1: Direct Mapping

Consider a Direct Mapping cache with 8 cache blocks (numbered 0-7) and the following sequence of memory block requests.

4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Which of the memory blocks will be present in the cache at the end of the sequence?

Also, calculate the hit ratio and miss ratio.

4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16 25 6,22 3,19,35 4 45 7

$j \bmod 8$

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on
 $j \bmod 8$

Line 0	
Line 1	
Line 2	
Line 3	
Line 4	4
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on
 $j \bmod 8$

Line 0	
Line 1	
Line 2	
Line 3	3
Line 4	4
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on
 $j \bmod 8$

Line 0	
Line 1	25
Line 2	
Line 3	3
Line 4	4
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on
 $j \bmod 8$

Line 0	8
Line 1	25
Line 2	
Line 3	3
Line 4	4
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on
 $j \bmod 8$

Line 0	8
Line 1	25
Line 2	
Line 3	3,19
Line 4	4
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on
 $j \bmod 8$

Line 0	8
Line 1	25
Line 2	
Line 3	3,19
Line 4	4
Line 5	
Line 6	6
Line 7	

[illegible]

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on
 $j \bmod 8$

Line 0	8,16
Line 1	25
Line 2	
Line 3	3,19
Line 4	4
Line 5	
Line 6	6
Line 7	

[illegible]

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on
 $j \bmod 8$

Line 0	8 , 16
Line 1	25
Line 2	
Line 3	3 , 19 , 35
Line 4	4
Line 5	
Line 6	6
Line 7	

[illegible]

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on $j \bmod 8$

Line 0	8,16
Line 1	25
Line 2	
Line 3	3,19,35
Line 4	4
Line 5	45
Line 6	6,22
Line 7	

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on
 $j \bmod 8$

Line 0	8 , 16 , 8
Line 1	25
Line 2	
Line 3	3 , 19 , 35
Line 4	4
Line 5	45
Line 6	6 , 22
Line 7	

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	MISS

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on $j \bmod 8$

Line 0	8,16,8
Line 1	25
Line 2	
Line 3	3,19,35,3
Line 4	4
Line 5	45
Line 6	6,22
Line 7	

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	MISS
3	MISS

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on
 $j \bmod 8$

Line 0	8,16 , 8,16
Line 1	25
Line 2	
Line 3	3,19 , 35,3
Line 4	4
Line 5	45
Line 6	6 ,22
Line 7	

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	MISS
3	MISS
16	MISS

Exercise 1: Direct Mapping

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

8,16

25

6,22

3,19,35

4

45

22

7

Color is based on $j \bmod 8$

Line 0	8 , 16 , 8 , 16
Line 1	25
Line 2	
Line 3	3 , 19 , 35 , 3
Line 4	4
Line 5	45
Line 6	6 , 22
Line 7	7

Miss= 14 out of 17 access

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	MISS
3	MISS
16	MISS
25	HIT
7	MISS

Exercise 2: FIFO-2 Way Set Associative

Consider a 2-way set associative cache with 8 cache blocks (numbered 0-7) and the following sequence of memory block requests:

4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Which of the memory blocks will be present in the cache at the end of the sequence if ***FIFO cache replacement policy is used***? Also, calculate the hit ratio and miss ratio.

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

$j \bmod 4$

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4
Set 1	
Set 2	
Set 3	

4,8,16 25,45 6,22 3,7,19,35

[illegible]

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4
Set 1	
Set 2	
Set 3	3

4	MISS
3	MISS

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4
Set 1	25
Set 2	
Set 3	3

4,8,16 25,45 6,22 3,7,19,35

[illegible]

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4
	8
Set 1	25
Set 2	
Set 3	3

4	MISS
3	MISS
25	MISS
8	MISS

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4
	8
Set 1	25
Set 2	
Set 3	3
	19

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4
	8
Set 1	25
Set 2	6
Set 3	3
	19

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4
	8
Set 1	25
Set 2	6
Set 3	3
	19

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4, 16
	8
Set 1	25
Set 2	6
Set 3	3
	19

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4, 16
	8
Set 1	25
Set 2	6
Set 3	3, 35
	19

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4, 16
	8
Set 1	25
	45
Set 2	6
Set 3	3, 35
	19

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4, 16
	8
Set 1	25
	45
Set 2	6
	22
Set 3	3, 35
	19

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4, 16
	8
Set 1	25
	45
Set 2	6
	22
Set 3	3, 35
	19

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	HIT

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4, 16
	8
Set 1	25
	45
Set 2	6
	22
Set 3	3, 35
	19, 3

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	HIT
3	MISS

4,8,16 25,45 6,22 3,7,19,35

Exercise 2: FIFO-2 Way Set Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Set 0	4,16
	8
Set 1	25
	45
Set 2	6
	22
Set 3	3,35,7
	19,3

Miss= 12 out of 17 access

4,8,16 25,45 6,22 3,7,19,35

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	HIT
3	MISS
16	HIT
25	HIT
7	MISS

Exercise 3: FIFO- Fully Associative

Consider a Fully Associative Mapping cache with 8 cache blocks (numbered 0-7) and the following sequence of memory block requests:

4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Which of the memory blocks will be present in the cache at the end of the sequence? Also, calculate the hit ratio and miss ratio.

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	
Line 1	
Line 2	
Line 3	
Line 4	
Line 5	
Line 6	
Line 7	

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	
Line 2	
Line 3	
Line 4	
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	
Line 3	
Line 4	
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	
Line 4	
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	8
Line 4	
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	8
Line 4	19
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	8
Line 4	19
Line 5	6
Line 6	
Line 7	

[illegible]

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	8
Line 4	19
Line 5	6
Line 6	16
Line 7	

[illegible]

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	8
Line 4	19
Line 5	6
Line 6	16
Line 7	35

[illegible]

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4, 45
Line 1	3, 22
Line 2	25
Line 3	8
Line 4	19
Line 5	6
Line 6	16
Line 7	35

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	HIT

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4, 45
Line 1	3, 22
Line 2	25, 3
Line 3	8
Line 4	19
Line 5	6
Line 6	16
Line 7	35

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	HIT
3	MISS

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4, 45
Line 1	3, 22
Line 2	25, 3
Line 3	8, 25
Line 4	19
Line 5	6
Line 6	16
Line 7	35

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	HIT
3	MISS
16	HIT
25	MISS

Exercise 3: FIFO- Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4, 45
Line 1	3, 22
Line 2	25, 3
Line 3	8, 25
Line 4	19, 7
Line 5	6
Line 6	16
Line 7	35

Miss= 13 out of 17 access

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	HIT
3	MISS
16	HIT
25	MISS
7	MISS

Exercise 4: LRU-Fully Associative

Consider a Fully Associative Mapping cache with 8 cache blocks (numbered 0-7) and the following sequence of memory block requests:

4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Which of the memory blocks will be present in the cache at the end of the sequence?

Also, calculate the hit ratio and miss ratio.

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	
Line 1	
Line 2	
Line 3	
Line 4	
Line 5	
Line 6	
Line 7	

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	
Line 2	
Line 3	
Line 4	
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	
Line 3	
Line 4	
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	
Line 4	
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	8
Line 4	
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	8
Line 4	19
Line 5	
Line 6	
Line 7	

[illegible]

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	8
Line 4	19
Line 5	6
Line 6	
Line 7	

[illegible]

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	8
Line 4	19
Line 5	6
Line 6	16
Line 7	

[illegible]

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4
Line 1	3
Line 2	25
Line 3	8
Line 4	19
Line 5	6
Line 6	16
Line 7	35

[illegible]

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4, 45
Line 1	3, 22
Line 2	25
Line 3	8
Line 4	19
Line 5	6
Line 6	16
Line 7	35

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	HIT

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4, 45
Line 1	3, 22
Line 2	25
Line 3	8
Line 4	19, 3
Line 5	6
Line 6	16
Line 7	35

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	HIT
3	MISS

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4, 45
Line 1	3, 22
Line 2	25
Line 3	8
Line 4	19, 3
Line 5	6
Line 6	16
Line 7	35

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	HIT
3	MISS
16	HIT
25	HIT

Exercise 4: LRU-Fully Associative

Given 4, 3, 25, 8, 19, 6, 25, 8, 16, 35, 45, 22, 8, 3, 16, 25, 7

Line 0	4, 45
Line 1	3, 22
Line 2	25
Line 3	8
Line 4	19, 3
Line 5	6, 7
Line 6	16
Line 7	35

Miss= 12 out of 17 access

4	MISS
3	MISS
25	MISS
8	MISS
19	MISS
6	MISS
25	HIT
8	HIT
16	MISS
35	MISS
45	MISS
22	MISS
8	HIT
3	MISS
16	HIT
25	HIT
7	MISS

Exercise 5: LRU-4 Way Set Associative

Consider a 4-way set associative mapping with 16 cache blocks. The memory block requests are in the order-

0, 255, 1, 4, 3, 8, 133, 159, 216, 129, 63, 8, 48, 32, 73, 92, 155

If LRU replacement policy is used, which cache block will not be present in the cache? Also, calculate the hit ratio and miss ratio.

Exercise 5: LRU-4 Way Set Associative

Consider a 4-way set associative mapping with 16 cache blocks. The memory block requests are in the order-
 0, 255, 1, 4, 3, 8, 133, 159, 216, 129, 63, 8, 48, 32, 73, 92, 155
 If LRU replacement policy is used, which cache block will not be present in the cache? Also, calculate the hit ratio and miss ratio.

Set 0	0, 48
	4, 32
	8
	216, 92
Set 1	1
	133
	129
	73

Set 2	
Set 3	255, 155
	3
	159
	63

0 % 4 = 0
 255 % 4 = 3
 1 % 4 = 1
 4 % 4 = 0
 3 % 4 = 3
 8 % 4 = 0
 133 % 4 = 1
 159 % 4 = 3
 216 % 4 = 0
 129 % 4 = 1
 63 % 4 = 3
 8 % 4 = 0
 48 % 4 = 0
 32 % 4 = 0
 73 % 4 = 1
 92 % 4 = 0
 155 % 4 = 3

0	MISS
255	MISS
1	MISS
4	MISS
3	MISS
8	MISS
133	MISS
159	MISS
216	MISS
129	MISS
63	MISS
8	HIT
48	MISS
32	MISS
73	MISS
92	MISS
155	MISS

Practice Exercise

Consider the cache has 4 blocks. For the memory references-

5, 12, 13, 17, 4, 12, 13, 17, 2, 13, 19, 13, 43, 61, 19

What is the hit ratio for the following cache replacement algorithms-

- i. Fully Associative FIFO
- ii. Fully Associative LRU
- iii. Direct mapping
- iv. 2-way set associative mapping using LRU

Fully Associative FIFO

5, 12, 13, 17, 4, 12, 13, 17, 2, 13, 19, 13, 43, 61, 19

5 , 4, 43
12 , 2, 61
13 , 19,
17 , 13

HIT RATIO = 5/15

5	MISS
12	MISS
13	MISS
17	MISS
4	MISS
12	HIT
13	HIT
17	HIT
2	MISS
13	HIT
19	MISS
13	MISS
43	MISS
61	MISS
19	HIT

Fully Associative LRU

5, 12, 13, 17, 4, 12, 13, 17, 2, 13, 19, 13, 43, 61, 19

5 , 4, 2 ,
12, 19
13,
17, 43

HIT RATIO = 6/15

5	MISS
12	MISS
13	MISS
17	MISS
4	MISS
12	HIT
13	HIT
17	HIT
2	MISS
13	HIT
19	MISS
13	HIT
43	MISS
61	MISS
19	HIT

Direct Mapping

5, 12, 13, 17, 4, 12, 13, 17, 2, 13, 19, 13, 43, 61, 19

12,4,12
5,13,17, 13,17, 13,61
2
19,43, 19

5%4	1
12%4	0
13%4	1
17%4	1
4%4	0
2%4	2
19%4	3
43%4	3
61%4	1

HIT RATIO = 1/15

5	Miss
12	Miss
13	Miss
17	Miss
4	Miss
12	Miss
13	Miss
17	Miss
2	Miss
13	Miss
19	Miss
13	Hit
43	Miss
61	Miss
19	Miss

Set Associative Mapping with LRU

5, 12, 13, 17, 4, 12, 13, 17, 2, 13, 19, 13, 43, 61, 19

Set 0	12
	4, 2
Set 1	5, 17, 19, 43, 19
	13, 61

5%2	1
12%2	0
13%2	1
17%2	1
4%2	0
2%2	0
19%2	1
43%2	1
61%2	1

HIT RATIO = 5/15

5	Miss
12	Miss
13	Miss
17	Miss
4	Miss
12	Hit
13	Hit
17	Hit
2	Miss
13	Hit
19	Miss
13	Hit
43	Miss
61	Miss
19	Miss

1. In a Direct Mapping cache, which block is chosen for replacement when a miss occurs?

- A) The least recently used block.
- B) A randomly chosen block.
- C) The block that has been in the cache the longest.
- D) There is no choice; the specific line the block maps to must be replaced.

2. According to the First In First Out (FIFO) replacement algorithm, which page is evicted when a miss occurs?

- A) The page that has been referenced the least.
- B) The page that has been in the cache the longest.
- C) A randomly selected page.
- D) The page that was most recently used.

1. In a Direct Mapping cache, which block is chosen for replacement when a miss occurs?

- A) The least recently used block.
- B) A randomly chosen block.
- C) The block that has been in the cache the longest.
- D) There is no choice; the specific line the block maps to must be replaced.

Answer: D

2. According to the First In First Out (FIFO) replacement algorithm, which page is evicted when a miss occurs?

- A) The page that has been referenced the least.
- B) The page that has been in the cache the longest.
- C) A randomly selected page.
- D) The page that was most recently used.

Answer: B

3. To calculate the number of Tag bits in a cache memory address, which formula is used?

- A) Number of bits in Physical Address + Number of bits in block offset
- B) Number of bits in Physical Address – Number of bits in block offset
- C) Block Size / Cache Size
- D) Total Cache Size * Number of lines

4. How is the cache hit ratio calculated?

- A) Number of misses / Number of accesses
- B) Number of hits / Number of misses
- C) Number of hits / Number of accesses
- D) Number of accesses / Number of hits

3. To calculate the number of Tag bits in a cache memory address, which formula is used?

- A) Number of bits in Physical Address + Number of bits in block offset
- B) Number of bits in Physical Address – Number of bits in block offset
- C) Block Size / Cache Size
- D) Total Cache Size * Number of lines

Answer: B

4. How is the cache hit ratio calculated?

- A) Number of misses / Number of accesses
- B) Number of hits / Number of misses
- C) Number of hits / Number of accesses
- D) Number of accesses / Number of hits

Answer: C



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Unit 3

Session - 3.6

Microprocessor & Computer Architecture (μ pCA)



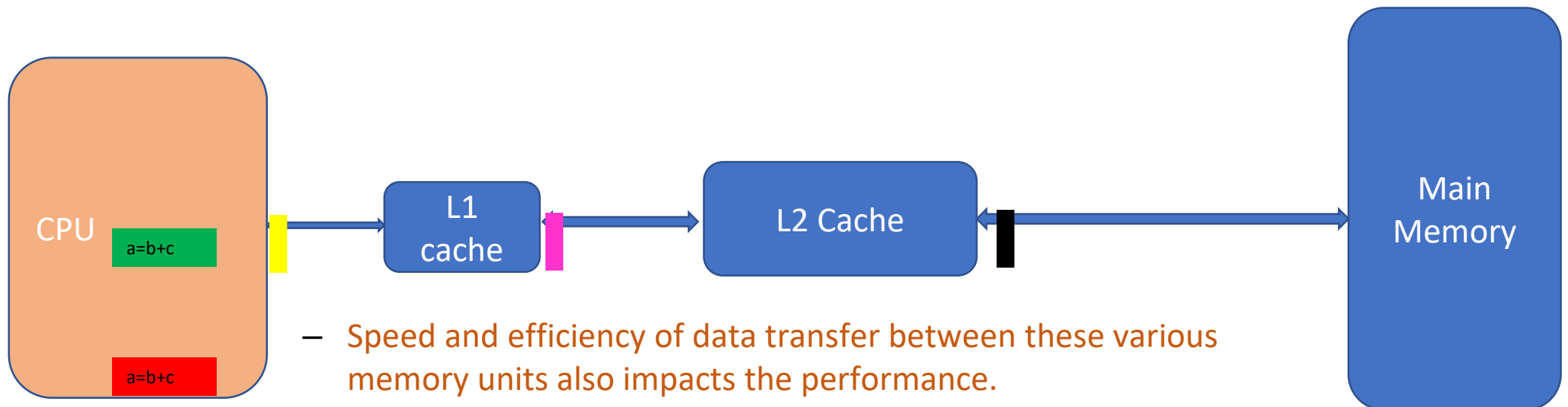
UE24CS251B

Performance Analysis

Performance Analysis

Known Facts

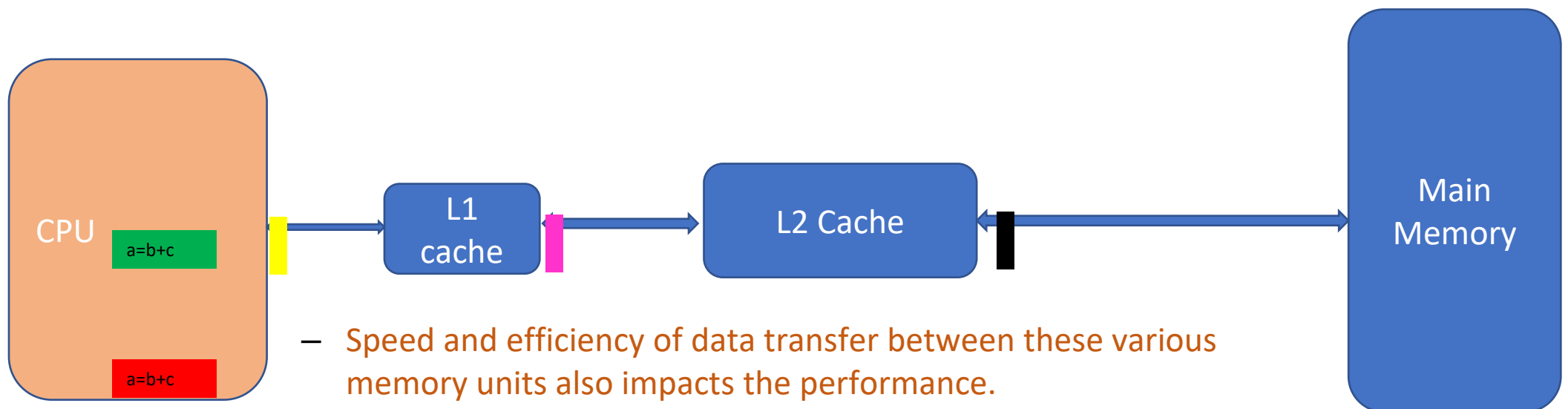
- User expect best possible performance at lowest possible cost.
- Performance depend on:-
 - ✓ How fast the Data / Instructions can be provided for execution?
 - ✓ How fast the Instructions can be executed?
- Memory **Hierarchy** is to balance Size, Cost and Speed
- However, data need to be transferred between various unit in the **Hierarchy**



Performance Analysis

To examine the performance of a memory system, the important factors are:

- How long does it take to send data from the cache to the CPU? (Hit Time)
- How long does it take to copy data from memory into the cache? (Miss Penalty)
- How often do we have to access main memory? (Miss Rate)



- CPU performance is measured by
 - **Instruction Count**
 - **Miss Rate**
 - Both independent of hardware
- A better measure of memory hierarchy performance is - **Average memory access time.**

Average memory access time = Hit time + Miss rate x Miss penalty.

where Hit time – Time to hit in the cache.

The components of an average access time can be measured either as

- Absolute time on a hit, say, 0.25 to 1.0 ns.
- Number of clock cycles that the processor waits for the memory, such as 150 to 200 clock cycles.

Note:

- Average memory access time is still an indirect measure of performance., although, better measure than miss rate.
- This formula can help to decide between split cache and a unified cache.

CACHE PERFORMANCE

- **CPU time = [CPU execution clock cycles + Memory stall clock cycles] x Clock cycle time.**

The equation has a question:

- Clock cycles for a cache hit should be considered part of
 - CPU execution clock cycles? or
 - Memory stall clock cycles?
- **Most widely accepted, is to include in CPU execution clock cycles.**

Miss Penalty

Three steps are taken when a cache needs to load data from the main memory.

- 1: Sending Address to RAM
- 2: Accessing data from RAM
- 3: Receiving data from RAM

Example: If the buses from the CPU to the cache and from the cache to RAM are all one word wide. Memory accesses take 15 cycles, If the cache has one-word blocks, How much time required for filling a block from RAM (*i.e.*, the miss penalty)?

1. It takes 1 cycle to send an address to the RAM.
2. If there is a 15-cycle latency for each RAM access.
3. It takes 1 cycle to return data from the RAM.

Miss Penalty is: $1 + 15 + 1 = 17$ clock cycles

Miss Penalty

Example2: If the buses from the CPU to the cache and from the cache to RAM are all one word wide. Memory accesses take 15 cycles, If the cache has **four-word blocks**, How much time required for filling a block from RAM (*i.e., the miss penalty*)?

Solution 1: If the cache has four-word blocks, then loading a single block would need four individual main memory accesses, and a miss penalty of 68 cycles!

$$4 \times (1 + 15 + 1) = 68 \text{ clock cycles}$$

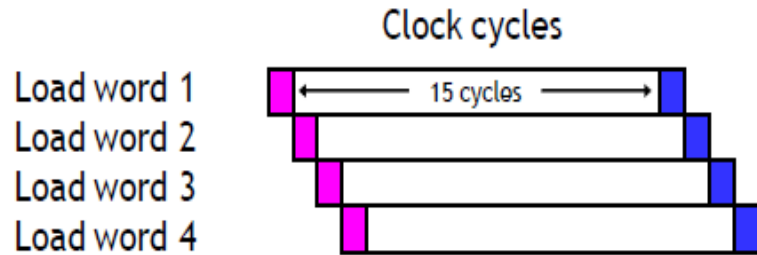
Solution 2: Widen the Bus to read multiple word in single shot.

$$1 + 15 + 1 = 17 \text{ cycles}$$

But it will be Costly! 😞

Miss Penalty

Solution 3: Create individual bank for each word and Overlap the Latencies.



$$1 + 15 + (4 \times 1) = 20 \text{ cycles}$$

- The magenta cycles represent sending an address to a memory bank.
- Each memory bank has a 15-cycle latency, and
- It takes another cycle (shown in blue) to return data from the memory.

This is the same basic idea as pipelining!

—As soon as we request data from one memory bank, we can go ahead and request data from another bank as well.

—Each individual load takes 17 clock cycles, but four overlapped loads require just 20 cycles.

Average Memory Access Time (AMAT)

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

- This is just averaging the amount of time for cache hits and the amount of time for cache misses.
- Lower AMAT is better
- Reduce Miss Penalty or Miss Rate

CACHE PERFORMANCE — EXAMPLE 1

	Cache #1	Cache #2
Block size	32-bytes	64-bytes
Miss rate	5%	4%

Which cache configuration would be better?

Assume both caches have single cycle hit times.

Memory accesses take 15 cycles, and the memory bus is 8-bytes wide

CACHE PERFORMANCE — EXAMPLE 1

Cache #1 : 32-byte memory access takes 20 cycles:
1 (send address) + 15 (memory access) + 4 (four 8-byte transfers)
Miss Penalty = $1 + 15 + 32\text{B}/8\text{B} = 20\text{cycles}$

$$\text{AMAT} = 1 + 0.05 \times 20 = 2$$

Cache #2 : 64-byte memory access takes 24 cycles:
1 (send address) + 15 (memory access) + 8 (four 8-byte transfers)
Miss Penalty = $1 + 15 + 64\text{B}/8\text{B} = 24\text{cycles}$

$$\text{AMAT} = 1 + 0.04 \times 24 = 1.9$$

Execution Time

We Know that

CPU Execution Time = IC x Overall CPI x Cycle time

Overall CPI = (Base CPI + CPU Stalls)

How does cache hits and misses affect system performance?

Overall CPI = (Base CPI + CPU Stalls + Memory Stalls)

Memory Stall Cycles = Memory Accesses x Miss Rate x Miss Penalty

Cache Performance Example 2

Assume that 33% of the instructions in a program are data accesses. The cache hit ratio is 97% and the hit time is one cycle, but the miss penalty is 20 cycles.

Solution

Cache Performance Example 2

Assume that 33% of the instructions in a program are data accesses. The cache hit ratio is 97% and the hit time is one cycle, but the miss penalty is 20 cycles.

Solution

Memory Stall Cycles = Memory Accesses x Miss Rate x Miss Penalty

$$\begin{aligned} &= (0.33) \times 0.03 \times 20 \text{ cycles} \\ &= 0.2 \text{ cycles} \end{aligned}$$

CPI = (Base CPI + CPU Stalls + Memory Stalls)

$$\text{CPI} = [1 + 0 + 0.2] = 1.2$$

CPU Execution Time = IC x 1.2 x cycle Time

This code is 1.2 times slower than a program with a “perfect” CPI of 1!

What if CPI becomes 0.5, but memory performance remained the same?

Memory systems are a Bottleneck

What if we could *double* the CPU performance so the CPI becomes 0.5, but memory performance remained the same?

$$\text{CPI} = 0.5 + 0.2 = 0.7$$

$$\text{CPU time} = \text{IC} \times (0.7) \times \text{Cycle time}$$

The overall CPU time improves by just $1.2/0.7 = 1.71$ times 😞

CACHE PERFORMANCE – EXAMPLE 3

Assume we have a computer where the cycles per instruction (CPI) is 1.0 when all memory accesses hit in the cache. The cache is a unified (data + instruction). The only data accesses are loads and stores, and these total 50% of the instructions. If the miss penalty is 25 clock cycles and the miss rate is 2%, how much faster would the computer be if all instructions were cache hits?

$$\text{Speedup} = \frac{\text{CPU Execution Time with real cache}}{\text{CPU Execution Time with all hits}}$$

CACHE PERFORMANCE — EXAMPLE 3

Assume we have a computer where the cycles per instruction (CPI) is 1.0 when all memory accesses hit in the cache. The cache is a unified (data + instruction). The only data accesses are loads and stores, and these total 50% of the instructions. If the miss penalty is 25 clock cycles and the miss rate is 2%, how much faster would the computer be if all instructions were cache hits?

SOLUTION:

First compute the performance for the computer that always hits:

$$\begin{aligned}\text{CPU execution time} &= \text{IC} \times (\text{CPI} + \text{Memory stall cycles}) \times \text{Clock cycle} \\ &= \text{IC} \times (\text{CPI} + 0) \times \text{Clock cycle} \\ &= \text{IC} \times 1.0 \times \text{Clock cycle}\end{aligned}$$

Now for the computer with the real cache, first we compute memory stall cycles:

$$\text{CPU Execution Time} = \text{IC} \times (\text{Base CPI} + \text{CPU Stalls} + \text{Memory Stalls}) \times \text{Cycle time}$$

CACHE PERFORMANCE — EXAMPLE 3

CPU Execution Time = IC x (Base CPI + CPU Stalls + Memory Stalls) x Cycle time

CPU Execution Time = IC x (1 + 0 + Memory Stalls) x Cycle time

$$\begin{aligned}\text{Memory Stall Cycles} &= \text{Memory Accesses} \times \text{Miss Rate} \times \text{Miss Penalty} \\ &= (1+0.5) \times .02 \times 25 \\ &= 0.75\end{aligned}$$

// Where memory access (Instruction Fetch + Data Access)

$$\begin{aligned}\text{CPU Execution Time} &= \text{IC} \times (1 + 0 + 0.75) \times \text{Cycle time} \\ &= \text{IC} \times (1.75) \times \text{Cycle time}\end{aligned}$$

The performance ratio is the inverse of the execution times:

$$\text{Speedup} = \frac{\text{CPU Execution Time with real cache}}{\text{CPU Execution Time with all hits}} = \frac{1.75}{1} = 1.75$$

Hence, The computer with no cache misses is 1.75 times faster.

$$\begin{aligned}
 \text{Memory stall cycles} &= IC \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss rate} \times \text{Miss penalty} \\
 &= IC \times (1 + 0.5) \times 0.02 \times 25 \\
 &= IC \times 0.75
 \end{aligned}$$

where the middle term $(1 + 0.5)$ represents one instruction access and 0.5 data accesses per instruction. The total performance is thus

$$\begin{aligned}
 \text{CPU execution time}_{\text{cache}} &= (IC \times 1.0 + IC \times 0.75) \times \text{Clock cycle} \\
 &= 1.75 \times IC \times \text{Clock cycle}
 \end{aligned}$$

Cache Performance - Example 4

Given data

- I-cache miss rate = 2%
- D-cache miss rate = 4%
- Miss penalty = 100 cycles
- Base CPI (ideal cache) = 2
- Load & stores are 36% of instructions

Cache Performance - Example 4

Given data

- I-cache miss rate = 2%
- D-cache miss rate = 4%
- Miss penalty = 100 cycles
- Base CPI (ideal cache) = 2
- Load & stores are 36% of instructions

Miss cycles per instruction

- I-cache: $0.02 \times 100 = 2$
- D-cache: $0.36 \times 0.04 \times 100 = 1.44$

$$\text{Actual CPI} = 2 + 2 + 1.44 = 5.44$$

$$\text{Speedup} = \text{IC} \times 5.44 \times \text{Clock Cycle} / \text{IC} \times 2 \times \text{Clock Cycle}$$

Conclusion: Ideal CPU is $5.44/2 = 2.72$ times faster

CACHE PERFORMANCE — EXAMPLE 5

Consider a pipelined processor that has an average CPI of 1.8 without accounting for memory stalls. I-Cache has a hit rate of 95% and the D-Cache has a hit rate of 98%. Assume that memory reference instructions account for 30% of all the instructions executed. Out of these 80% are loads and 20% are stores. On average, the read-miss penalty is 20 cycles and the write-miss penalty is 5 cycles. Compute the effective CPI of the processor accounting for the memory stalls.

CACHE PERFORMANCE — EXAMPLE 5

Consider a pipelined processor that has an average CPI of 1.8 without accounting for memory stalls. I-Cache has a hit rate of 95% and the D-Cache has a hit rate of 98%. Assume that memory reference instructions account for 30% of all the instructions executed. Out of these 80% are loads and 20% are stores. On average, the read-miss penalty is 20 cycles and the write-miss penalty is 5 cycles. Compute the effective CPI of the processor accounting for the memory stalls.

Note:

Avg CPI = 1.8;

I-cache miss ratio = $(1 - 0.95) = 0.05$;

D-cache miss ratio = $(1 - 0.98) = 0.02$; 30% mem references (80% loads & 20% stores)

Read miss penalty = 20 cycles; Write miss penalty = 5 cycles;

CACHE PERFORMANCE — EXAMPLE 5

Cost of instruction misses = cache miss rate * read miss penalty

$$= 1 * \mathbf{0.05} * 20$$

$$= 1 \text{ cycle per instruction}$$

- Cost of data read misses = fraction of memory reference instructions in program *
fraction of memory reference instructions that are **loads** *

$$\mathbf{D\text{-}cache \text{ miss rate}} * \mathbf{Read \text{ miss penalty}}$$

$$= 0.3 * 0.8 * 0.02 * 20$$

$$= 0.096 \text{ cycles per instruction}$$

- Cost of data write misses = fraction of memory reference instructions in the program *
fraction of memory reference instructions that are **stores** *

$$\mathbf{D\text{-}cache \text{ miss rate}} * \mathbf{Write \text{ miss penalty}}$$

$$= 0.3 * 0.2 * 0.02 * 5$$

$$= 0.006 \text{ cycles per instruction}$$

Effective CPI = Avg CPI + Effect of I-Cache on CPI + Effect of D-Cache on CPI

$$= 1.8 + 1 + 0.096 + 0.006$$

$$= \mathbf{2.902}$$

1. What is the correct formula for Average Memory Access Time (AMAT)?

- A) Hit time + (Miss rate x Miss penalty)
- B) Hit time x (Miss rate + Miss penalty)
- C) Miss rate + (Hit time x Miss penalty)
- D) Hit time - Miss penalty

2. Which of the following best describes "Miss Penalty"?

- A) The time it takes to send data from the cache to the CPU.
- B) The percentage of accesses that result in a cache miss.
- C) The time it takes to copy data from the main memory into the cache.
- D) The number of clock cycles required to execute an instruction.

1. What is the correct formula for Average Memory Access Time (AMAT)?

- A) Hit time + (Miss rate x Miss penalty)
- B) Hit time x (Miss rate + Miss penalty)
- C) Miss rate + (Hit time x Miss penalty)
- D) Hit time - Miss penalty

Answer: A

2. Which of the following best describes "Miss Penalty"?

- A) The time it takes to send data from the cache to the CPU.
- B) The percentage of accesses that result in a cache miss.
- C) The time it takes to copy data from the main memory into the cache.
- D) The number of clock cycles required to execute an instruction.

Answer: C

3. What is the accepted way to categorize clock cycles for a cache hit when calculating CPU time?

- A) They are considered part of memory stall clock cycles.
- B) They are considered part of CPU execution clock cycles.
- C) They are excluded from the performance calculation entirely.
- D) They are multiplied by the miss penalty.

4. How can the miss penalty be reduced when filling a multi-word block from RAM?

- A) By decreasing the cache size.
- B) By creating individual memory banks for each word and overlapping the latencies.
- C) By increasing the number of clock cycles per instruction.
- D) By disabling the L1 cache.

3. What is the accepted way to categorize clock cycles for a cache hit when calculating CPU time?

- A) They are considered part of memory stall clock cycles.
- B) They are considered part of CPU execution clock cycles.
- C) They are excluded from the performance calculation entirely.
- D) They are multiplied by the miss penalty.

Answer: B

4. How can the miss penalty be reduced when filling a multi-word block from RAM?

- A) By decreasing the cache size.
- B) By creating individual memory banks for each word and overlapping the latencies.
- C) By increasing the number of clock cycles per instruction.
- D) By disabling the L1 cache.

Answer: B



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Unit 3

Session - 3.7

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Cache Optimizations - Introduction

CACHE OPTIMIZATIONS

The equation,

Average memory access time = Hit time + Miss rate x Miss penalty.

Gives a frame work to present cache optimizations for improving cache performance. Hence, six basic cache optimizations can be organized into three categories.

- **Reducing the miss rate**
 - Larger block size, Larger cache size, and higher associativity.
- **Reducing the miss penalty**
 - Multilevel caches and giving reads priority over the writes.
- **Reducing the time to hit in the cache**
 - Avoiding address translation when indexing the cache.

CACHE OPTIMIZATIONS

The classical approach to improve cache behaviour is to reduce miss rates.

Three categories misses are:

- **Compulsory :**
 - The very first access to block cannot be in the cache. So, the block must be brought into the cache.
 - These are called cold-start misses or first reference misses.
- **Capacity:**
 - If the cache cannot contain all the blocks needed during execution of a program, capacity misses [in addition to compulsory misses] will occur because of blocks being discarded and later retrieved.

CACHE OPTIMIZATIONS

Three categories misses are:

- **Conflict:**
 - If the block placement strategy is set associative or direct mapped, conflict misses [in addition to compulsory and capacity misses] will occur because a block may be discarded and later retrieved if too many blocks map to its set. These misses are also called collision misses.
 - The idea is that hits in a fully associative cache that become misses in an n-way set associative cache, due to more than n requests on some popular sets.

NOTE: Further adding a 4th C , for **coherence misses** due to flushes to keep multiple caches coherent in a multiprocessor environment.

CACHE OPTIMIZATIONS

- **Compulsory Misses :**
 - Those that occur in an infinite cache.
- **Capacity Misses:**
 - Those that occur in a fully associative cache.
- **Conflict Misses:**
 - Those that occur going from fully associative to eight-way associative, four-way associative , and so on...

To show the benefit of associativity, conflict misses are divided into misses caused by each decrease in associativity.

Here are the four divisions of conflict misses and how they are calculated:

- **Eight-way**—Conflict misses due to going from fully associative to eight-way associative
- **Four-way**—Conflict misses due to going from eight-way associative to four way associative
- **Two-way**—Conflict misses due to going from four-way associative to two way associative
- **One-way**—Conflict misses due to going from two-way associative to one way associative

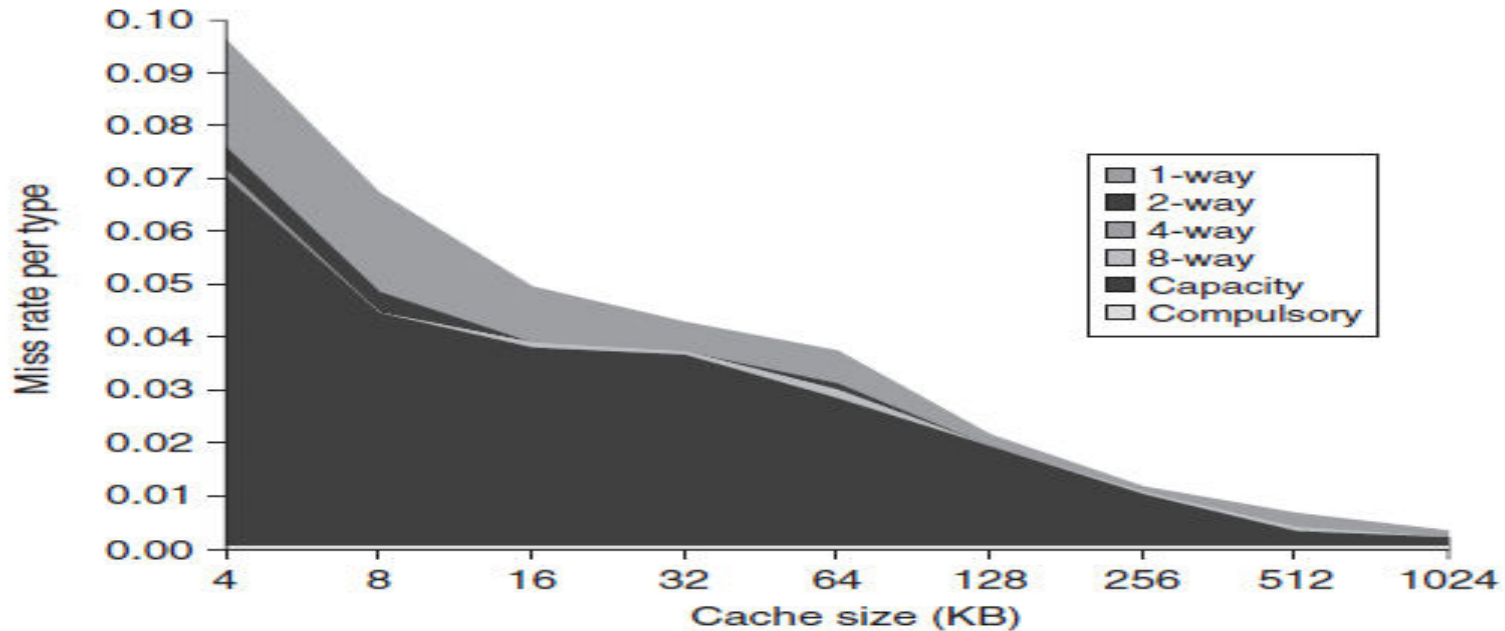
CACHE OPTIMIZATIONS

Conceptually, conflicts are the easiest:

- Fully associative placement avoids all conflict misses,
- Expensive in hardware and
- May slow down the processor clock rate leads to lower overall performance.

Cache size (KB)	Degree associative	Total miss rate	Miss rate components (relative percent) (sum = 100% of total miss rate)					
			Compulsory		Capacity		Conflict	
4	1-way	0.098	0.0001	0.1%	0.070	72%	0.027	28%
4	2-way	0.076	0.0001	0.1%	0.070	93%	0.005	7%
4	4-way	0.071	0.0001	0.1%	0.070	99%	0.001	1%
4	8-way	0.071	0.0001	0.1%	0.070	100%	0.000	0%
8	1-way	0.068	0.0001	0.1%	0.044	65%	0.024	35%
8	2-way	0.049	0.0001	0.1%	0.044	90%	0.005	10%
8	4-way	0.044	0.0001	0.1%	0.044	99%	0.000	1%
8	8-way	0.044	0.0001	0.1%	0.044	100%	0.000	0%
16	1-way	0.049	0.0001	0.1%	0.040	82%	0.009	17%
16	2-way	0.041	0.0001	0.2%	0.040	98%	0.001	2%
16	4-way	0.041	0.0001	0.2%	0.040	99%	0.000	0%
16	8-way	0.041	0.0001	0.2%	0.040	100%	0.000	0%

CACHE OPTIMIZATIONS



Total miss rate for each size cache according to the three C's for the data diagram shows the actual data cache miss rates,

CACHE OPTIMIZATIONS

- **First Optimization:** Larger Block Size to Reduce Miss Rate.
- **Second Optimization:** Larger Caches to Reduce Miss Rate.
- **Third Optimization:** Higher Associativity to Reduce Miss Rate.
- **Fourth Optimization :** Multilevel Caches to Reduce Miss Penalty.
- **Fifth Optimization:** Giving Priority to Read Misses over Writes to Reduce Miss Penalty.
- **Sixth Optimization:** Avoiding Address Translation during Indexing of the Cache to Reduce Hit Time.

1. Misses that occur because it is the very first access to a block are called:

- A) Conflict misses
- B) Collision misses
- C) Capacity misses
- D) Compulsory misses

2. If a cache cannot contain all the blocks needed during the execution of a program, what type of misses will occur?

- A) Compulsory misses
- B) Capacity misses
- C) Conflict misses
- D) Coherence misses

1. Misses that occur because it is the very first access to a block are called:

- A) Conflict misses
- B) Collision misses
- C) Capacity misses
- D) Compulsory misses

Answer: D

2. If a cache cannot contain all the blocks needed during the execution of a program, what type of misses will occur?

- A) Compulsory misses
- B) Capacity misses
- C) Conflict misses
- D) Coherence misses

Answer: B

3. Which of the following optimizations is primarily used to reduce the Miss Penalty?

- A) Using a larger block size.
- B) Using a larger cache size.
- C) Using multilevel caches and giving reads priority over writes.
- D) Avoiding address translation during indexing.

4. Conflict misses (or collision misses) can be entirely avoided by using which placement strategy?

- A) Direct mapped cache
- B) 2-way set-associative cache
- C) Fully associative placement
- D) 4-way set-associative cache

3. Which of the following optimizations is primarily used to reduce the Miss Penalty?

- A) Using a larger block size.
- B) Using a larger cache size.
- C) Using multilevel caches and giving reads priority over writes.
- D) Avoiding address translation during indexing.

Answer: C

4. Conflict misses (or collision misses) can be entirely avoided by using which placement strategy?

- A) Direct mapped cache
- B) 2-way set-associative cache
- C) Fully associative placement
- D) 4-way set-associative cache

Answer: C



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

**Unit 3 : Cache Optimizations:
1, 2 & 3 techniques.**

Session - 3.8

Cache Optimization : Quick Recap

- **Average access time = Hit time + Miss rate x Miss penalty**

Six “easy” ways to improve above equation

- Reducing miss rate:
 1. Larger block size (compulsory misses)
 2. Larger cache size (capacity misses)
 3. Higher associativity (conflict misses)
- Reducing miss penalty:
 4. Multilevel caches
 5. Prioritize reads over writes
- Reducing hit time:
 6. Avoiding address translation

Cache Optimization : Quick Recap

Uniprocessor **misses** fall into **three categories**:

- **Compulsory**: First access (ever to location)
- **Capacity** : Program accesses more than will fit in cache
- **Conflict**: Too many blocks map to given set
 - Sometimes called collision miss
 - Can't happen in fully associative caches

Optimization 1: Large Block Size to Reduce Miss Rate

Advantage:

- Satisfy Spatial Locality
- Reduces Compulsory Misses

Example: Instead of 4 word Block, use 8 word Block or 16 word Block

Optimization 1: Large Block Size to Reduce Miss Rate

Disadvantage:

- Increase Miss Penalty (Bus width Issues)

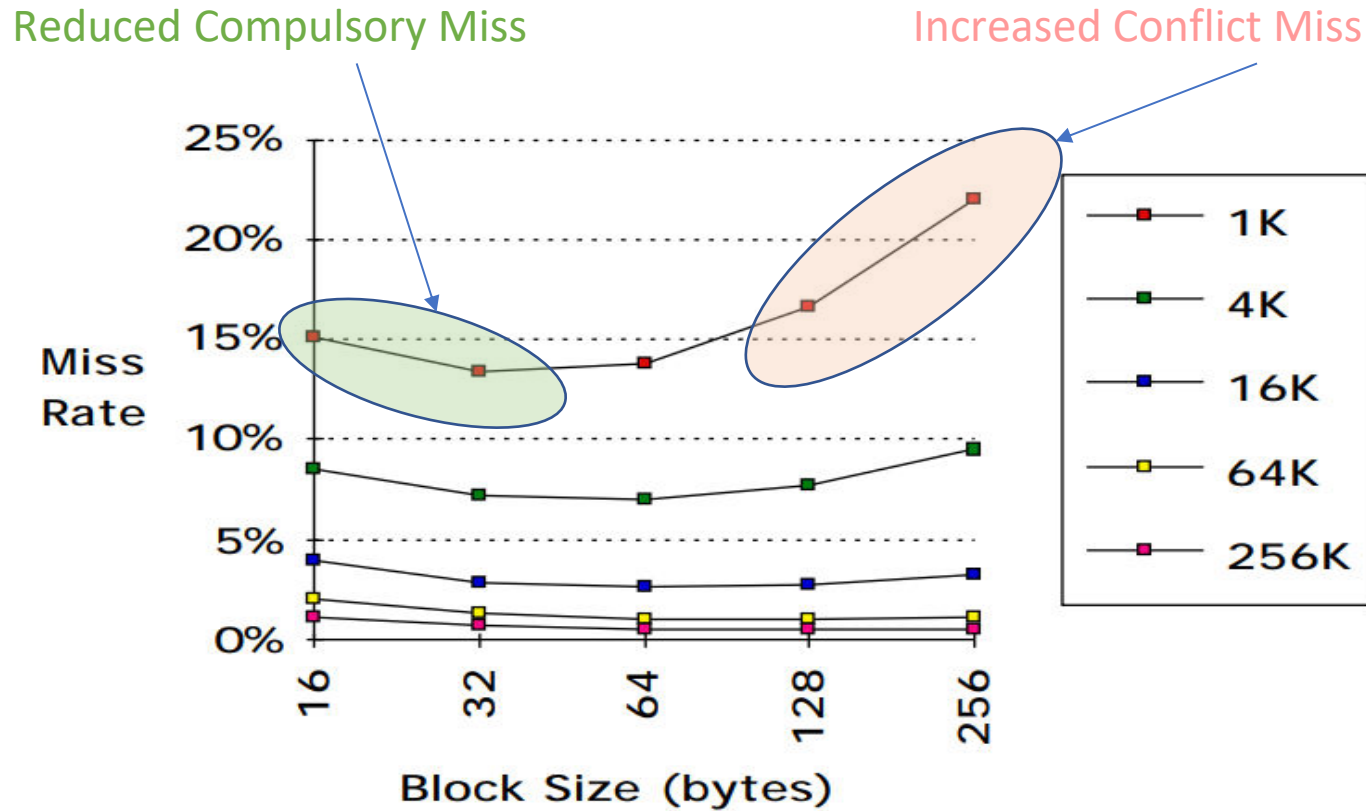
Example: If Width of Bus is 4 word, 2 transfers for 8 word Block and 4 transfers for 16 word Block are required.

- Increase Conflict Miss, as Number of Cache Blocks reduces due increase in Block Size.

Example: A 32 word cache can be designed as

- Eight, 4 word block
- Four, 8 word block or
- Two, 16 word block
- May bring useless data into cache as Spatial Locality cannot be maintained in entire program consistently.

Optimization 1: Large Block Size to Reduce Miss Rate



- Performance keeps improving to a limit.
- With fewer number of cache block, increases conflict misses and thus overall cache miss rate

Optimization 2: Large Cache to Reduce Miss Rate

Advantage:

- Reduces Capacity Miss
- Can accommodate large memory footprint.

Example: If number of Blocks are 4096:

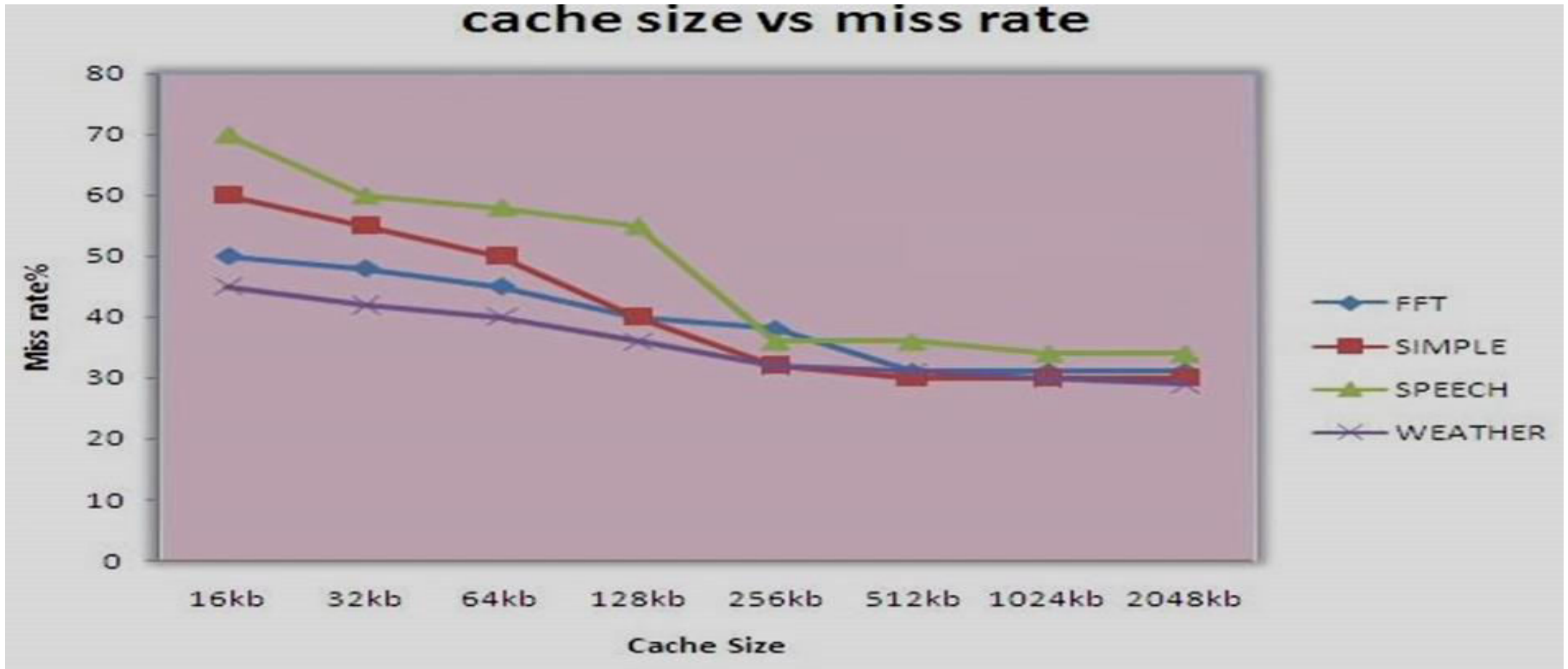
- On a cache with 128 Lines, 32 possible Blocks are mapped under direct mapping.
- On a cache with 256 Lines, 16 possible Blocks are mapped under direct mapping.

Optimization 2: Large Cache to Reduce Miss Rate

Disadvantage:

- Increases Hit Time
- High Cost, Area and Power

Optimization 2: Large Cache to Reduce Miss Rate



Optimization 1 & 2: Reduce Miss Rate

Block Size vs Cache Size

Block size	Cache size			
	4K	16K	64K	256K
16	8.57%	3.94%	2.04%	1.09%
32	7.24%	2.87%	1.35%	0.70%
64	7.00%	2.64%	1.06%	0.51%
128	7.78%	2.77%	1.02%	0.49%
256	9.51%	3.29%	1.15%	0.49%

Observation1:

Miss Rate Increases if Cache Size is small and Block size is Large

4 K Cache with 256 Block size has highest Miss Rate

Optimization 1 & 2: Reduce Miss Rate

Block Size vs Cache Size

Block size	Cache size			
	4K	16K	64K	256K
16	8.57%	3.94%	2.04%	1.09%
32	7.24%	2.87%	1.35%	0.70%
64	7.00%	2.64%	1.06%	0.51%
128	7.78%	2.77%	1.02%	0.49%
256	9.51%	3.29%	1.15%	0.49%

Observation2:

Miss Rate Decreases if Cache Size is Large and Block size is Large

256 k Cache with 256 Block Size has less Miss Rate

Optimization 1 & 2: Reduce Miss Rate

Block Size vs Cache Size

Block size	Cache size			
	4K	16K	64K	256K
16	8.57%	3.94%	2.04%	1.09%
32	7.24%	2.87%	1.35%	0.70%
64	7.00%	2.64%	1.06%	0.51%
128	7.78%	2.77%	1.02%	0.49%
256	9.51%	3.29%	1.15%	0.49%

Observation 3:

Miss Rate Decreases if Cache Size is Large and Block size is Small

Miss Rate Decreases if Cache Size is Large and Block Size is Large

However, If Cache Size is large, Hit Time, Space, Power and Cost Increases.

Optimization 1 & 2: Reduce Miss Rate

Example 1: Assume the memory system takes 80 clock cycles of overhead and then delivers 16 bytes every 2 clock cycles. That is, it can supply 16 bytes in 82 clock cycles, 32 bytes in 84 clock cycles, and so on... Which block size has the smallest average memory access time for each cache size?

Block size	Cache size			
	4K	16K	64K	256K
16	8.57%	3.94%	2.04%	1.09%
32	7.24%	2.87%	1.35%	0.70%
64	7.00%	2.64%	1.06%	0.51%
128	7.78%	2.77%	1.02%	0.49%
256	9.51%	3.29%	1.15%	0.49%

Optimization 1 & 2: Reduce Miss Rate

Example 1: Assume the memory system takes 80 clock cycles of overhead and then delivers 16 bytes every 2 clock cycles. That is, it can supply 16 bytes in 82 clock cycles, 32 bytes in 84 clock cycles, and so on... Which block size has the smallest average memory access time for each cache size?

Ans: Average access time = Hit time + Miss rate x Miss penalty

If we assume that the hit time is 1 clock cycle independent of the block size, then,

The access time for a 16- byte block in a 4KB cache is

$$\begin{aligned} \text{Average access time} &= 1 + (8.57\% \times 82) \\ &= 1 + 7.0274 \\ &= 8.0274 \text{ clock cycles.} \end{aligned}$$

For, 256 byte block in a 256 KB cache the

$$\begin{aligned} \text{Average access time} &= 1 + (0.49\% \times (80+32)) \\ &= 1 + (0.5488) \\ &= 1.5488 \text{ clock cycles.} \end{aligned}$$

Block size	Cache size			
	4K	16K	64K	256K
16	8.57%	3.94%	2.04%	1.09%
32	7.24%	2.87%	1.35%	0.70%
64	7.00%	2.64%	1.06%	0.51%
128	7.78%	2.77%	1.02%	0.49%
256	9.51%	3.29%	1.15%	0.49%

Optimization 3: Large Associativity to Reduce Miss Rate

Higher Associativity to Reduce Miss Rate

- Fully Associative cache are best, as no replacement is required till cache is full.
- Set Associative cache balances the Search Time and Replacement.

Advantage:

- Reduce Conflict Miss
- Reduce Miss Rate and Eviction rate.

Disadvantage:

- Increases Hit Time (Due to increase in search time by comparing each TAG)
- Complex design than Direct Map.

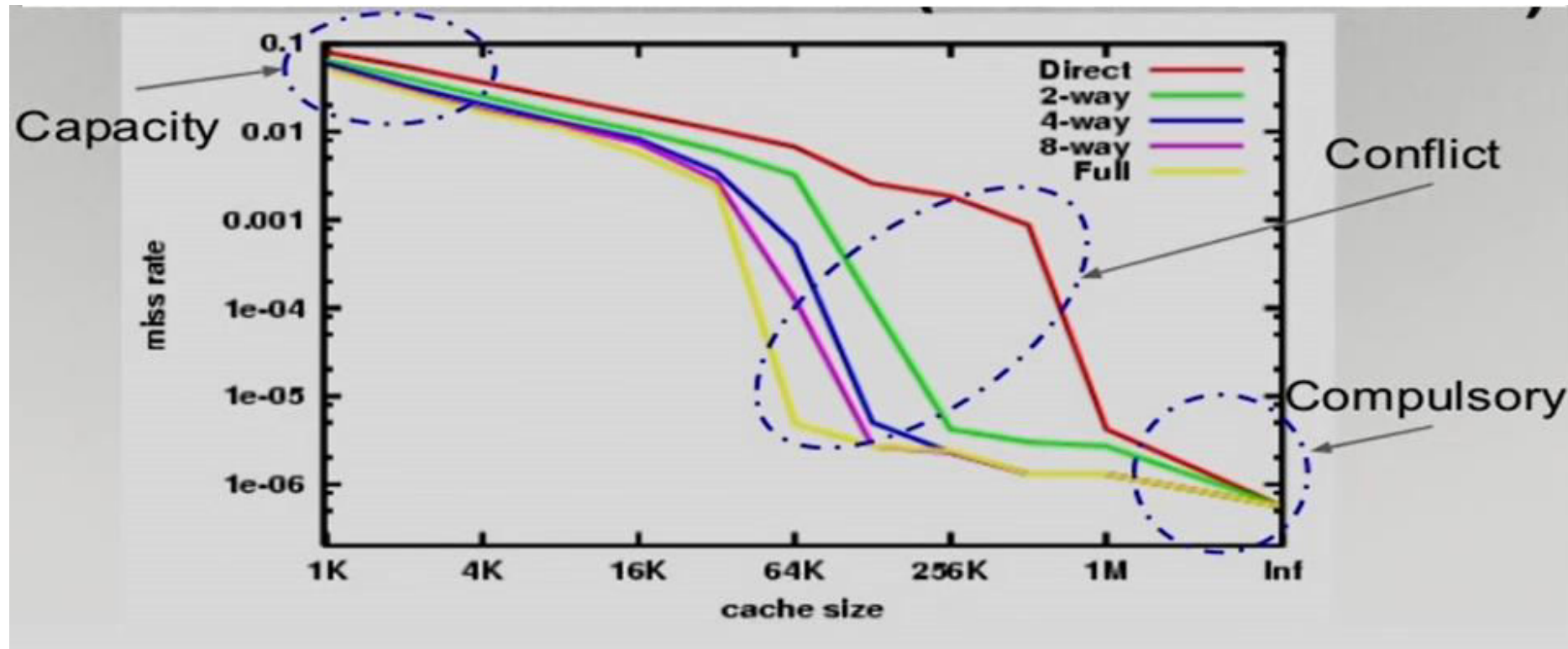
Replacement

- 2-way is better than Direct Mapping
- 4-way is better than 2-way
- 8-way is better than 4-way,

Search time

8-way, 16-way may become closer to Fully Associative.

Optimization 3: Large Associativity to Reduce Miss Rate



Replacement

- 2-way is better than Direct Mapping
- 4-way is better than 2-way
- 8-way is better than 4-way,

Search time

8-way, 16-way may become closer to Fully Associative.

Optimization 3: Large Associativity to Reduce Miss Rate

What about AMAT?

AMAT vs Cache Associativity

Assumption 1: Higher Associativity would increase the clock cycle time and is as listed below:

$$\text{Clock Cycle Time}_{2_way} = 1.36 \times \text{Clock Cycle Time}_{1_way}$$

$$\text{Clock Cycle Time}_{4_way} = 1.42 \times \text{Clock Cycle Time}_{1_way}$$

$$\text{Clock Cycle Time}_{8_way} = 1.52 \times \text{Clock Cycle Time}_{1_way}$$

Assumption 2: Hit time is 1 Clock and Miss Penalty= 25 Clock Cycle

AMAT= Hit Time + Miss Rate x Miss Penalty

$$\text{access time}_{8_way} = 1.52 + \text{Miss Rate} \times 25$$

$$\text{access time}_{4_way} = 1.44 + \text{Miss Rate} \times 25$$

$$\text{access time}_{2_way} = 1.36 + \text{Miss Rate} \times 25$$

$$\text{access time}_{1way} = 1.00 + \text{Miss Rate} \times 25$$

Optimization 3: Large Associativity to Reduce Miss Rate

Cache size (KB)	associative	rate
4	1-way	0.098
4	2-way	0.076
4	4-way	0.071
4	8-way	0.071
8	1-way	0.068
8	2-way	0.049
8	4-way	0.044
8	8-way	0.044
16	1-way	0.049
16	2-way	0.041
16	4-way	0.041
16	8-way	0.041
32	1-way	0.042
32	2-way	0.038
32	4-way	0.037
32	8-way	0.037
64	1-way	0.037
64	2-way	0.031
64	4-way	0.030
64	8-way	0.029
128	1-way	0.021
128	2-way	0.019
128	4-way	0.019
128	8-way	0.019
256	1-way	0.013
256	2-way	0.012
256	4-way	0.012
256	8-way	0.012
512	1-way	0.008
512	2-way	0.007
512	4-way	0.006
512	8-way	0.006

AMAT vs Cache Associativity

Example

The time for a 4KB Direct–Mapped cache is

The time for a 512 KB, Eight-Way Set Associative cache is

Optimization 3: Large Associativity to Reduce Miss Rate

Cache size (KB)	associative	rate
4	1-way	0.098
4	2-way	0.076
4	4-way	0.071
4	8-way	0.071
8	1-way	0.068
8	2-way	0.049
8	4-way	0.044
8	8-way	0.044
16	1-way	0.049
16	2-way	0.041
16	4-way	0.041
16	8-way	0.041
32	1-way	0.042
32	2-way	0.038
32	4-way	0.037
32	8-way	0.037
64	1-way	0.037
64	2-way	0.031
64	4-way	0.030
64	8-way	0.029
128	1-way	0.021
128	2-way	0.019
128	4-way	0.019
128	8-way	0.019
256	1-way	0.013
256	2-way	0.012
256	4-way	0.012
256	8-way	0.012
512	1-way	0.008
512	2-way	0.007
512	4-way	0.006
512	8-way	0.006

AMAT vs Cache Associativity

Example

The time for a 4KB Direct-Mapped cache is

$$\text{Average memory access time}_{1\text{-way}} = 1.00 + (0.098 \times 25) = 3.45$$

The time for a 512 KB, Eight-Way Set Associative cache is

$$\text{Average memory access time}_{8\text{-way}} = 1.52 + (0.006 \times 25) = 1.67$$

Example will make us to believe that

$$\text{Average memory access time}_{8\text{-way}} < \text{Average memory access time}_{4\text{-way}}$$

$$\text{Average memory access time}_{4\text{-way}} < \text{Average memory access time}_{2\text{-way}}$$

$$\text{Average memory access time}_{2\text{-way}} < \text{Average memory access time}_{1\text{-way}}$$

However ☹️

Optimization 3: Large Associativity to Reduce Miss Rate

AMAT vs Cache Associativity

Cache size (KB)	Associativity			
	1-way	2-way	4-way	8-way
4	3.44	3.25	3.22	3.28
8	2.69	2.58	2.55	2.62
16	2.23	2.40	2.46	2.53
32	2.06	2.30	2.37	2.45
64	1.92	2.14	2.18	2.25
128	1.52	1.84	1.92	2.00
256	1.32	1.66	1.74	1.82
512	1.20	1.55	1.59	1.66

High Associativity Leads to Higher Access Time

1. What is a significant disadvantage of using a very large block size to reduce the miss rate?

- A) It increases compulsory misses.
- B) It increases the miss penalty and can increase conflict misses due to having fewer cache blocks.
- C) It decreases the hit time significantly.
- D) It perfectly maintains spatial locality in all programs without fetching useless data.

2. What is the main advantage of increasing the cache size (Optimization 2)?

- A) It reduces the Hit Time.
- B) It reduces Compulsory misses.
- C) It lowers power consumption and cost. ‘
- D) It reduces Capacity misses by accommodating a larger memory footprint.

1. What is a significant disadvantage of using a very large block size to reduce the miss rate?

- A) It increases compulsory misses.
- B) It increases the miss penalty and can increase conflict misses due to having fewer cache blocks.
- C) It decreases the hit time significantly.
- D) It perfectly maintains spatial locality in all programs without fetching useless data.

Answer: B

2. What is the main advantage of increasing the cache size (Optimization 2)?

- A) It reduces the Hit Time.
- B) It reduces Compulsory misses.
- C) It lowers power consumption and cost. ‘
- D) It reduces Capacity misses by accommodating a larger memory footprint.

Answer: D

3. As cache associativity increases (e.g., moving from Direct Mapped to 8-way Set Associative), what is the typical trade-off?

- A) The miss rate increases significantly.
- B) The hit time (access time) increases because it requires comparing each tag.
- C) Conflict misses increase.
- D) Compulsory misses decrease to zero.

4. Based on performance charts, if cache size is small, what happens to the miss rate if you implement a very large block size?

- A) The miss rate approaches zero.
- B) The miss rate remains unaffected.
- C) The miss rate actually increases because of severe conflict misses.
- D) The hit time is reduced.

3. As cache associativity increases (e.g., moving from Direct Mapped to 8-way Set Associative), what is the typical trade-off?

- A) The miss rate increases significantly.
- B) The hit time (access time) increases because it requires comparing each tag.
- C) Conflict misses increase.
- D) Compulsory misses decrease to zero.

Answer: B

4. Based on performance charts, if cache size is small, what happens to the miss rate if you implement a very large block size?

- A) The miss rate approaches zero.
- B) The miss rate remains unaffected.
- C) The miss rate actually increases because of severe conflict misses.
- D) The hit time is reduced.

Answer: C



THANK YOU



Team MPCA

Department of Computer Science and
Engineering

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Unit 3: Cache Memory

4th & 5th Optimization

Session - 3.9

Multilevel Caches to Reduce Miss Penalty

- Observing the cache performance formula,
 - **Avg. memory access time = Hit time + Miss rate x Miss penalty,**
- Improvements in Miss penalty is as advantageous as improvements in miss rate.
- The performance gap between processor & memory raises a question:
 - Should I make the **cache faster** to keep the pace with the speed of the processor? **Or**
 - Make the **cache larger** to overcome the widening gap between the processor & the main memory?
- Answer for these questions is to do both.
 - **Adding another level of cache between memory & original cache simplifies the decision.**

Multilevel Caches to Reduce Miss Penalty

- First level cache can be small enough to match the clock cycle time of the processor.
- Second level cache can be large enough to capture many accesses that would go to main memory, thus reducing miss penalty.
- Multilevel cache will complicate performance analysis.
- Considering Memory access time for a two level cache using subscripts L1 & L2 to refer, respectively, to the first & second level,
- The original formula is:

$$\text{AMAT} = \text{Hit Time}_{L1} + \text{Miss Rate}_{L1} \times \text{Miss Penalty}_{L1}$$

Multilevel Caches to Reduce Miss Penalty

and

$$\text{Miss Penalty}_{L1} = \text{Hit Time}_{L2} + \text{Miss Rate}_{L2} \times \text{Miss Penalty}_{L2}$$

Substituting in original equation, we get,

$$\text{AMAT} = \text{Hit Time}_{L1} + \text{Miss Rate}_{L1} \times [\text{Hit Time}_{L2} + \text{Miss Rate}_{L2} \times \text{Miss Penalty}_{L2}]$$

Here,

Second level miss rate is measured on the leftovers from the first level cache.

To avoid ambiguity, the following terms are used for a two level cache system.

- ***Local miss rate***
- ***Global miss rate***

Multilevel Caches to Reduce Miss Penalty

Local miss rate:

- The number of misses in the cache divided by the total number of memory accesses to this cache.

Ex: For first level cache it is, **Miss Rate_{L1}**

For second level cache it is, **Miss Rate_{L2}**

Global miss rate:

- The number of misses in the cache divided by the total number of memory accesses generated by the processor.

Ex: Global miss rate for level1 cache is still **Miss Rate_{L1}**

but, for level2 cache it is : **Miss Rate_{L1} x Miss Rate_{L2}**

- $P(\text{MissL1})$: Probability of a miss in Level 1 cache (Miss RateL1).
- $P(\text{MissL2}|\text{MissL1})$: Probability of a miss in Level 2 cache given that there was a miss in Level 1 cache (Miss RateL2).

To obtain the global miss rate for Level 2 cache, we can rewrite $P(\text{MissL1 and MissL2})$ as:

$$P(\text{MissL1 and MissL2}) = P(\text{MissL1}) \times P(\text{MissL2}|\text{MissL1})$$

$$P(\text{MissL2}|\text{MissL1}) = \text{Miss RateL2}$$

$$P(\text{MissL1 and MissL2}) = \text{Miss RateL1} \times \text{Miss RateL2}$$

Multilevel Caches to Reduce Miss Penalty

- Local miss rate is large for second level caches because
 - The first level cache skims the cream of the memory accesses.
 - Global miss rate is more useful measure.
 - It indicates what fraction of the memory accesses that leave the processor go all the way to memory.
 - Here, the misses per instruction metric shines.

Expanding the memory stalls per instruction to add the impact of a second level,

Avg. Memory stalls per instruction = Misses per instruction_{L1} x Hit Time_{L2} + Misses per instruction_{L2} x Miss Penalty_{L2}

Misses Per Instruction (MPI) refers to the **average number of cache misses per instruction** executed in a program.

$$MPI_{L1} = \text{Miss Rate}_{L1} \times \text{Memory References Per Instruction}$$

$$MPI_{L2} = MPI_{L1} \times \text{Miss Rate}_{L2}$$

- The **Miss Rate of L1** tells us **what fraction of memory accesses miss in L1**.
- We multiply this by **Memory References Per Instruction** to account for **how many times an instruction accesses memory** on average.

Difference Between Misses Per Instruction (MPI) and Miss Rate

Metric	Miss Rate	Misses Per Instruction (MPI)
Definition	Fraction of memory accesses that result in a cache miss.	Average number of cache misses per instruction executed.
Formula	$\text{Miss Rate} = \frac{\text{Cache Misses}}{\text{Total Memory Accesses}}$	$MPI = \text{Miss Rate} \times \text{Memory References Per Instruction}$
Unit	Dimensionless (percentage or fraction)	Misses per instruction
Scope	Measures how often a cache miss occurs when a memory access is made.	Measures how often a cache miss occurs relative to the number of instructions executed.
Usage	Used to evaluate cache efficiency at a specific level (L1, L2, etc.).	Used to understand the overall impact of cache misses on program execution.

MPCA - Fourth optimization: Multilevel Caches to Reduce Miss Penalty

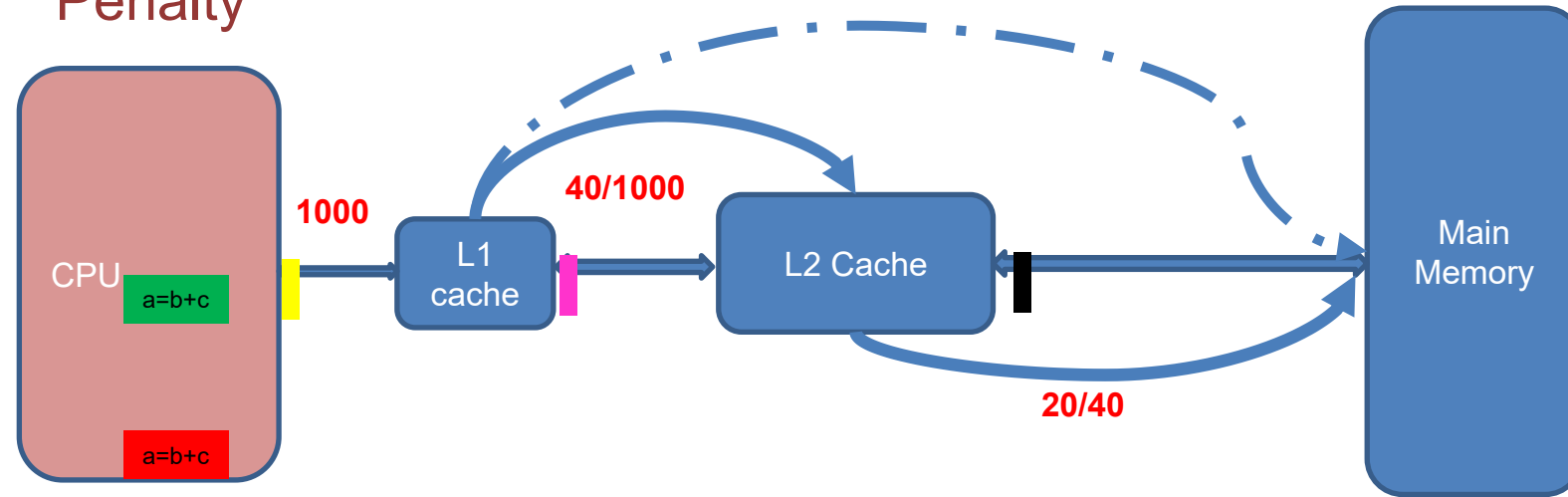
Suppose that in 1000 memory references there are 40 misses in the first level cache and 20 misses in the second –level cache. What are the various miss rates?

Assume the miss penalty from the L2 cache to memory is 200 clock cycles, the hit time of the L2 cache is 10 clock cycles, hit time for L1 cache is 1 clock cycle and there are 1.5 memory references per instruction.

What is the average memory access time and average stall cycles per instruction?

Ignore impact of writes.

4th Optimization: Multilevel Caches to Reduce Miss Penalty



Suppose that in 1000 memory references there are 40 misses in the first level cache and 20 misses in the second level cache.

What is Miss Rate of Level 1 Cache? **40/1000**

What is Miss Rate of Level 2 Cache? **20/40**

Multilevel Caches to Reduce Miss Penalty

Suppose that in 1000 memory references there are 40 misses in the first level cache and 20 misses in the second –level cache. What are the various miss rates?

Assume the miss penalty from the L2 cache to memory is 200 clock cycles, the hit time of the L2 cache is 10 clock cycles, hit time for L1 cache is 1 clock cycle and there are 1.5 memory references per instruction.

What is the average memory access time and average stall cycles per instruction?

Ignore impact of writes.

Answer

The miss rate [either global or local] for the first level cache is $40/1000 = 4\%$.

The local miss rate for the second-level cache is $20/40 = 50\%$.

The global miss rate of the second level cache is $20/1000 = 2\%$.

Then,

$$\begin{aligned} \text{AMAT} &= \text{Hit Time}_{L1} + \text{Miss Rate}_{L1} \times [\text{Hit Time}_{L2} + \text{Miss Rate}_{L2} \times \text{Miss Penalty}_{L2}] \\ &= 1 + 4\% \times (10 + 50\% \times 200) = 1 + 4\% \times 110 = 5.4 \text{ clock cycles.} \end{aligned}$$

Multilevel Caches to Reduce Miss Penalty

of instruction =

$$\begin{aligned} \text{Average memory stalls per instruction} &= \text{Misses per instruction}_{L1} \times \text{Hit time}_{L2} + \\ &\quad \text{Misses per instruction}_{L2} \times \text{Miss Penalty}_{L2} \\ &= \end{aligned}$$

Multilevel Caches to Reduce Miss Penalty

of instruction = # of memory references / # of memory references per instruction.
 $= 1000 / 1.5 = 667$ instructions.

Thus, for L1 cache ,

misses for 40 memory accesses for 1000 instructions = $40 \times 1.5 = 60$ misses

and for 20 misses for L2 cache it is $1.5 \times 20 = 30$ misses.

Average memory stalls per instruction = **Misses per instruction_{L1} x Hit time_{L2} +**
Misses per instruction_{L2} x Miss Penalty_{L2}
 $= (60/1000) \times 10 + (30/1000) \times 200$
 $= 0.06 \times 10 + 0.03 \times 200 = 0.6 + 6$
= 6.6 clock cycles.

Then,

Average memory stalls per instruction = (AMAT - Hit time_{L1}) x Average # of memory references per instruction
 $= (5.4 - 1.0) \times 1.5 = 6.6$ **clock cycles.**

Note: The computation of the memory stalls per instruction is same for either way.

Multilevel Caches to Reduce Miss Penalty

Note:

- Formulae are for combined reads & writes.
- A write back first level cache

A write-through first level cache will send

- Misses
- All writes to the second level
 - May use a write buffer.

Two insights observed from the diagrams.

Write-Through Cache:

- Every write to L1 is also written to L2 immediately.
- Requires a **write buffer** to improve performance by temporarily holding data before writing to L2.

Write-Back Cache:

- Writes are done only in L1 initially.
- Data is written to L2 only when it is replaced in L1.

Miss Handling in a Multilevel Cache System:

- L1 cache sends **misses** to L2.
- If L2 also misses, the request is forwarded to the **main memory**.
- Reducing miss penalty is crucial for performance improvement.

MPCA - Fourth optimization:

Multilevel Caches to Reduce Miss Penalty

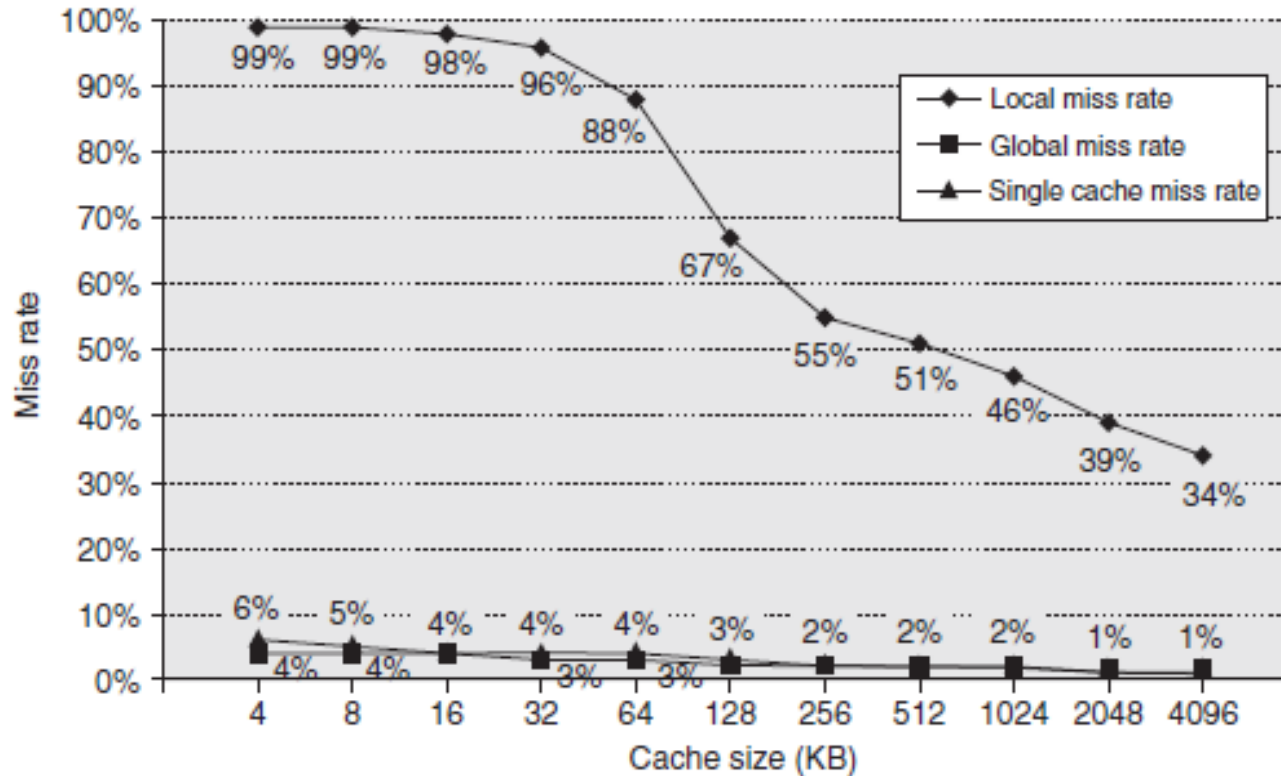


Figure B.14 Miss rates versus cache size for multilevel caches. Second-level caches smaller than the sum of the two 64 KB first-level caches make little sense, as reflected in the high miss rates. After 256 KB the single cache is within 10% of the global miss rates.

First perspective :

Global cache miss rate is very similar to the single cache miss rate of the second level cache.

- Provided that the second level cache is much larger than the first level cache.

MPCA - Fourth optimization:

Multilevel Caches to Reduce Miss Penalty

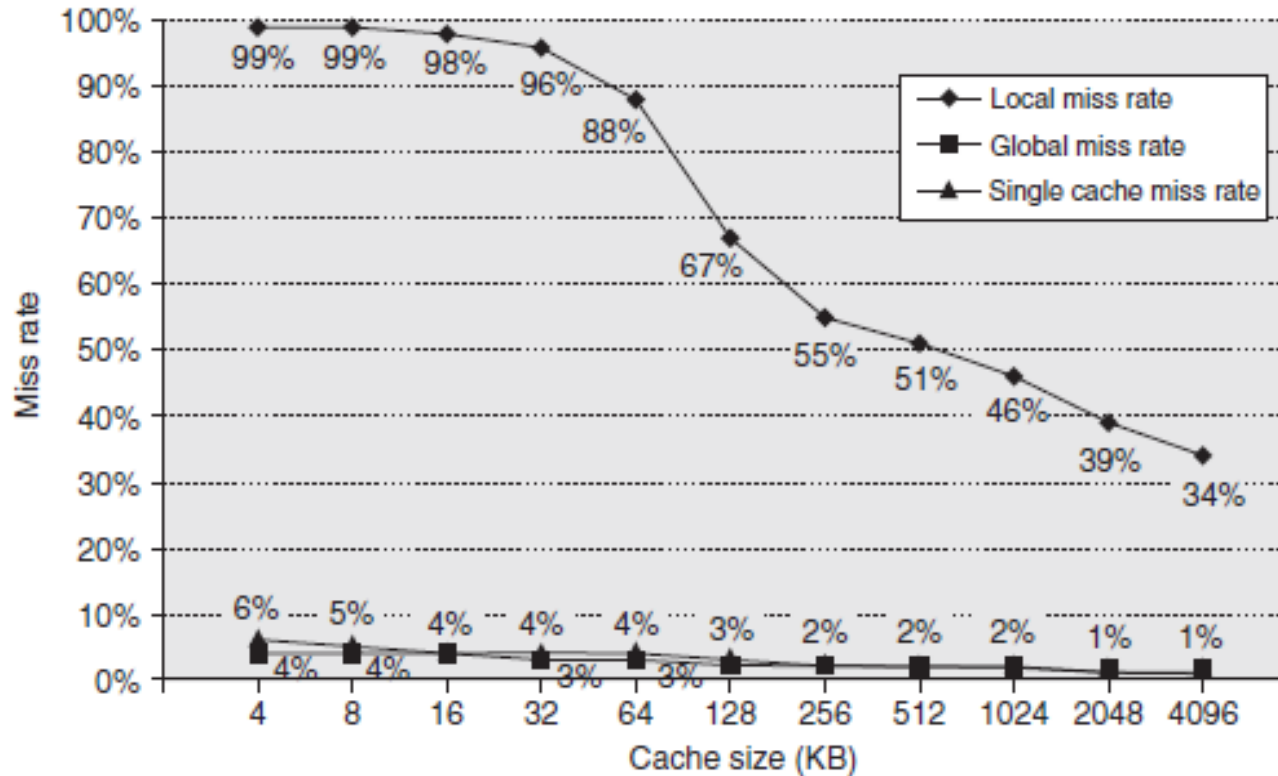


Figure B.14 Miss rates versus cache size for multilevel caches. Second-level caches smaller than the sum of the two 64 KB first-level caches make little sense, as reflected in the high miss rates. After 256 KB the single cache is within 10% of the global miss rates.

At very small cache sizes, the miss rate is extremely high (>90%).

Beyond **256 KB**, the **global miss rate stabilizes** and becomes close to the L2 miss rate.

Having **larger cache sizes significantly reduces miss rates**, but there is a point of diminishing returns.

A **larger L2 cache** significantly improves performance by reducing the overall miss rate. The **global miss rate** is close to the **L2 cache miss rate**, provided that **L2 is much larger than L1**.

Multilevel Caches to Reduce Miss Penalty

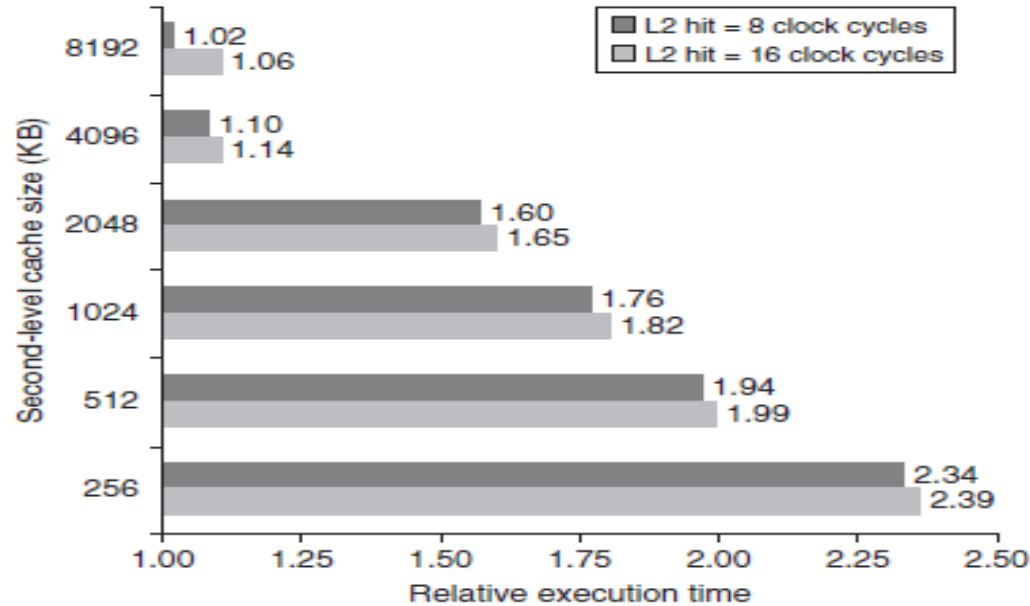


Figure B.15 Relative execution time by second-level cache size. The two bars are for different clock cycles for an L2 cache hit. The reference execution time of 1.00 is for an 8192 KB second-level cache with a 1-clock-cycle latency on a second-level hit. These data were collected the same way as in Figure B.14, using a simulator to imitate the Alpha 21264.

Second Perspective:

- Local cache miss rate is not a good measure of secondary caches.
- It is a function of the miss rate of the first level cache.
- Can vary by changing the first – level cache.

Note: Global cache miss rate should be used when evaluating second level caches.

- Perspective 1:
- A **larger L2 cache** significantly improves performance by reducing the overall miss rate.
- The **global miss rate** is close to the **L2 cache miss rate**, provided that **L2 is much larger than L1**.

•Perspective 2

- Local miss rate** is not an effective metric for evaluating **L2 caches** because it depends on L1 misses.
- Global miss rate** provides a better measure for **multilevel cache efficiency**.
- Increasing **L2 cache size improves performance**, but the benefits diminish beyond a certain threshold.

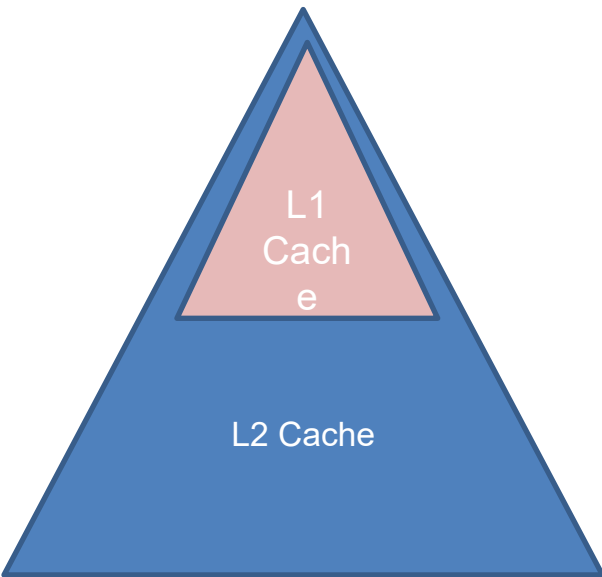
Why Global Miss Rate of 2nd Level Cache is Important?

First perspective :

Global cache miss rate is very similar to the single cache miss rate of the second level cache.

Second Perspective:

- Local cache miss rate is not a good measure of secondary caches.
- It is a function of the miss rate of the first level cache.
- Can vary by changing the first – level cache.



Note: Global cache miss rate should be used when evaluating second level caches.

Key Takeaways

1. **Global miss rate** is useful for **evaluating second-level cache performance** and overall system efficiency.
2. **Local miss rate** is used in **AMAT** because it correctly models the **sequential nature of cache accesses**.
3. **Using global miss rate directly in AMAT** would incorrectly assume that all memory accesses bypass L2 and go straight to main memory.

Multilevel Caches to Reduce Miss Penalty

Parameters for second level caches:

- Difference between two levels is the speed of the first level cache
 - That affects the clock rate of the processor.
 - While, Speed of the second level cache only affects the miss penalty of the first level cache.
 - Thus, many alternatives can be considered in second level cache that are ill chosen for the first level cache.

Two major questions for the design of the second level cache:

1. Will it lower the average memory access time portion of the CPI?
2. How much does it cost ?

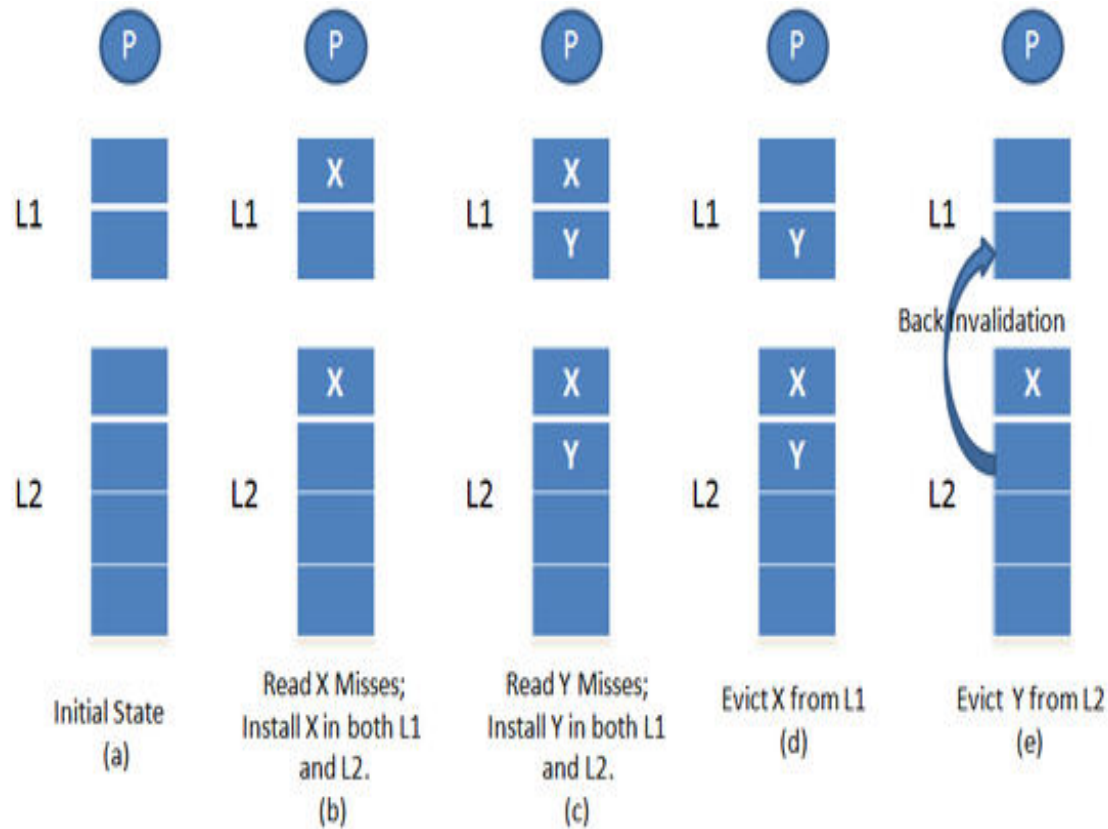
Multilevel Caches to Reduce Miss Penalty

- Initial decision is the **size of a second level cache**.
- Everything in first level cache is likely in the second level cache.
- The **size of the second level cache should be much higher than the first**.
- **If second level caches are just a little bigger, the local miss rate will be higher.**
- Thus, inspires the design of huge second level caches.
 - Probably to the size of the main memory in older computers!!!

NOTE:

- Multi level inclusion is the natural policy for memory hierarchies.
 - L1 data is always present in L2.

EXTRA-One example of inclusive cache is [Intel quad core processor](#) with 4x256KB L2 caches and 8MB (inclusive) L3 cache.

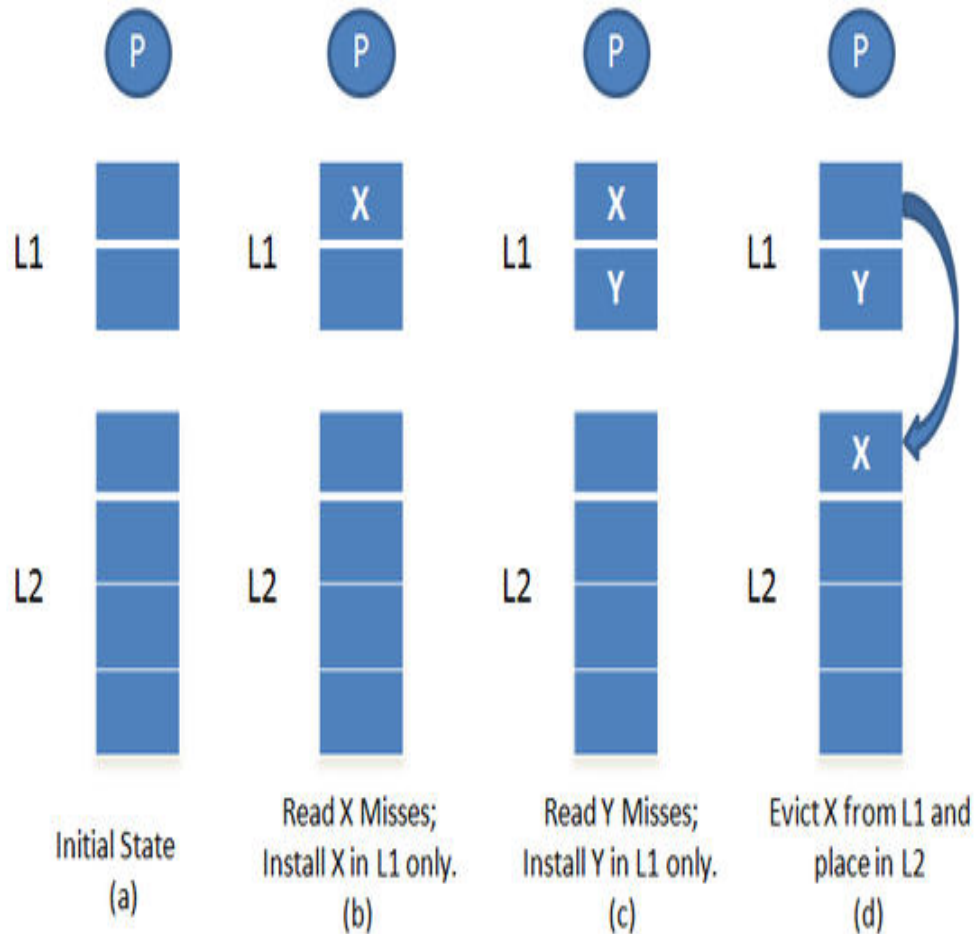


(A) Assume that the processor sends a read X request. It will be a miss in both L1 and L2 and hence the block is brought into both L1 and L2 from the [main memory](#) as shown in (b).

Now, assume the processor issues a read Y request which is a miss in both L1 and L2. So, block Y is placed in both L1 and L2 as shown in (c). **If block X has to be evicted from L1, then it is removed from L1 only as shown in (d).**

If block Y has to be evicted from L2, it sends a back invalidation request to L1 and hence block Y is evicted from L1 as shown in (e).

EXTRA-An example of exclusive cache is AMD Opteron with 512 KB (per core) L2 cache, exclusive of L1



- initially consider both L1 and L2 caches to be empty (a). Assume that the processor sends a read X request. It will be a miss in both L1 and L2 and hence the block is brought into L1 from the main memory as shown in (b). Now, again the processor issues a read Y request which is a miss in both L1 and L2. So, block Y is placed in L1 as shown in (c). If block X has to be evicted from L1, then it is removed from L1 and placed in L2 as shown in (d).

Metric	Inclusive Cache	Exclusive Cache
Cache Utilization	● Less efficient (duplicate data in L1 & L2)	● More efficient (L1 & L2 store unique data)
Memory Coherency	● Easier to manage	● More complex to track data
Miss Penalty	● Higher due to forced evictions	● Lower, data moves between caches
Global Miss Rate	● Higher (redundant storage)	● Lower (maximizes cache space)
Prefetching Efficiency	● Prefetched data stays in higher caches	● Prefetched data might not be in L1
Performance in Multi-Core CPUs	● Intel uses it for coherence	● AMD avoids it for larger cache size

- **Inclusive caches** are better for **multi-core cache coherence** (e.g., Intel).
- **Exclusive caches** provide **better cache utilization and lower miss rates** (e.g., AMD).

1.Initial State:

1. Initially, both **Core 1 and Core 2 have empty L1 caches.**
2. A particular set of data, let's say data **block X, is fetched from main memory by Core 1 and placed into its L1 cache.** Since this data is also included in the L2 cache due to the inclusion policy, it's also copied into the L2 cache.

2.Data Sharing:

1. Now, if Core 2 requires access to data block X, it doesn't need to fetch it directly from main memory. Instead, it can access the data from the shared L2 cache, where it's already present due to the inclusion policy.
2. This eliminates the need for expensive data transfers between caches or main memory. Core 2 can directly access the data from the L2 cache, thereby reducing latency and improving overall system performance.

3.Data Modification:

1. If Core 1 or Core 2 modifies data block X, the modification is reflected in both L1 and L2 caches due to cache coherence mechanisms. This ensures that all cores have consistent access to the updated data.

4.Efficient Data Sharing:Since both L1 and L2 caches contain the same data, data sharing between different cores becomes seamless and efficient. Cores can access shared data without the need for explicit data transfers or synchronization mechanisms, enhancing the overall efficiency of the multi-core processor.

- Whether set associativity makes more sense for second-level caches.

Example Given the data below, what is the impact of second-level cache associativity on its miss penalty?

- Hit time_{L2} for direct mapped = 10 clock cycles.
- Two-way set associativity increases hit time by 0.1 clock cycle to 10.1 clock cycles.
- Local miss rate_{L2} for direct mapped = 25%.
- Local miss rate_{L2} for two-way set associative = 20%.
- Miss penalty_{L2} = 200 clock cycles.

Multilevel Caches to Reduce Miss Penalty

Example Given the data below, what is the impact of second-level cache associativity on its miss penalty?

- Hit time_{L2} for direct mapped = 10 clock cycles.
- Two-way set associativity increases hit time by 0.1 clock cycle to 10.1 clock cycles.
- Local miss rate_{L2} for direct mapped = 25%.
- Local miss rate_{L2} for two-way set associative = 20%.
- Miss penalty_{L2} = 200 clock cycles.

Answer For a direct-mapped second-level cache, the first-level cache miss penalty is

$$\text{Miss penalty}_{1\text{-way } L2} = 10 + 25\% \times 200 = 60.0 \text{ clock cycles}$$

Adding the cost of associativity increases the hit cost only 0.1 clock cycle, making the new first-level cache miss penalty:

$$\text{Miss penalty}_{2\text{-way } L2} = 10.1 + 20\% \times 200 = 50.1 \text{ clock cycles}$$

Multilevel Caches to Reduce Miss Penalty

In reality, second-level caches are almost always synchronized with the first-level cache and processor. Accordingly, the second-level hit time must be an integral number of clock cycles. If we are lucky, we shave the second-level hit time to 10 cycles; if not, we round up to 11 cycles. Either choice is an improvement over the direct-mapped second-level cache:

$$\text{Miss penalty}_{2\text{-way L2}} = 10 + 20\% \times 200 = 50.0 \text{ clock cycles}$$

$$\text{Miss penalty}_{2\text{-way L2}} = 11 + 20\% \times 200 = 51.0 \text{ clock cycles}$$

**Miss penalty for 2 way associativity is less than direct mapped.
Set associativity is good design decision for second level cache**

Multilevel Caches to Reduce Miss Penalty

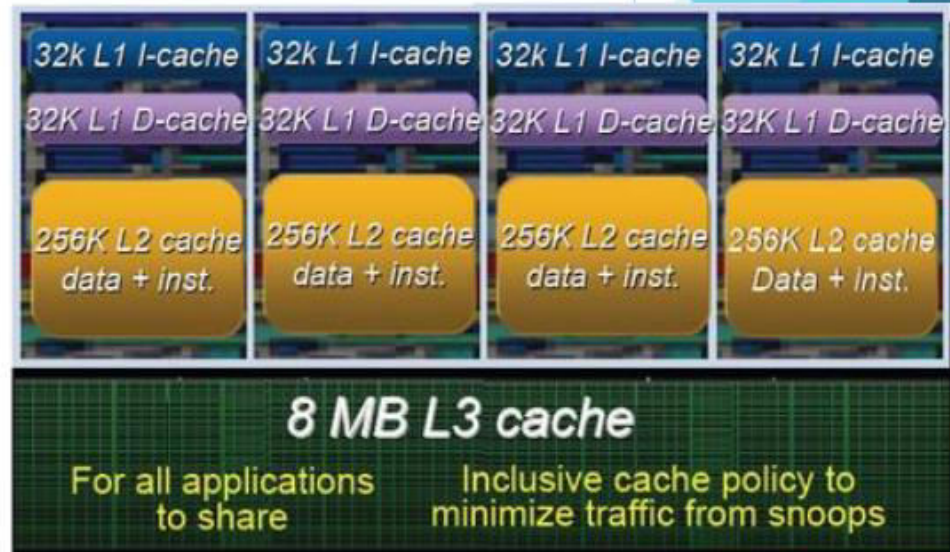
The essence of all cache designs is:

- Balancing fast hits and few misses.
- For second level caches,
 - Many fewer hits than in the first level cache.
 - Emphasis shifts to fewer misses.
- Insight leads to much larger caches and techniques to lower the miss rate:
 - Such as higher associativity & larger blocks.

EXTRA

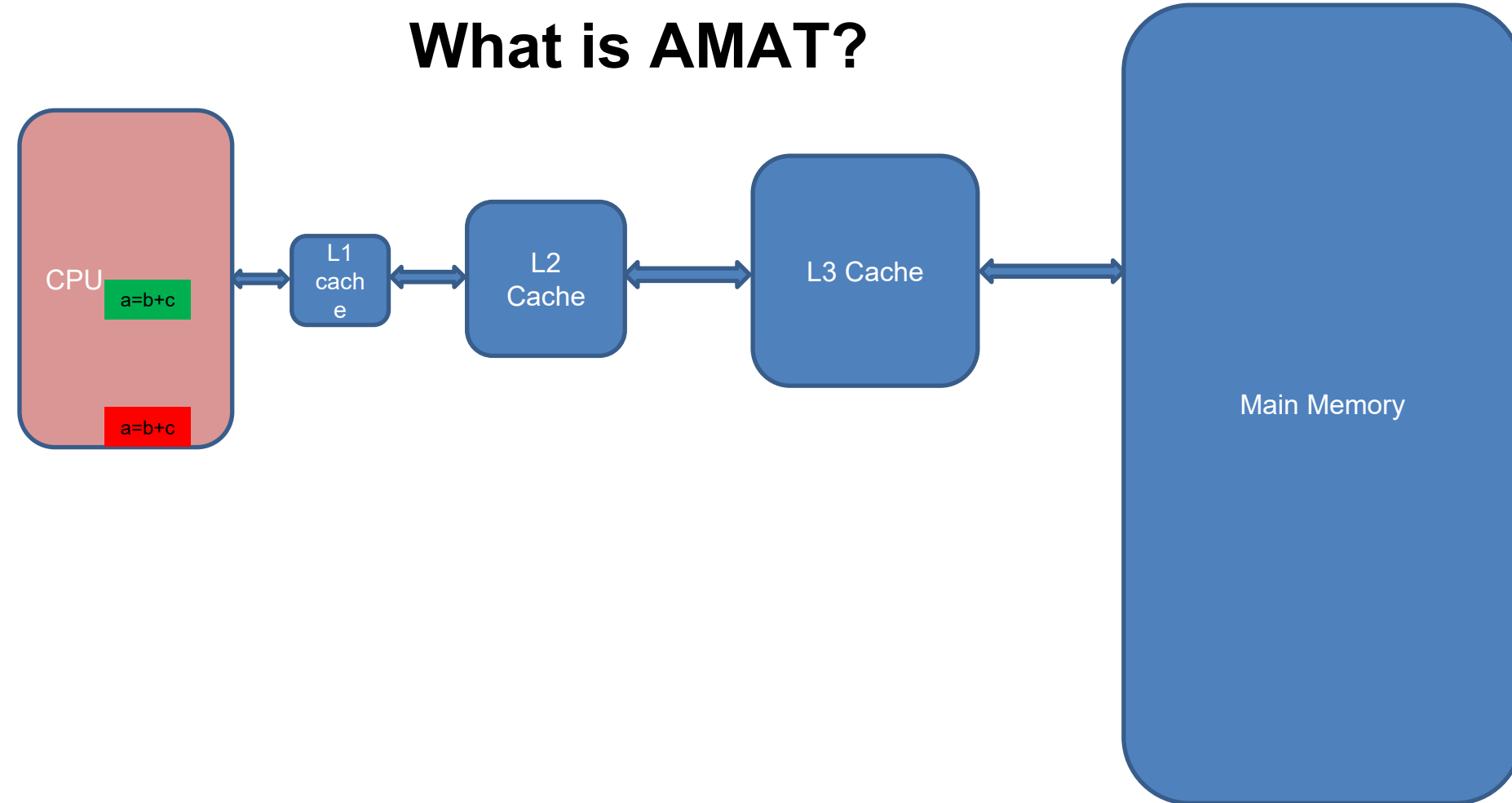
Intel Core i7 Caching Basics

- ▶ Intel core i7 processors feature three levels of caching.
 - ▶ Separate L1 and L2 cache for each core.
 - ▶ L1 cache broken up into to halves, instruction/data.
 - ▶ L3 cache shared among all cores and is inclusive.



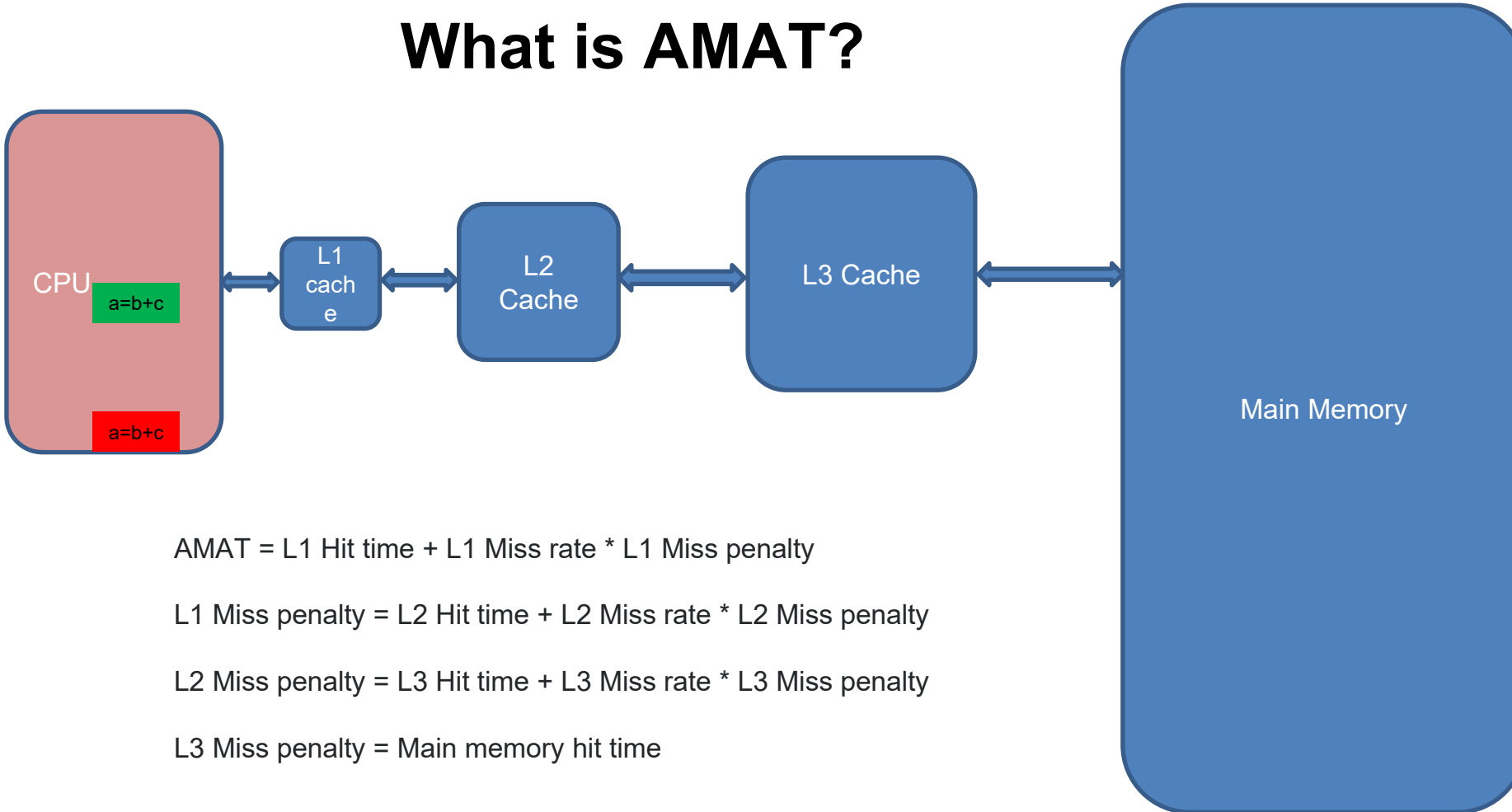
Think About it?

What is AMAT?



Think About it?

What is AMAT?



$AMAT = L1 \text{ Hit time} + L1 \text{ Miss rate} * L1 \text{ Miss penalty}$

$L1 \text{ Miss penalty} = L2 \text{ Hit time} + L2 \text{ Miss rate} * L2 \text{ Miss penalty}$

$L2 \text{ Miss penalty} = L3 \text{ Hit time} + L3 \text{ Miss rate} * L3 \text{ Miss penalty}$

$L3 \text{ Miss penalty} = \text{Main memory hit time}$

Prioritizes reads over writes

- This optimization serves reads before writes have been completed.
- Complexities of a write buffer with a **write-through cache**:
 - The most improvement is a write buffer of a proper size.
 - Memory accesses are complicated as it may hold the updated value of a location needed on a read miss.

To resolve this ambiguity,

- Read miss to wait until the write buffer is empty.
or
- To check the contents of the write buffer on a read miss and if there are no conflicts and the memory system is available.
- Let the read miss continue.
- Virtually, all processors use the later approach.
- Gives priority reads over writes.

Prioritizes reads over writes

Write-back cache:

- The cost of the writes by the processor in a write back can also be reduced.
- Consider a read miss replacing a dirty block.
- Instead of writing the dirty block to the memory, and then reading memory, we could copy the dirty block to a buffer, then read memory and then write memory will finish sooner.
- Thus, if a read miss occurs, the processor can either stall until the buffer is empty or
- Check the addresses of the words in the buffer for conflicts.

5th Optimization: Giving priority to Read Misses over Write Misses

Case 1: Write Back Policy

Scenario

- **P is referred by the Processor.**
- **P Should replace A.**
- **A is Dirty**

Known Solution

- **Write A back in Memory**
- **Replace P by A**

Cache			
Valid Bit	Dirty Bit	TAG	Data
1	1	10100	A=11
1	0	10100	B=5
1	0	10100	C=6

Main Memory

10100.....1010..00 00	A= 0
10100.....1010..00 01	B=5
10100.....1010..00 10	C=6
00010.....1010..00 00	P=100

Optimized Solution (Give Priority for CPU Reference / Read)

- **Place A on to buffer to save time.**
- **Replace P by A, to provide quick access .**
- **Write A into memory parallely when Processor is using P.**

5th Optimization: Giving priority to Read Misses over Write Misses

Case 2: Write Through

Scenario

- **P** is referred by the Processor.
- **P** Should replace **A**.

Cache		
Valid Bit	TAG	Data
1	10100	A=11
1	10100	B=5
1	10100	C=6

		A=11
--	--	------

Write Buffer

Main Memory

10100.....1010..00 00	A= 0
10100.....1010..00 01	B=5
10100.....1010..00 10	C=6
00010.....1010..00 00	P=100

5th Optimization: Giving priority to Read Misses over Write Misses

Case 2: Write Through

Scenario

- **P** is referred by the Processor.
- **P** Should replace **A**.

Known Solution

- **Place A on the Write Buffer**
- **Replace P by A**

What may go wrong?
If A is referred again!

Cache		
Valid Bit	TAG	Data
1	00010	P=100
1	10100	B=5
1	10100	C=6
		A=11

Write Buffer

Main Memory

10100.....1010..00 00	A= 0
10100.....1010..00 01	B=5
10100.....1010..00 10	C=6
00010.....1010..00 00	P=100

Solution1: Wait till A is written back in Memory

Solution2: (Give Priority for CPU Reference / Read)

- Search for A in buffer, if found compare A in Memory,
- i. if both are same, Replace.
 - ii. Otherwise, read from Buffer

Prioritizes reads over writes

Consider the following code sequence.

Ex: SW R3, 512 (R0) // Store R3 into memory location 512

LW R1, 1024 (R0) // Load from memory location 1024 into R1

LW R2, 512 (R0) // Load from memory location 512 into R2

Assume Direct Mapped Cache

Write-through cache that maps 512 and 1024 to the same block. Four word write buffer that is not checked on a read miss. Will the value in R2 always be equal to the value in R3

Prioritizes reads over writes

Consider the following code sequence.

Ex: **SW R3, 512 (R0) // Store R3 into memory location 512**

LW R1, 1024 (R0) // Load from memory location 1024 into R1

LW R2, 512 (R0) // Load from memory location 512 into R2

Assume Direct Mapped Cache

Write-through cache that maps 512 and 1024 to the same block. Four word write buffer that is not checked on a read miss. Will the value in R2 always be equal to the value in R3? Ans:

- This is a read-after-write data hazard in memory.
- The data in R3 are placed into the write buffer after the STR.
- The following LDR instruction uses the same cache index and is therefore a miss.
- The second LDR instruction, tries to put the value in location 512 into the register R2.
- This also results in a miss.

- If the write buffer hasn't completed writing to location 512 in memory,
- The read of location 512 will put the old, wrong value into the cache block and then into R2.
- Without proper precautions, R3 would not be equal to R2!

1. In a two-level cache system, how is the "Local Miss Rate" of the L2 cache defined?

- A) The total misses in L2 divided by the total memory accesses generated by the processor.
- B) The number of misses in L2 divided by the total number of memory accesses to the L2 cache.
- C) The hits in L1 divided by the misses in L2.
- D) The number of misses in L1 divided by the total memory accesses.

2. How is the "Global Miss Rate" for an L2 cache calculated?

- A) Miss Rate of L1 $\times$ Miss Rate of L2
- B) Miss Rate of L1 + Miss Rate of L2
- C) Miss Rate of L2 / Miss Rate of L1
- D) Miss Penalty of L1 $\times$ Miss Rate of L2

1. In a two-level cache system, how is the "Local Miss Rate" of the L2 cache defined?

- A) The total misses in L2 divided by the total memory accesses generated by the processor.
- B) The number of misses in L2 divided by the total number of memory accesses to the L2 cache.
- C) The hits in L1 divided by the misses in L2.
- D) The number of misses in L1 divided by the total memory accesses.

Answer: B

2. How is the "Global Miss Rate" for an L2 cache calculated?

- A) Miss Rate of L1 $\times$ Miss Rate of L2
- B) Miss Rate of L1 + Miss Rate of L2
- C) Miss Rate of L2 / Miss Rate of L1
- D) Miss Penalty of L1 $\times$ Miss Rate of L2

Answer: A

3. When giving priority to read misses over writes (Optimization 5), what does the processor typically do to prevent a read-after-write data hazard?

- A) It forces the read miss to wait until the entire cache is flushed.
- B) It ignores the write buffer completely and reads directly from main memory.
- C) It checks the addresses in the write buffer for conflicts; if none exist, the read miss continues.
- D) It drops the read operation entirely and retries the instruction.

4. Why is a multilevel cache architecture primarily used?

- A) To slow down the processor so memory can catch up.
- B) To have a small, fast L1 cache to match processor speed and a large L2 cache to capture accesses and reduce the miss penalty.
- C) To eliminate compulsory misses completely.
- D) To replace the need for virtual memory.

3. When giving priority to read misses over writes (Optimization 5), what does the processor typically do to prevent a read-after-write data hazard?

- A) It forces the read miss to wait until the entire cache is flushed.
- B) It ignores the write buffer completely and reads directly from main memory.
- C) It checks the addresses in the write buffer for conflicts; if none exist, the read miss continues.
- D) It drops the read operation entirely and retries the instruction.

Answer: C

4. Why is a multilevel cache architecture primarily used?

- A) To slow down the processor so memory can catch up.
- B) To have a small, fast L1 cache to match processor speed and a large L2 cache to capture accesses and reduce the miss penalty.
- C) To eliminate compulsory misses completely.
- D) To replace the need for virtual memory.

Answer: B



THANK YOU



Team MPCA

Department of Computer Science and
Engineering

Microprocessor & Computer Architecture (μ pCA)



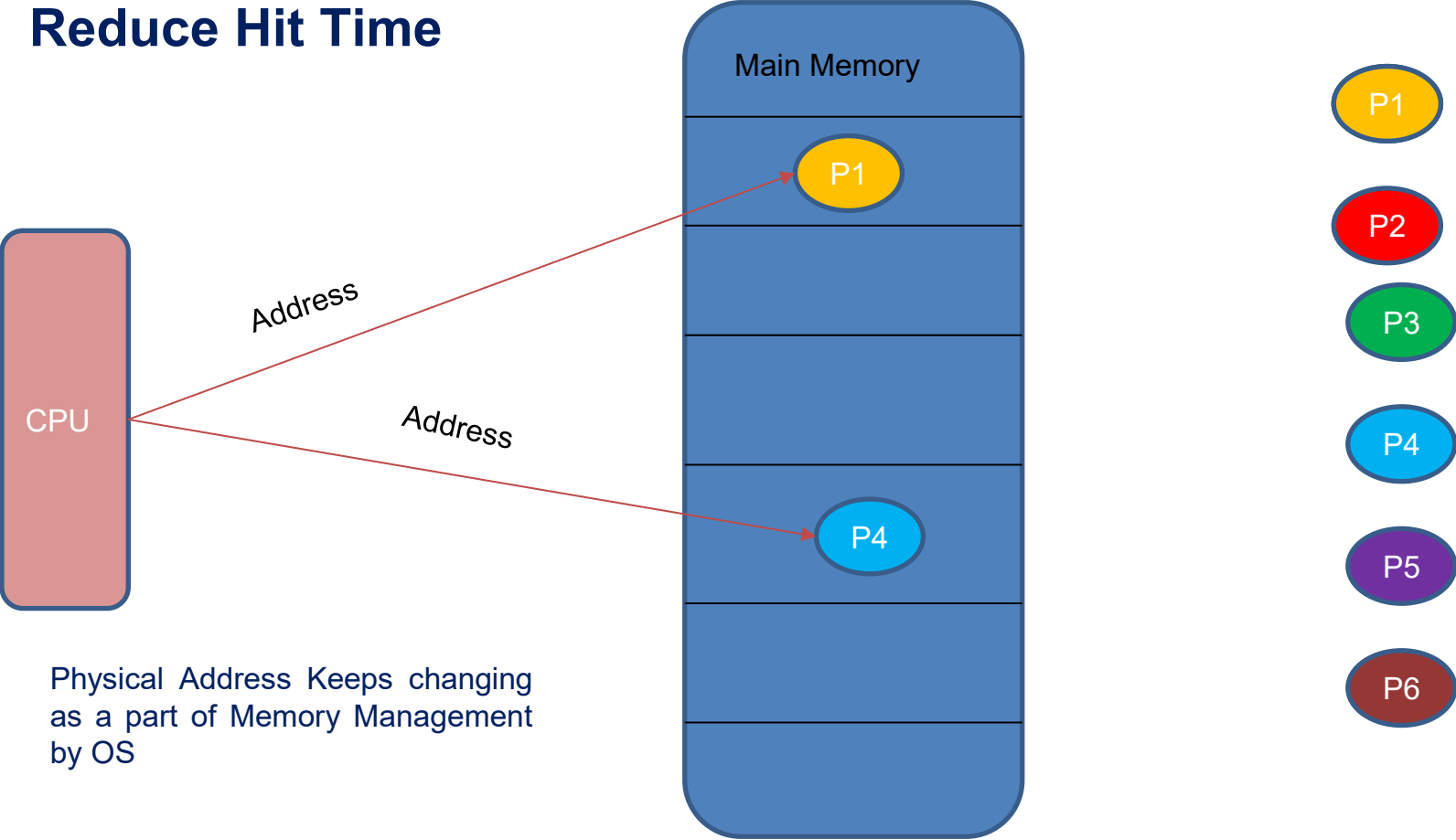
UE24CS251B

Unit 3: Cache - 6th OPTIMIZATION

Session 3.10

6th Optimization: Reduce Hit Time

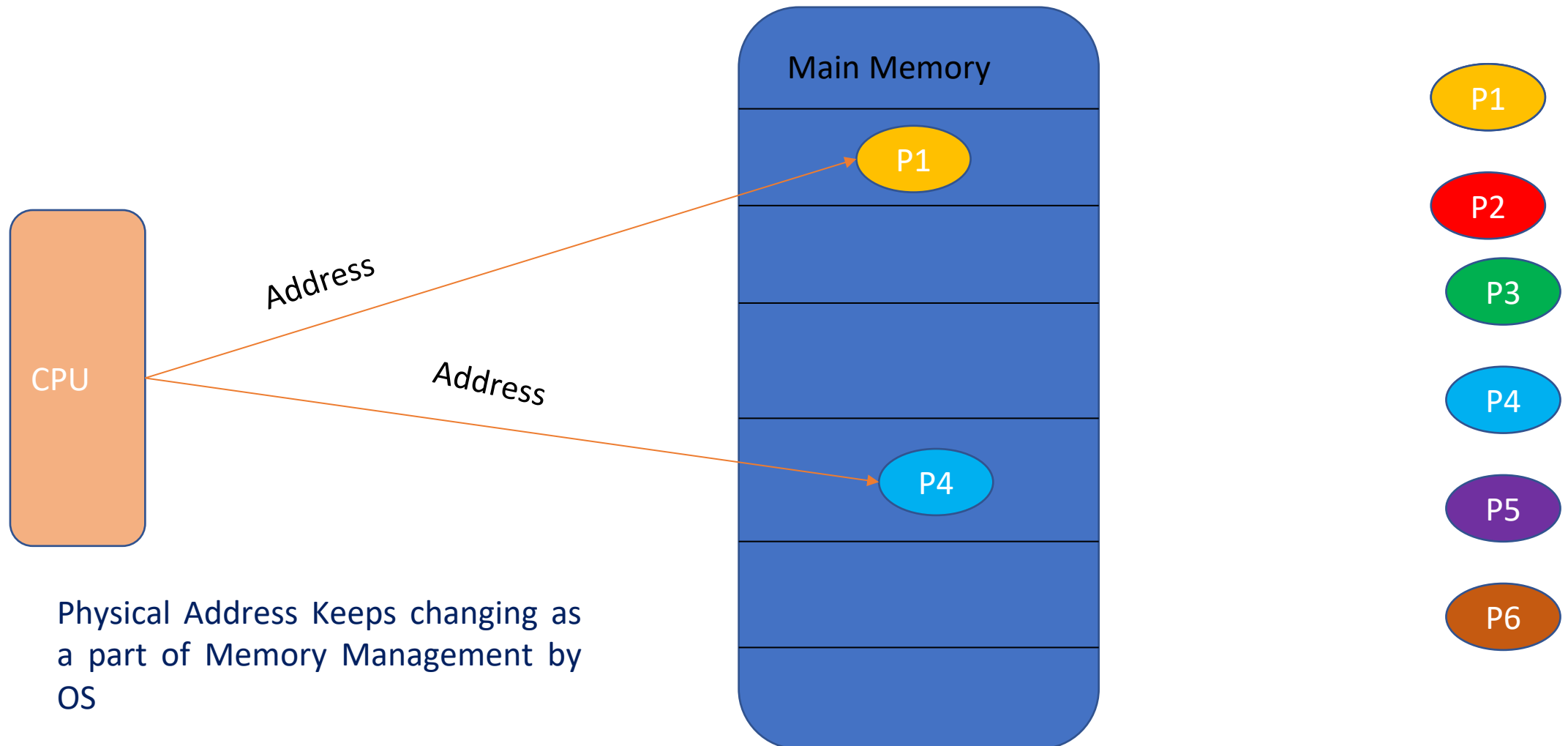
Avoid Address Translation In Cache Indexing To Reduce Hit Time



Physical Address Keeps changing as a part of Memory Management by OS

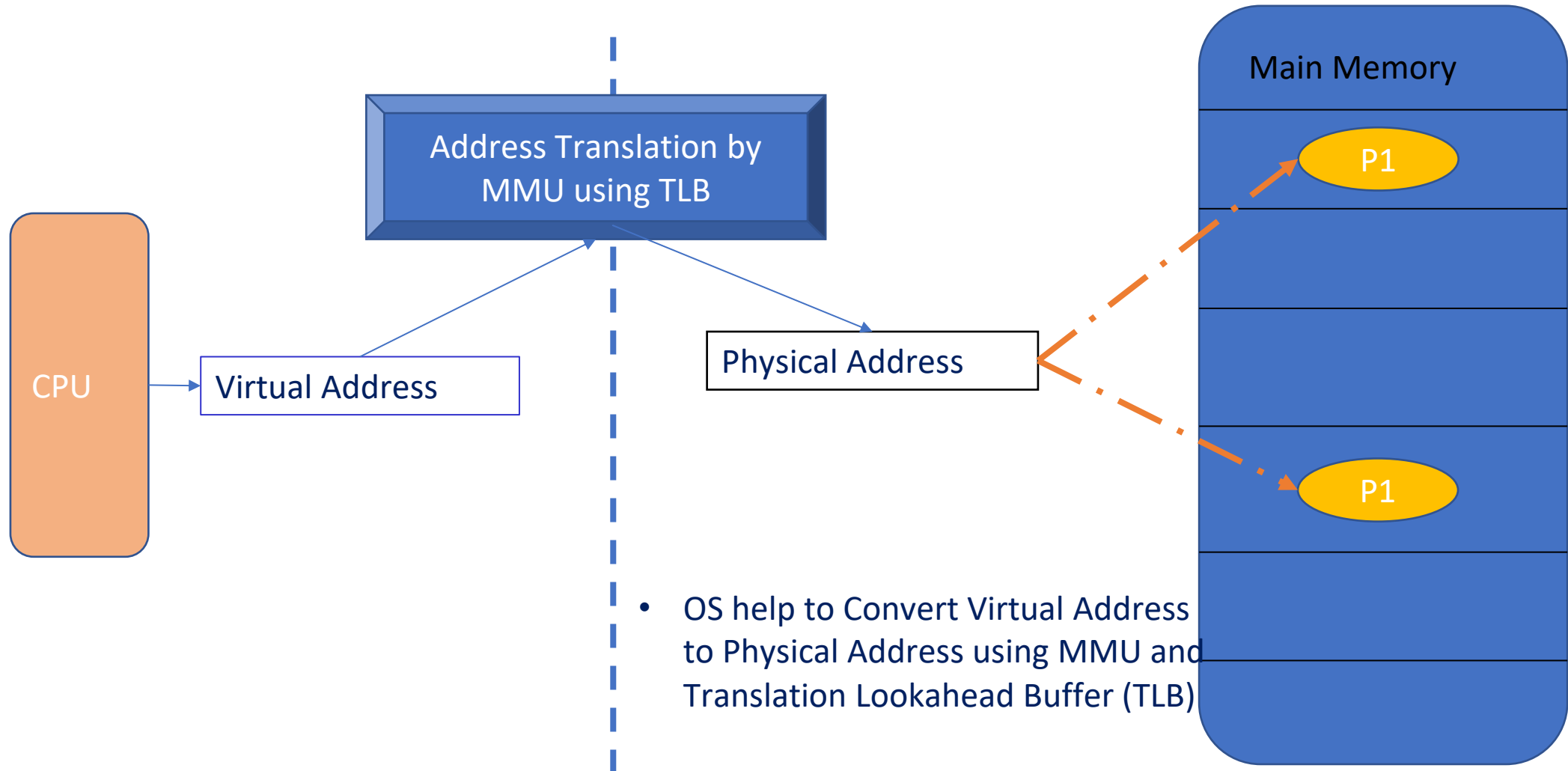
6th Optimization: Reduce Miss Time

Avoid Address Translation In Cache Indexing To Reduce Hit Time



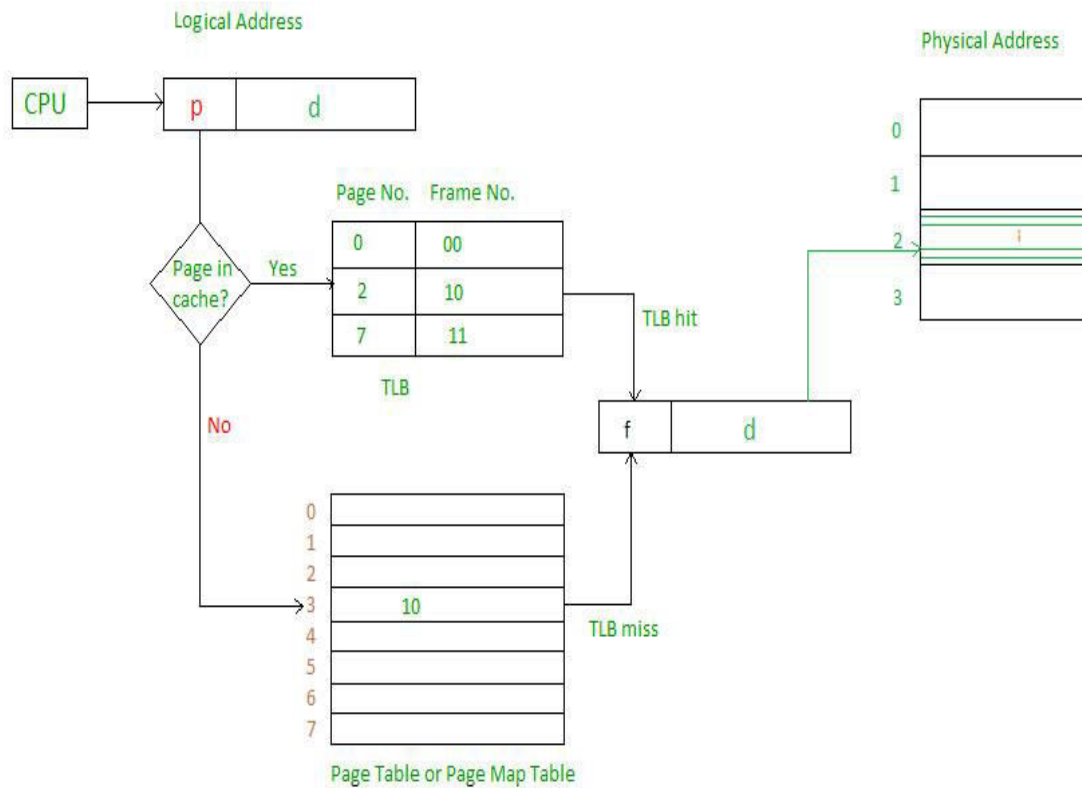
6th Optimization: Reduce Miss Time

Avoid Address Translation In Cache Indexing To Reduce Hit Time



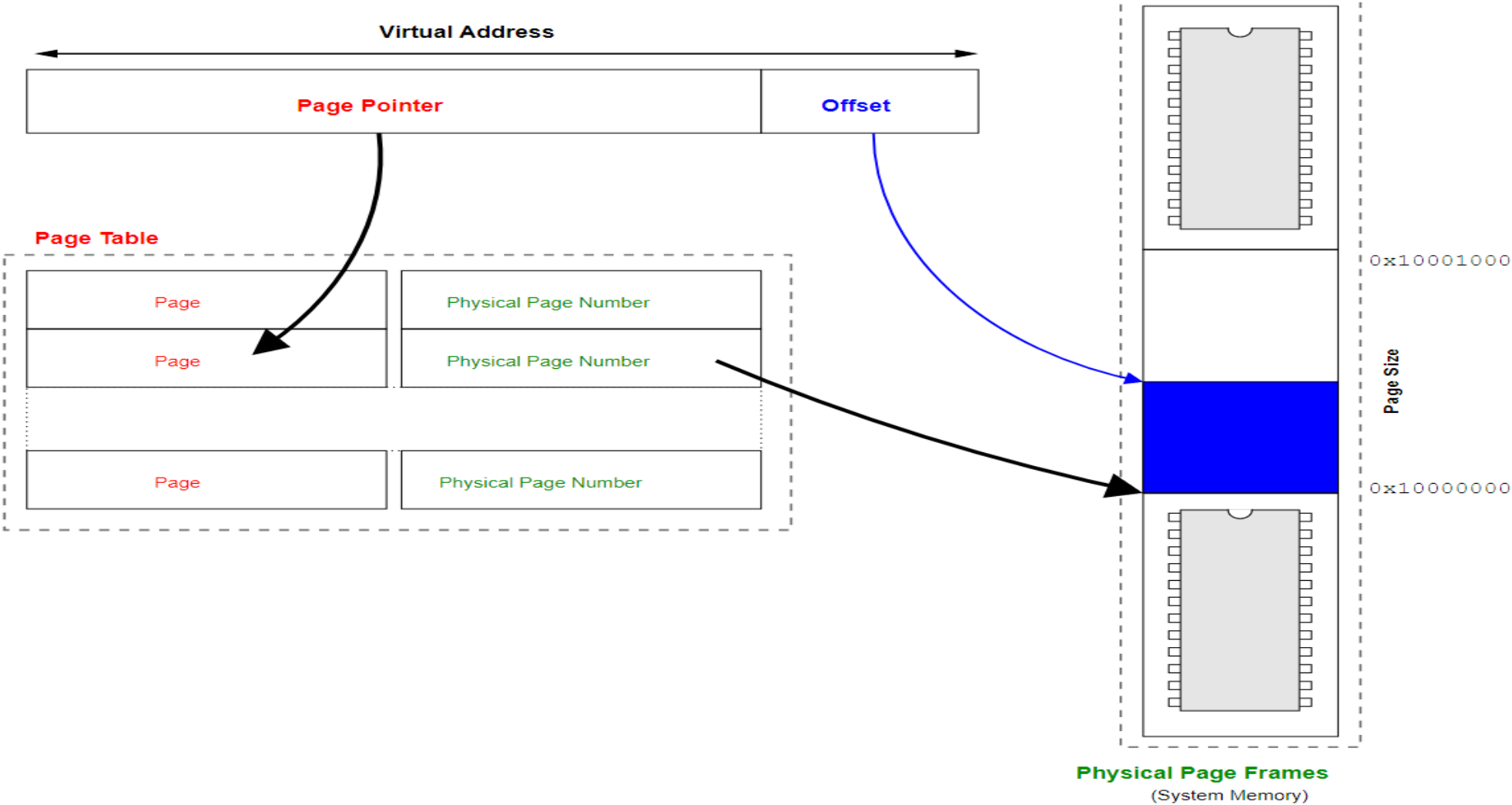
EXTRA Basic concept

Translation Lookaside Buffer (TLB) is nothing but a special cache used to keep track of recently used transactions.

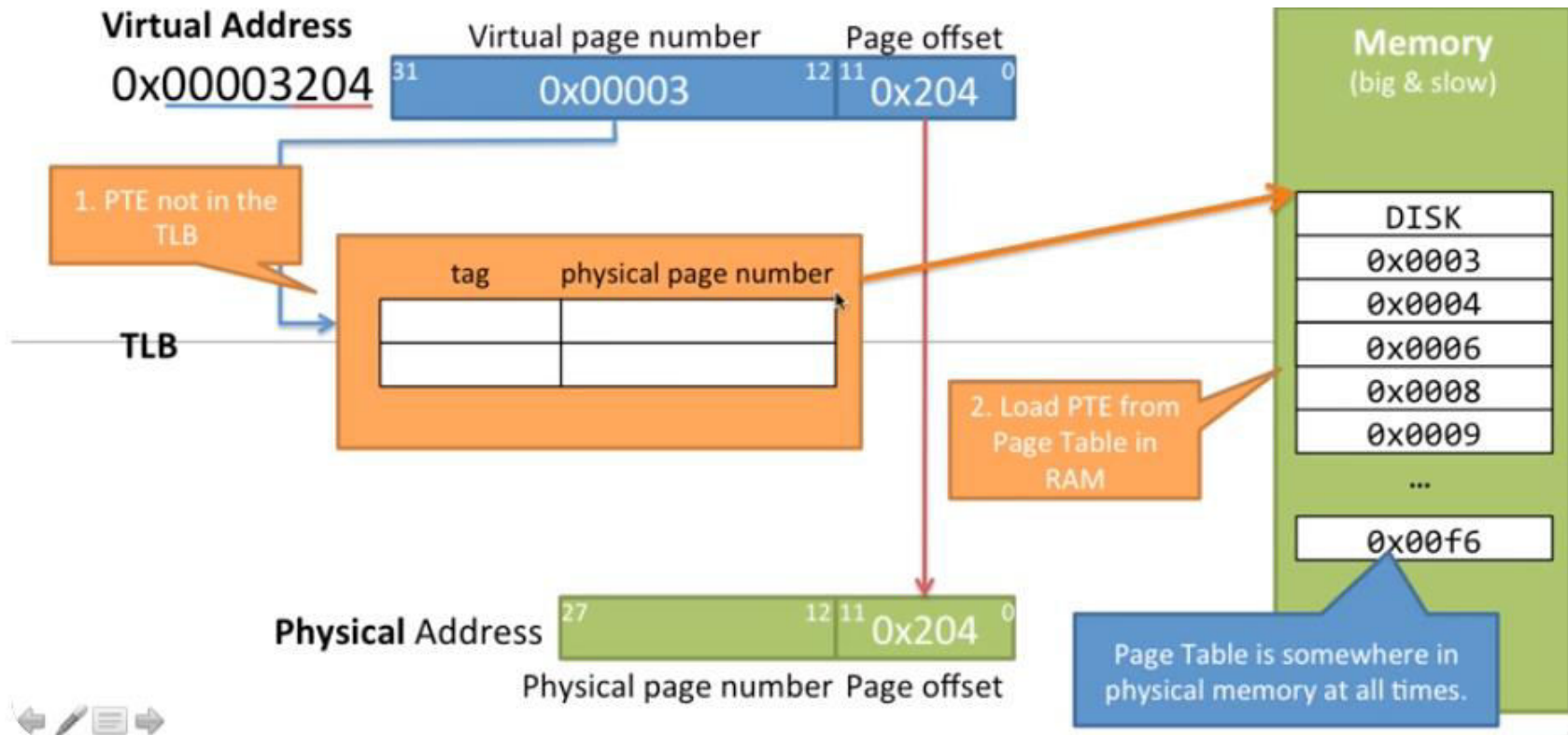


- 1.CPU generates virtual (logical) address.
- 2.It is checked in TLB (present).
- 3.Corresponding frame number is retrieved, which now tells where in the main memory page lies.

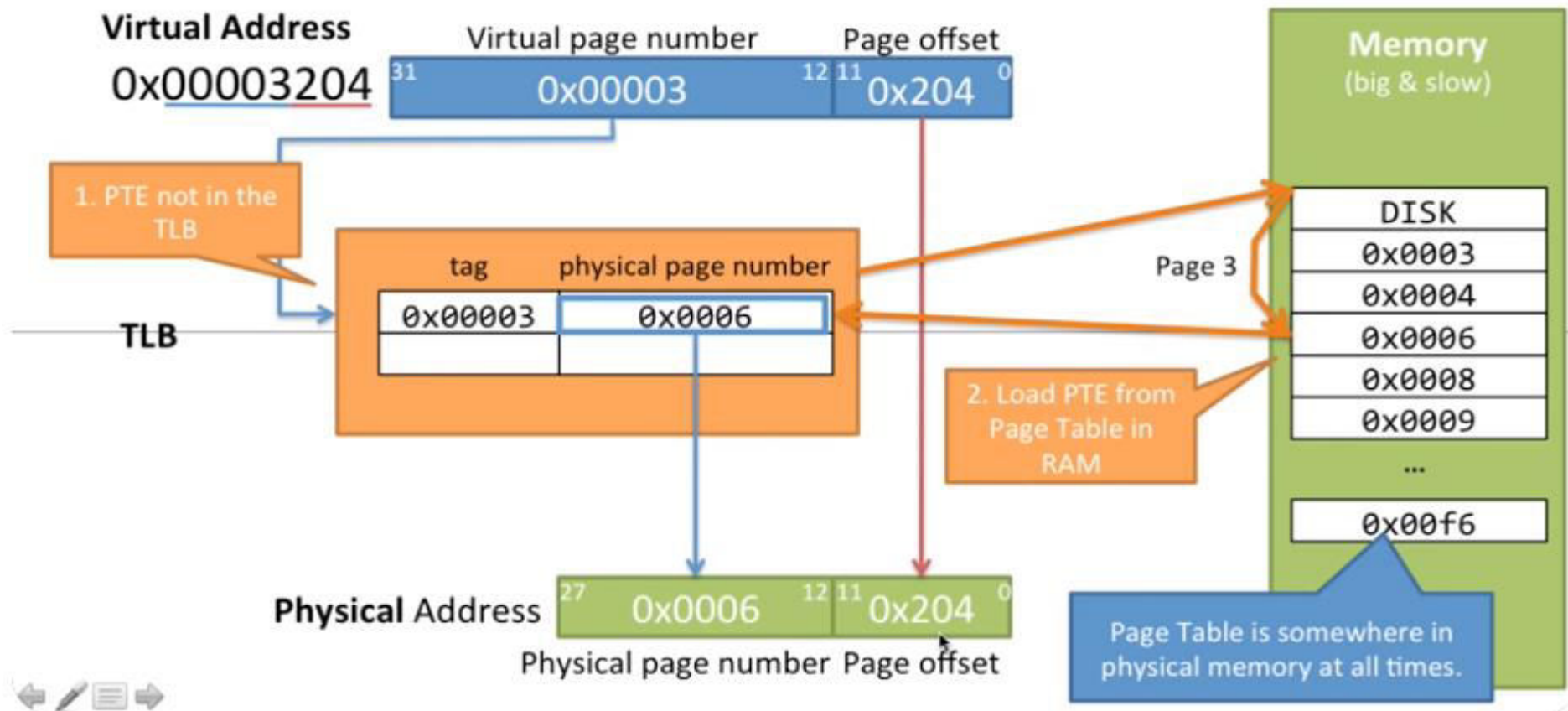
EXTRA Basic concept



EXTRA Working of TLB



EXTRA_Working of TLB



Sixth Optimization: Avoiding Address Translation during indexing of the Cache to Reduce Hit Time

- The CPU uses virtual addresses that must be mapped to a physical address.
- Cache must cope with the translation of a virtual address to physical address to access memory.
- The addresses of the programme on disk are virtual addresses. They must be translated to a physical address in order to be placed in memory.
- Since, processor treats main memory as just another memory in hierarchy, the address of the virtual memory that exists on the disk must be mapped on to the main memory.

Disk will have largest space. Disk_virtual address

The entire program will not fit in main memory, so we go for loading part of program from disk to main memory

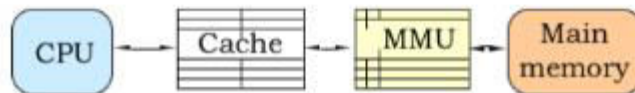
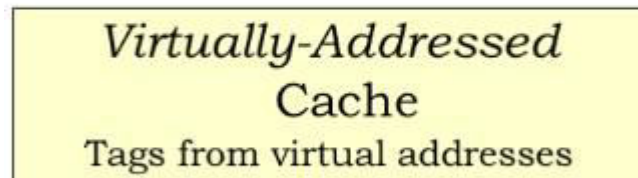
Sixth Optimization: Avoiding Address Translation during indexing of the Cache to Reduce Hit Time

The cache may either use virtual or physical addresses.

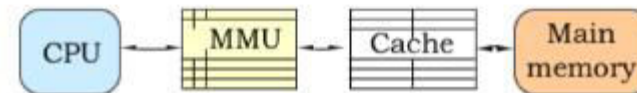
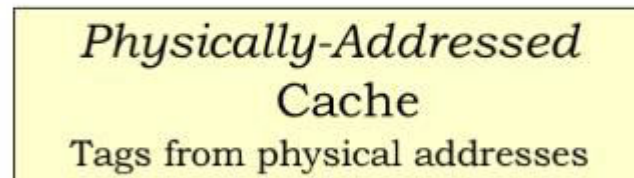
Virtual Cache

- A cache that indexes by virtual addresses is called a virtual cache , as opposed to a physical cache .
- In virtual cache, hits are common than misses. **A virtual cache reduces hit time since a translation from a virtual address to a physical address is not necessary on hits.**
- Also, address translation can be done in parallel with cache access, **so penalties for misses** are reduced as well.

Using Caches with Virtual Memory



- FAST: No MMU time on HIT
- Problem: Must flush cache after context switch



- Avoids stale cache data after context switch
- SLOW: MMU time on HIT

Virtual cache and physical cache

Virtual cache doesn't involve address translation

Cache has indexing and tagging-whether to use VA or PA to index Cache.

Why not use virtually addressed cache always?

Checking of page level protection is done in address translation which cant be enforced if no conversion from virtual to physical address. (copy info from TLB on miss)

- Every time process is switched VA refers to different PA, requires cache flush(uses- Process ID)
- 2 VA refer to same PA(aliasing)
- I/O uses PA

What is the Problem?

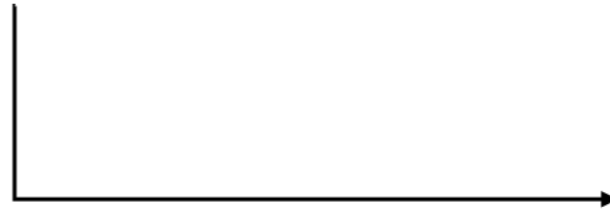
Accessing data from Cache involve:

- Indexing
- Tagging

EXTRA-Indexing and Tagging

3. Read from address 1010000001100000

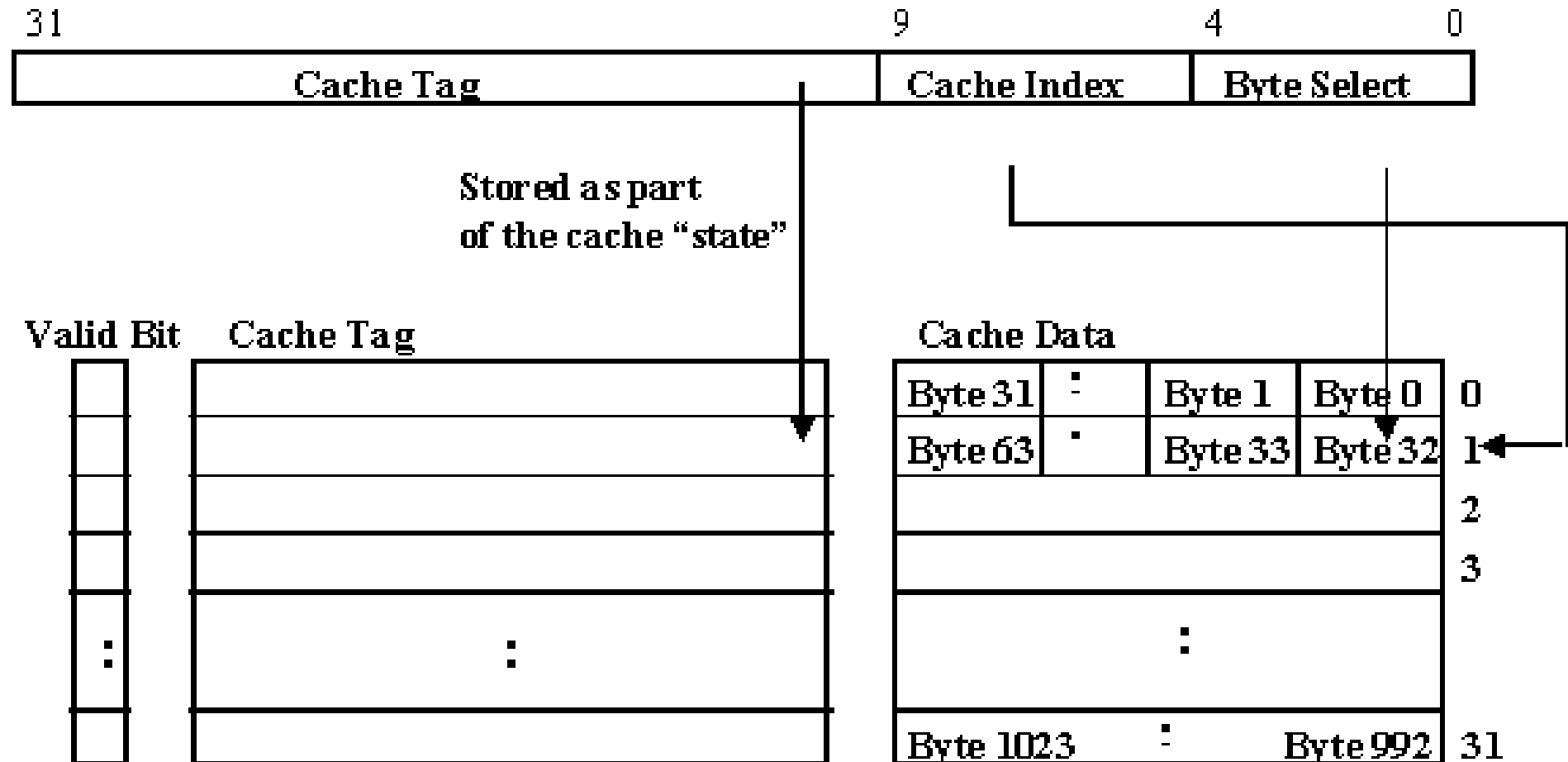
1010 0000011 00000
Tag Index Offset



Result: hit, the line is valid,
and the tag matches.

Direct-Mapped Cache				
Line #	V	D	Tag	Data (32-byte block)
0:	0	0		
1:	1	1	1111	
2:	0	0		
→ 3:	1	0	1010	
4:	0	0		
...				
127:	0	0		

EXTRA-Indexing and Tagging
for direct mapped



What is the Problem?

Accessing data from Cache involve:

- Indexing
- Tagging

scheme

PIPT-problem- CPU uses VA.

VIVT-problems-I/o, if miss get pA

What is the Problem?

Accessing data from Cache involve:

- Indexing
- Tagging

Converting Virtual Address to Physical Address take some time.

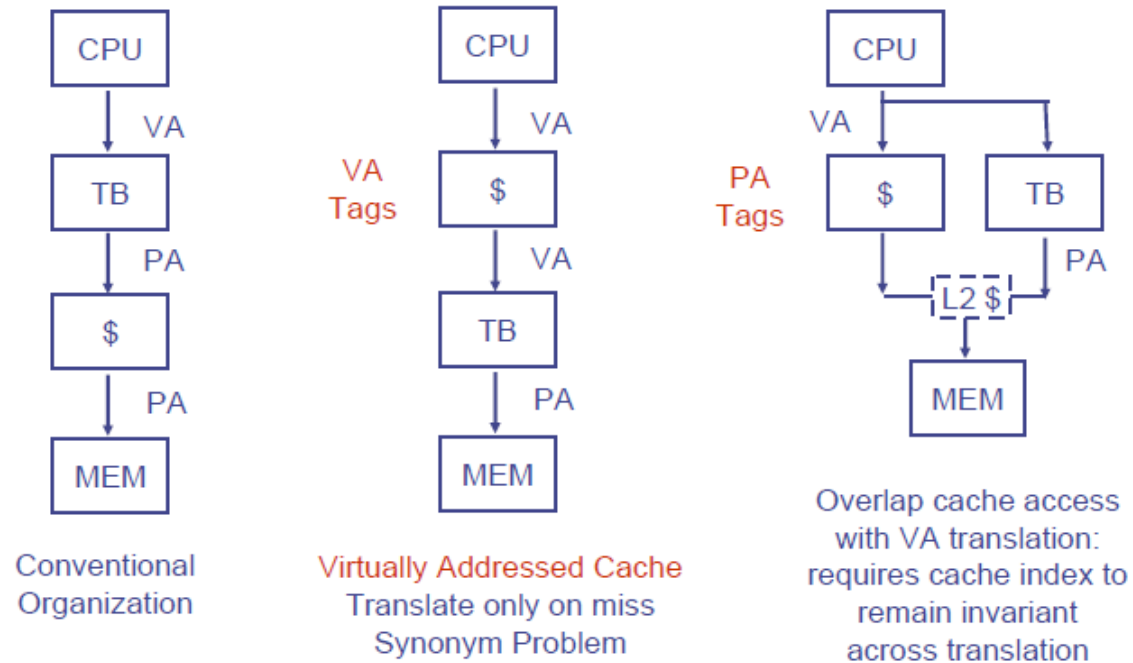
Solution 😊

Virtually Indexed & Physically Tagged

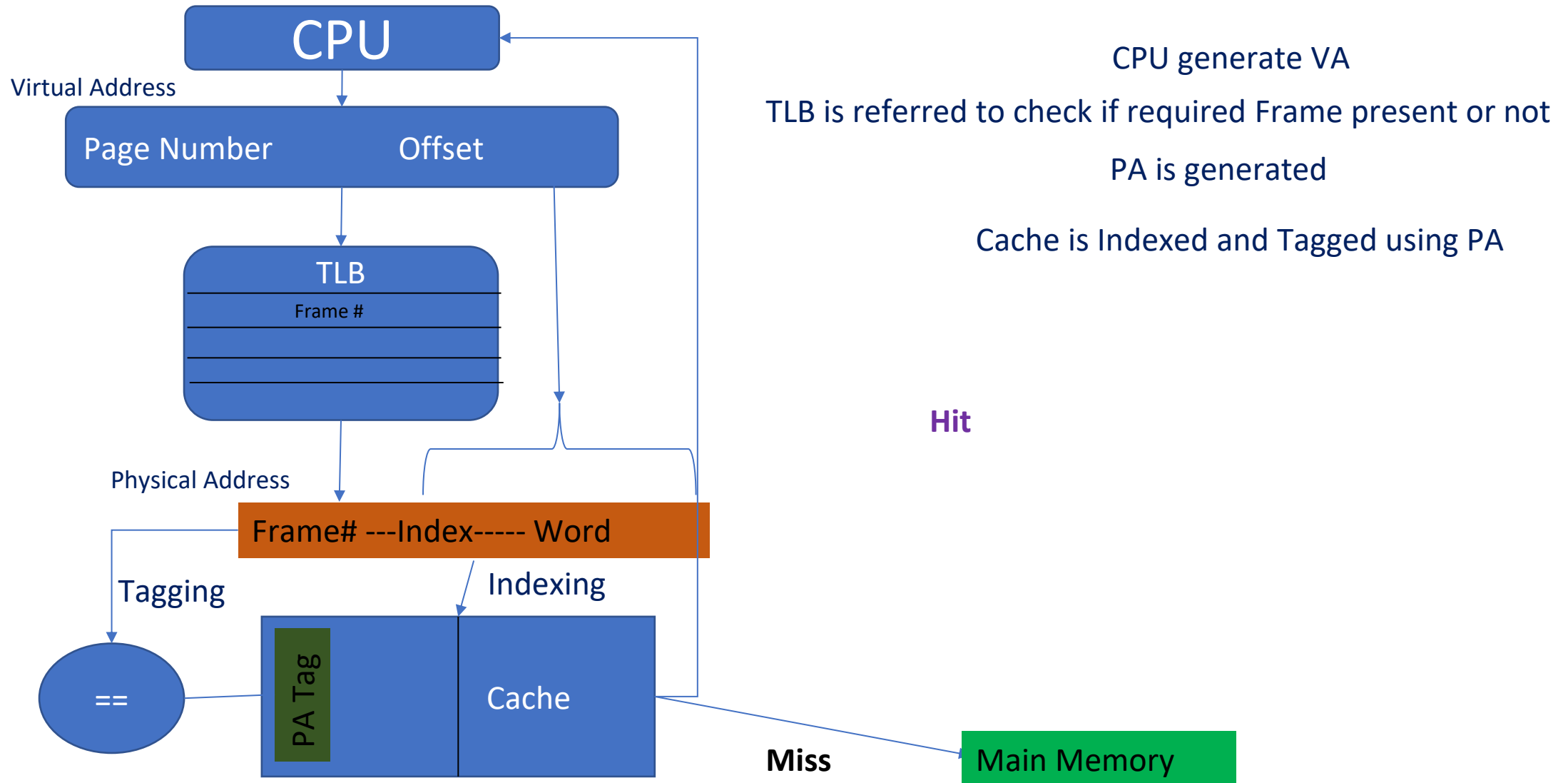
- Do not wait for VA to PA translation.
- Extract Index information from Virtual Address
- Extract Tag Information from the Physical Address.

Sixth Optimization: Avoiding Address Translation during indexing of the Cache to Reduce Hit Time

Fast hits by Avoiding Address Translation



Physically Indexed and Physically Tagged (PIPT)



EXTRA-Virtually Indexed and Physically Tagged

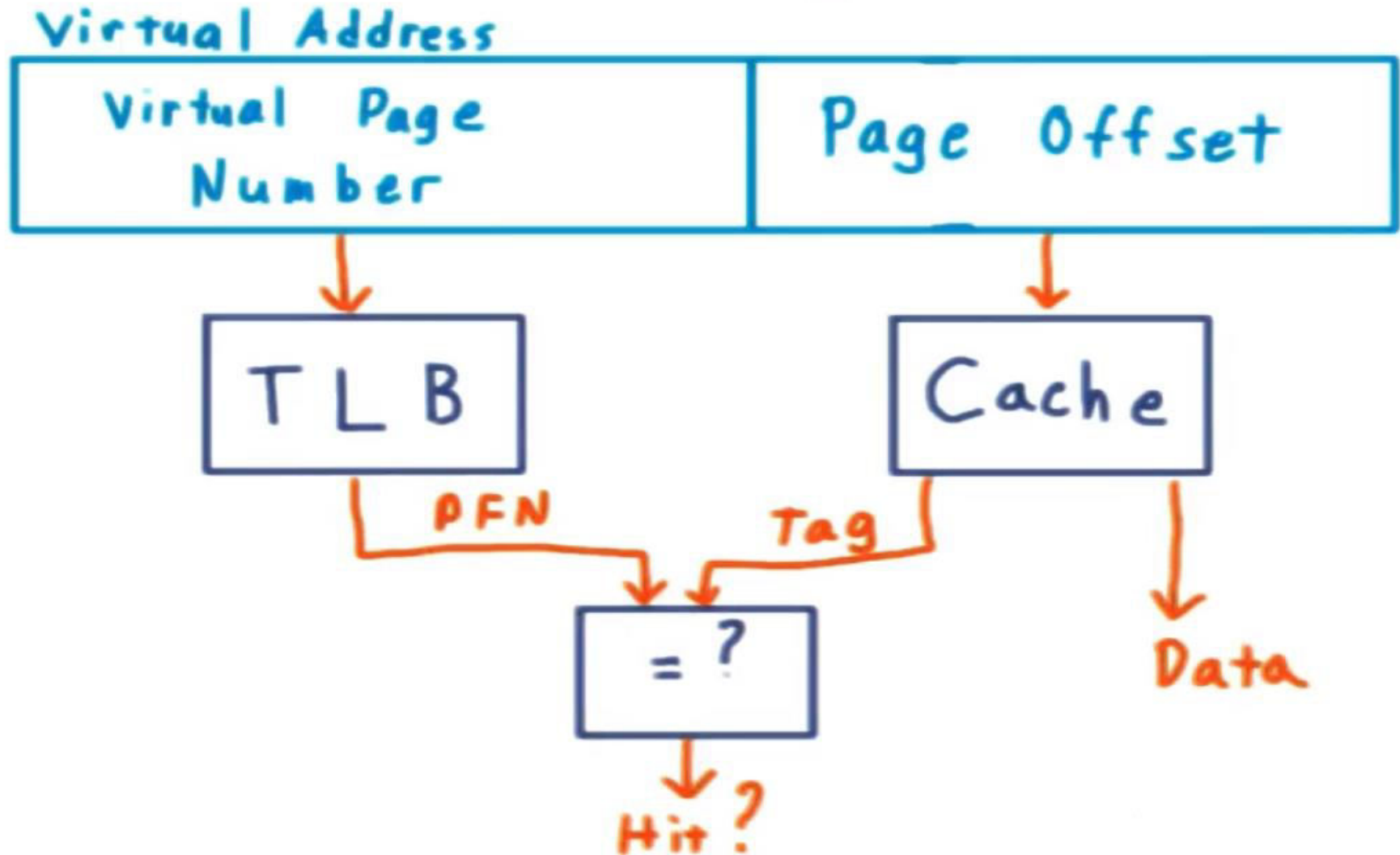
Virtual Address

Virtual Page Number	Page Offset
------------------------	-------------

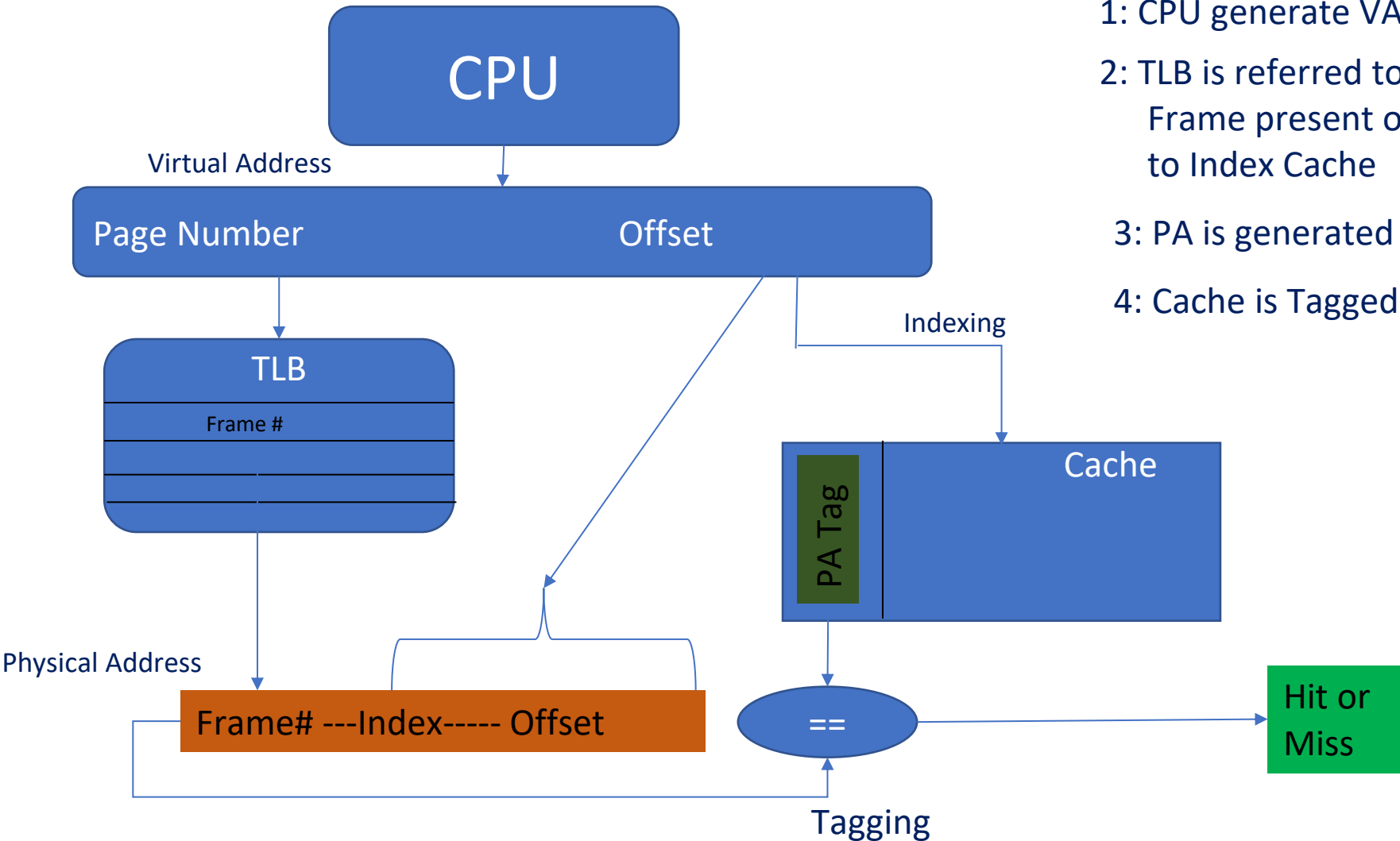
Physical Address

	Index	Cache Offset
--	-------	-----------------

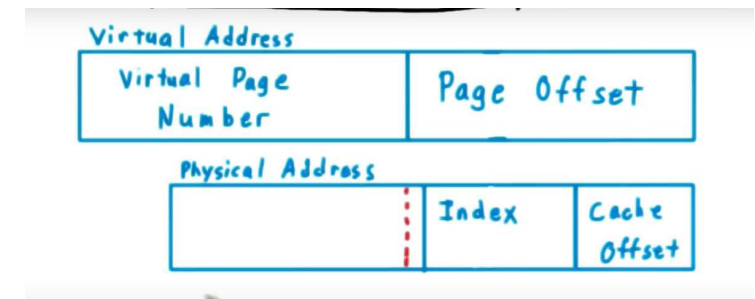
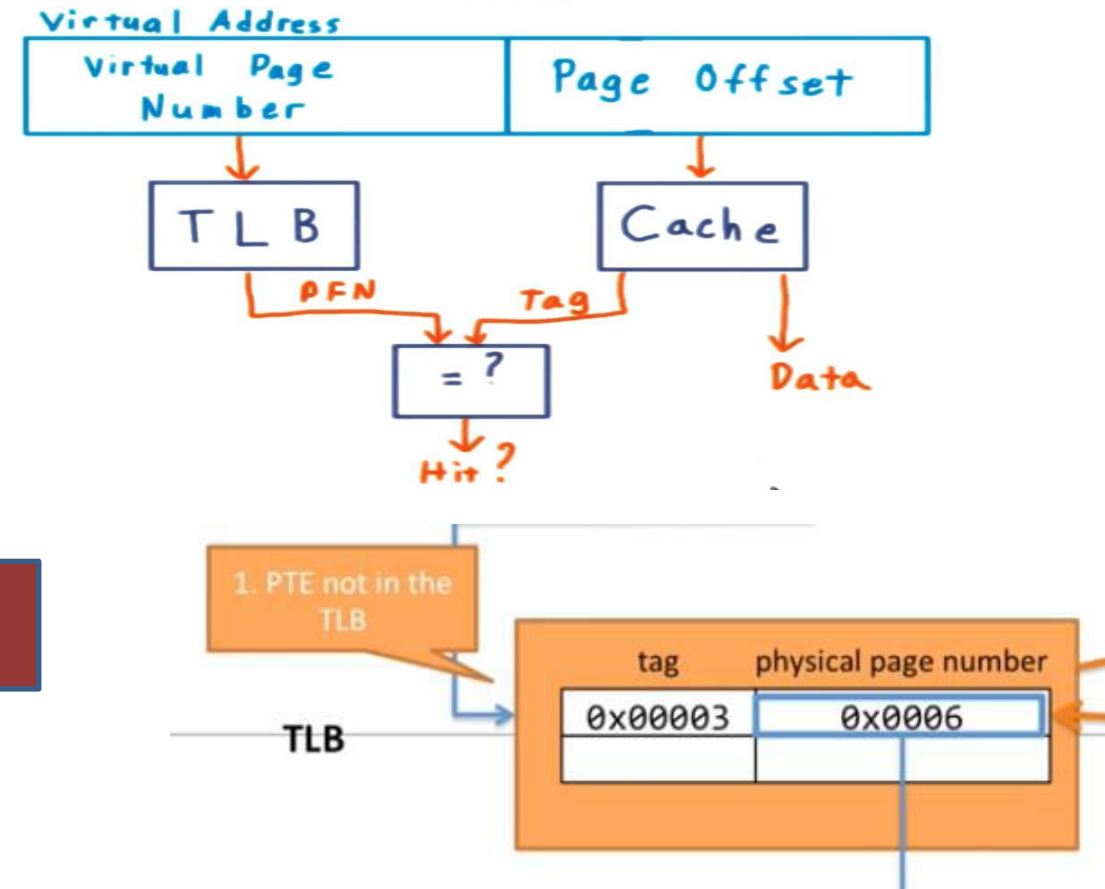
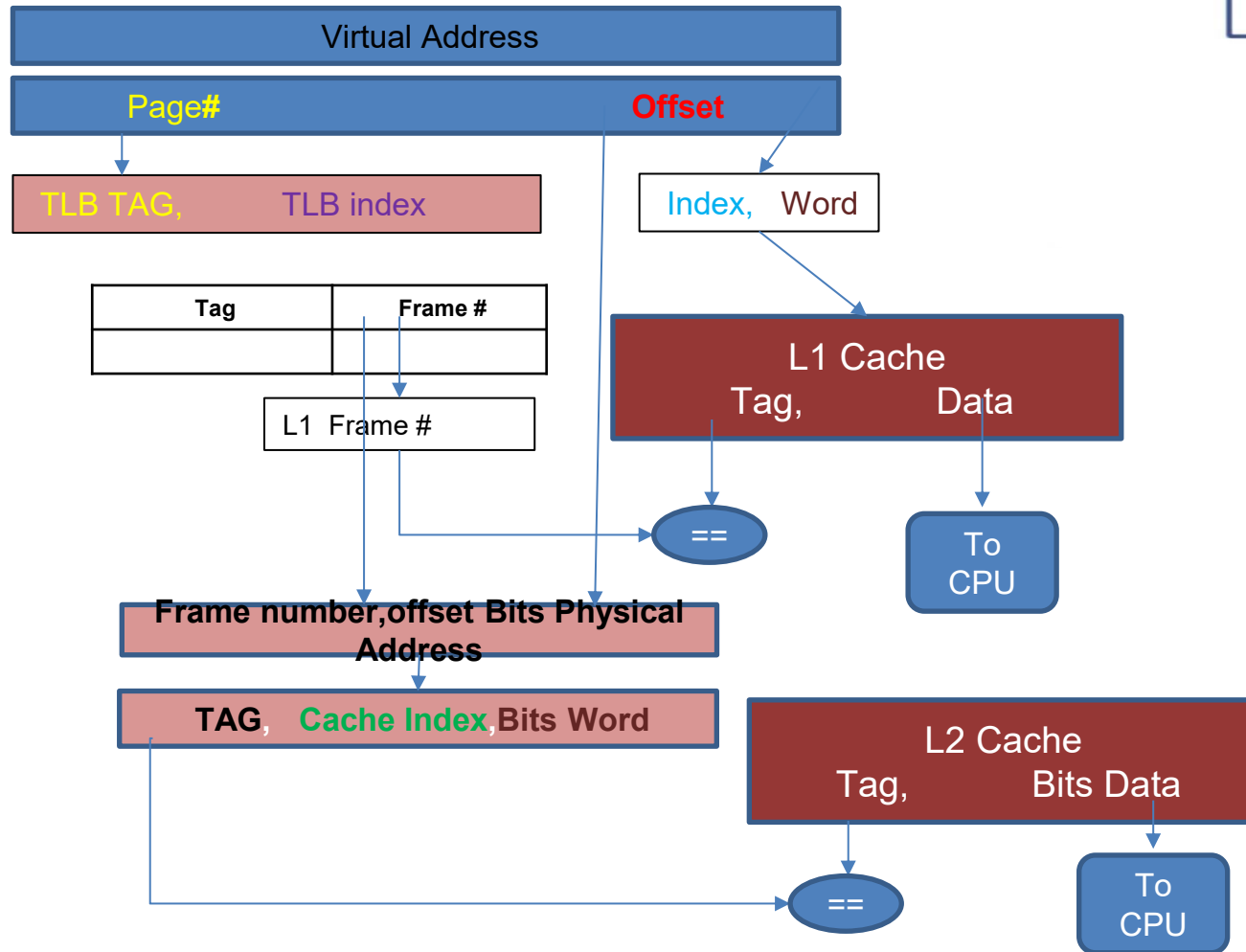
EXTRA-Virtually Indexed and Physically Tagged



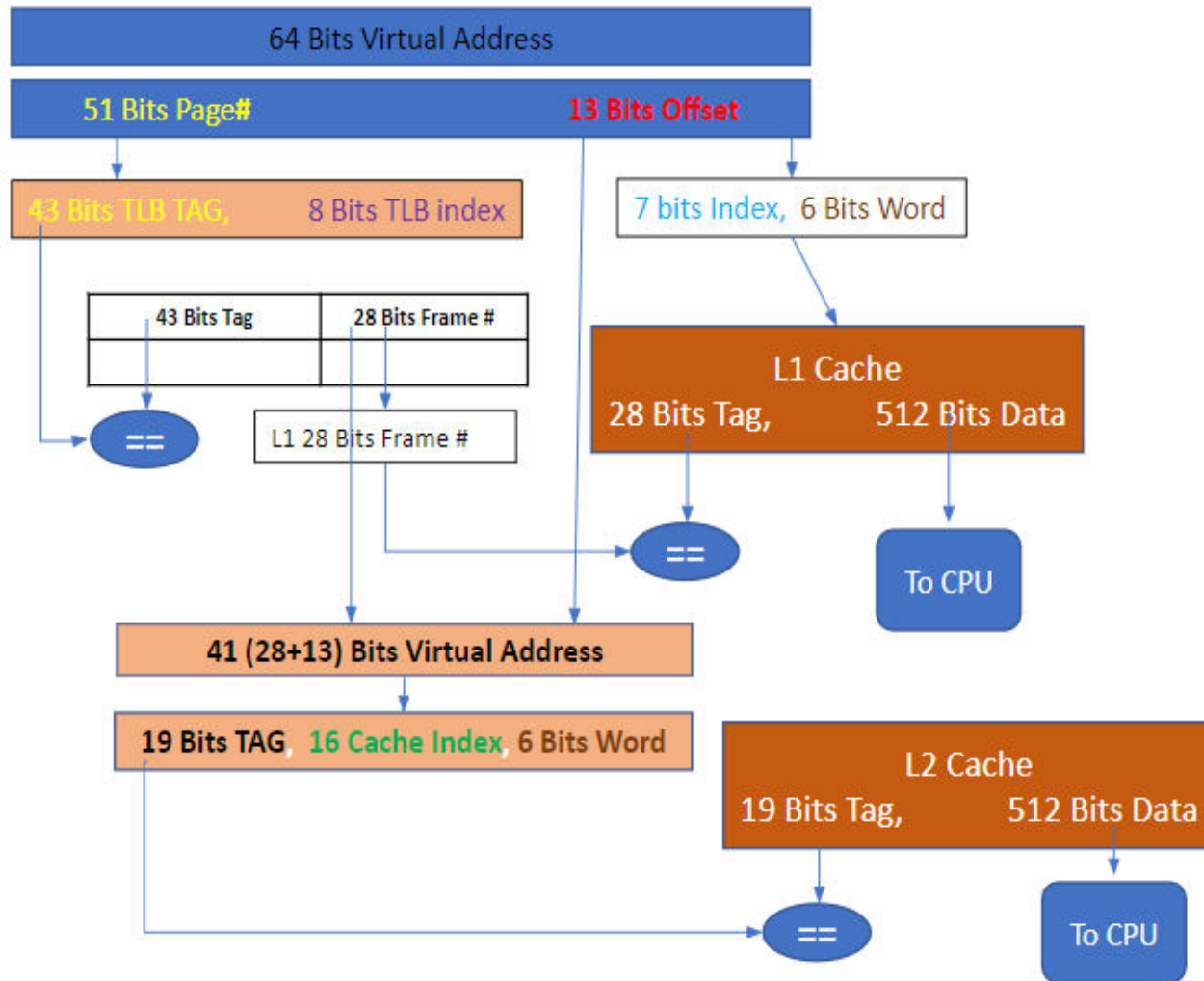
Virtually Indexed and Physically Tagged



6th Optimization : Case Study

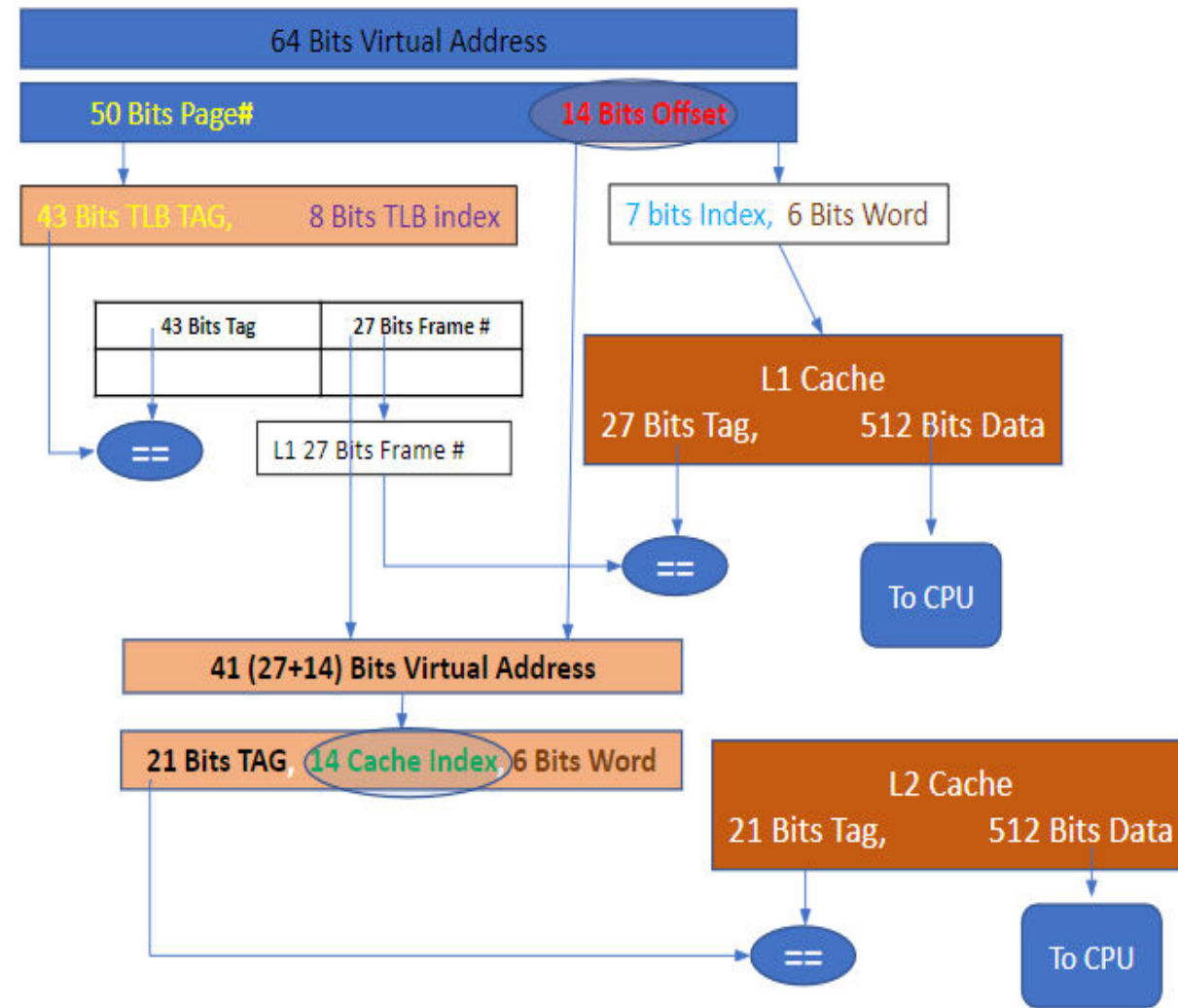


6th Optimization : Case Study



- Consider the following Specification
- 64 bit Virtual Address
- 41 bit Physical Address
- 8 KB (2^{13}) Page Size
- TLB is Direct Mapped with 256 Entries
- 8 KB (2^{13}) Direct Mapped L1 Cache
- Block Size = 64 Bytes (2^6 words)
- 4 MB Direct Mapped L2 Cache
- $2^{22}/2^6 = 2^{16}$

6th Optimization : Optimized to Improve the Hit Time



Modified Specification for Cache

64 bit Virtual Address

41 bit Physical Address

16 KB (2^{14}) Page Size

2 Way, 256 Entry TLB

16 KB (2^{13}) Direct Mapped L1 Cache

Block Size = 64 Bytes (2^6 words)

4 MB, 4 Way, L2 Cache

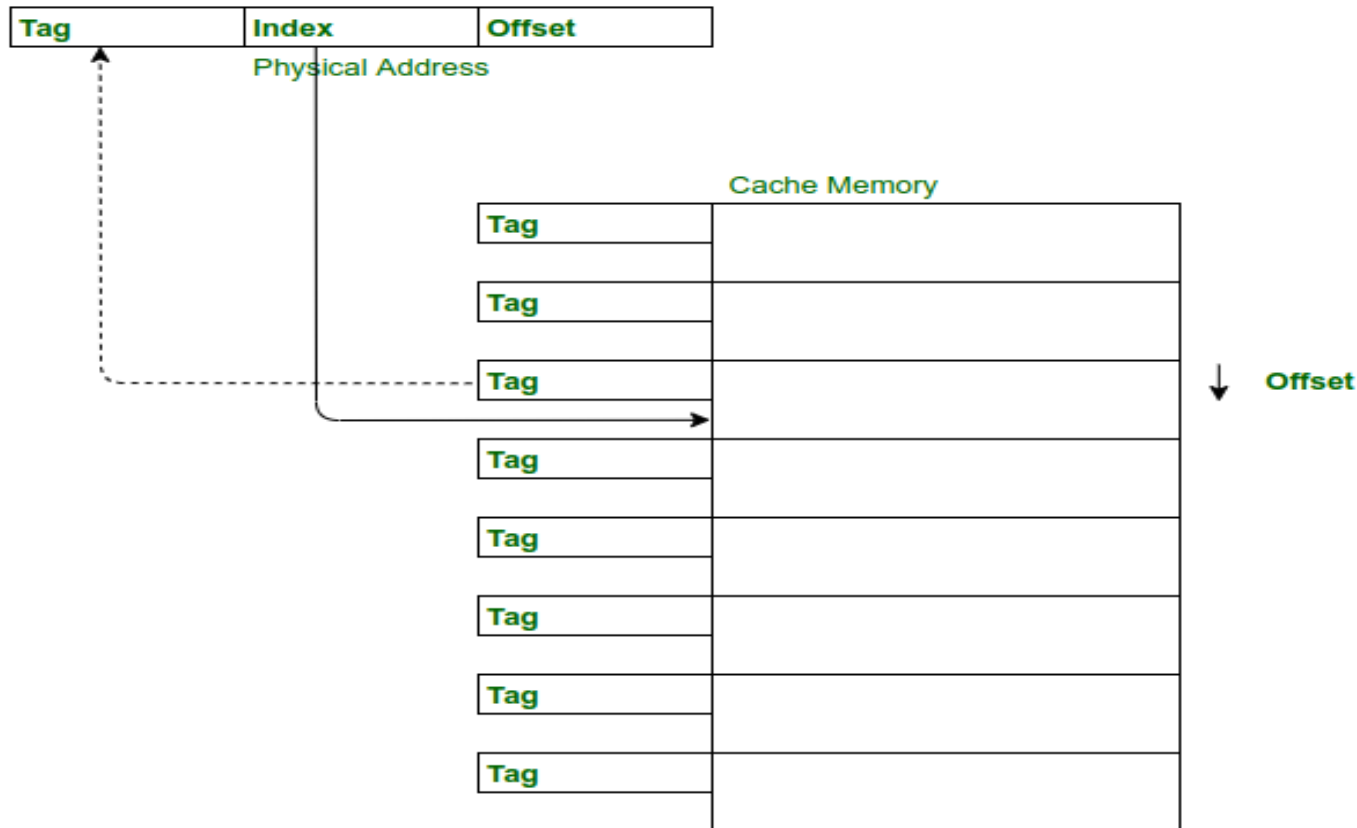
$2^{22}/2^6 \times 2^2 = 2^{14}$

SUMMARY OF SIX BASIC CACHE OPTIMIZATIONS

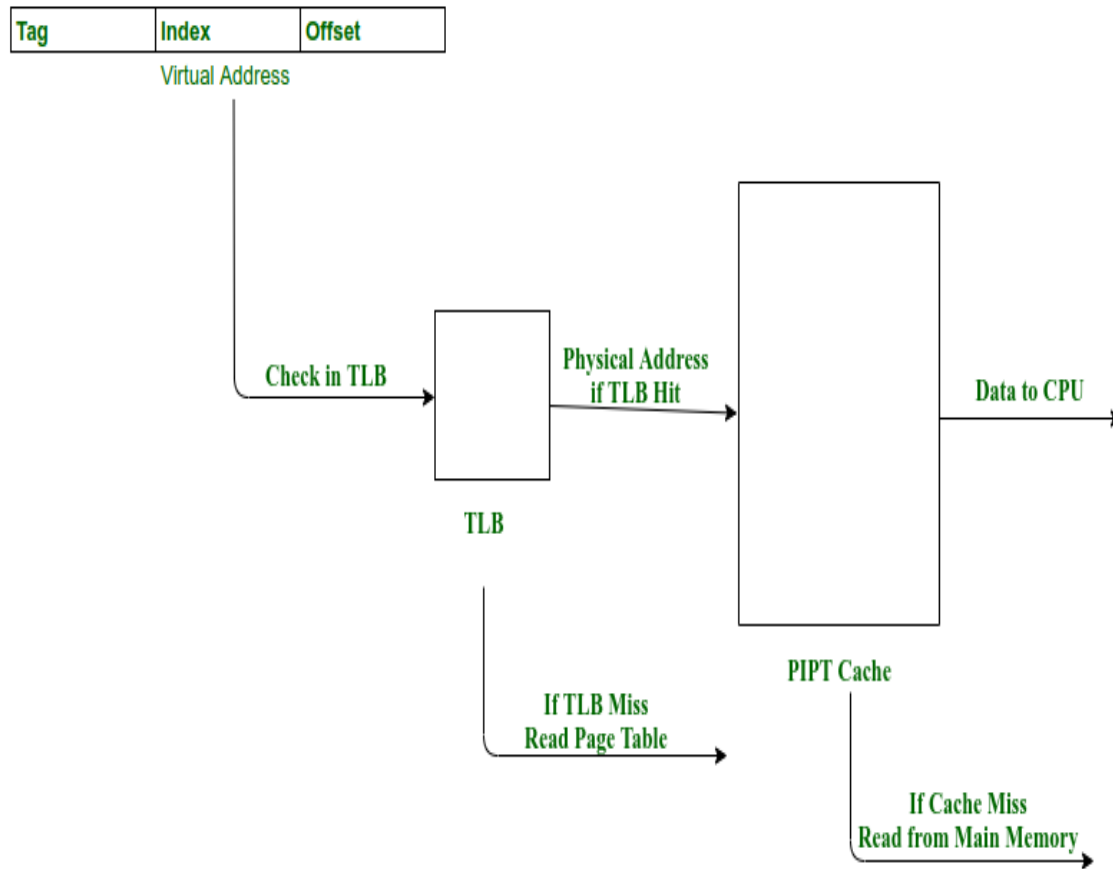
Average access time = Hit time + Miss rate x Miss penalty

1. Larger block size to reduce miss rate
 - Reduces compulsory misses
 - Increases capacity and conflict misses, increases miss penalty
2. Larger total cache capacity to reduce miss rate
 - Reduces capacity misses
 - Increases hit time, increases power consumption
3. Higher associativity to reduce miss rate
 - Reduces conflict misses
 - Increases hit time, increases power consumption
4. Higher number of cache levels to reduce miss penalty
 - Reduces overall memory access time
5. Giving priority to read misses over writes to reduce miss penalty
6. Avoiding address translation in cache indexing to reduce hit time

EXTRA-indexing and tagging



EXTRA-Physically indexed, physically tagged



1. Check the logical address in the Transition Look Aside Buffer (TLB) and If it is present in TLB, get the physical address of page from the TLB.
 2. If it is not present, access the page table from physical memory and then use the page table to obtain the physical address.
- All this time adds to the hit time. Hence, hit time = **TLB latency + cache latency**

Limitations of PIPT Cache

1. This process is quite sequential hence high hit time.
2. Not ideal for inner level caches.
3. Data cache is accessed frequently by any system and TLB access for data every time will slow the system significantly.

EXTRA- Virtually Indexed Virtually Tagged Cache

Tag	Index	Offset
-----	-------	--------

Virtual Address

If Present in Cache
i.e. Cache Hit

Data to CPU

VIVT Cache

If not Present in Cache

Check in main memory

TLB

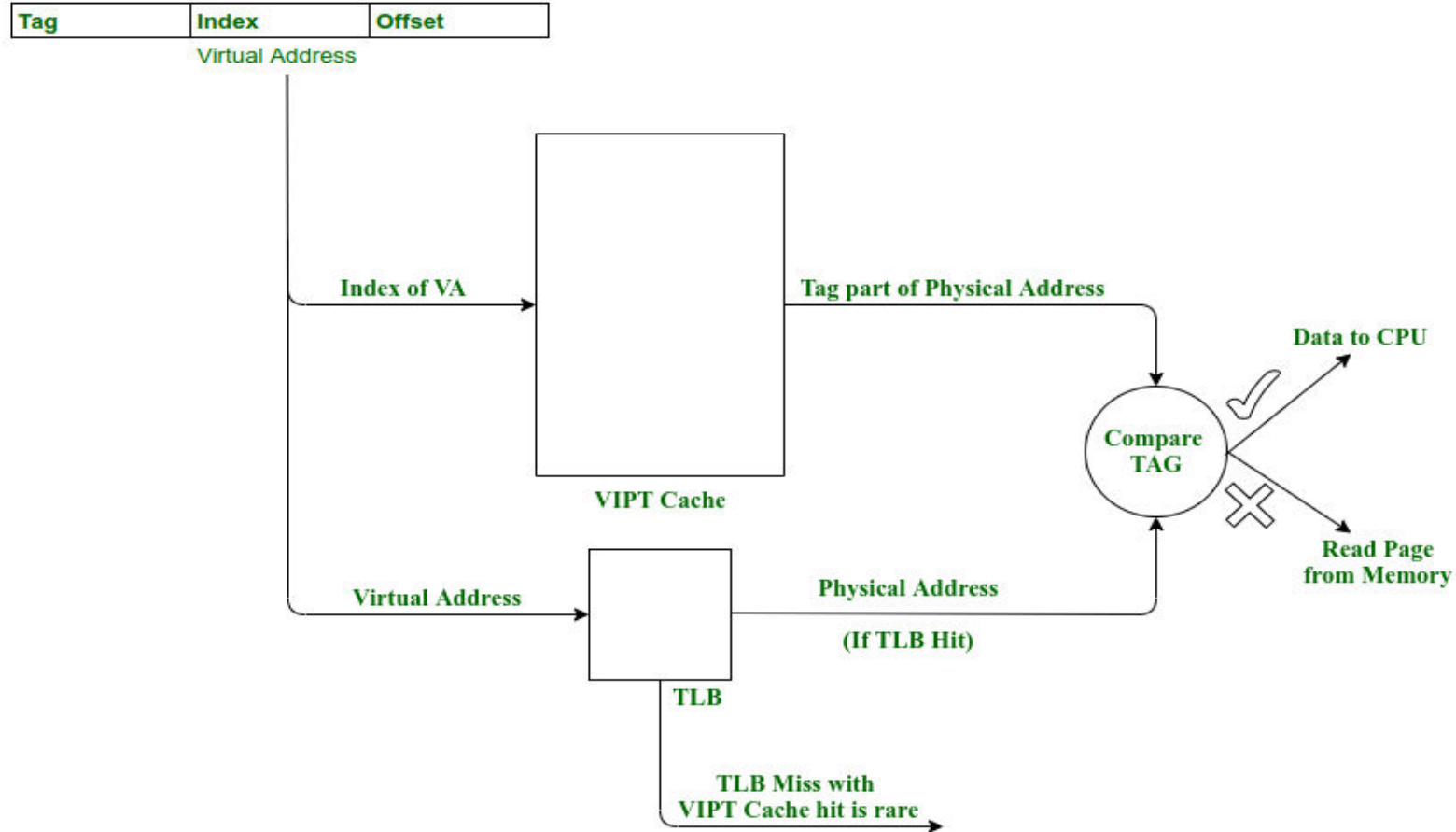
hit time for VIVT = cache hit time.

- Limitation- Works if data is in cache, upon TLB miss access main memory which requires physical address
- Two different virtual address for same physical address- aliasing

EXTRA- Virtually Indexed Physically Tagged Cache

- The cache is searched using the virtual address and tag part of physical address is obtained.
- The TLB is searched with virtual address, and physical address is obtained.
- Finally, the tag part of physical address obtained from the VIPT cache is compared with physical address's tag obtained from TLB.
- If they both are same, then it is a cache hit else a cache miss.

EXTRA



EXTRA

- TLB's access time will be lesser than Cache's access time.
Hence, hit time= **cache hit time**.
VIPT cache takes same time as VIVT cache during a hit and solves the problem VIVT cache:
 1. Since the TLB is also accessed in parallel the flags can be checked at the same time.
 2. The VIPT cache uses part of physical address as index and since every memory access in the system will correspond to a unique physical address, data for multiple processes can exist in the cache and hence no need to flush data for every context switch.

EXTRA- Advantages

1. Avoids sequential access reducing the hit time
2. Useful in data caches where the access to cache is frequent
3. Avoids cache misses on context switch
4. TLB flags can be accessed in parallel with cache access

Example: Direct-Mapped Caches

64-line direct-mapped cache $\rightarrow$ 64 indexes $\rightarrow$ 6 index bits

Read Mem[0x400C]

0100 0000 0000 1100

TAG: 0x40

INDEX: 0x3

OFFSET: 0x0

HIT, DATA 0x42424242

Would 0x4008 hit?

INDEX: 0x2 $\rightarrow$ tag mismatch $\rightarrow$ miss

	Valid bit	Tag (24 bits)	Data (32 bits)
0	1	0x000058	0xDEADBEEF
1	1	0x000058	0x00000000
2	0	0x000058	0x00000007
3	1	0x000040	0x42424242
4	1	0x000007	0x6FBA2381
	⋮	⋮	⋮
63	1	0x000058	0xF7324A32

1. What is the primary problem that the 6th cache optimization attempts to solve to reduce hit time?

- A) The limited capacity of the L1 cache.
- B) The time delay caused by converting a Virtual Address to a Physical Address before indexing the cache.
- C) The high rate of conflict misses in direct-mapped caches.
- D) The slow speed of the Main Memory compared to the CPU. **Answer: B**

2. Which strategy is introduced as a solution to avoid waiting for the Virtual Address (VA) to Physical Address (PA) translation?

- A) Physically Indexed and Physically Tagged (PIPT)
- B) Virtually Indexed and Physically Tagged (VIPT)
- C) Avoiding the use of a Translation Lookaside Buffer (TLB)
- D) Physically Indexed and Virtually Tagged (PIVT) **Answer: B**

3. In a standard Physically Indexed and Physically Tagged (PIPT) cache system, what must occur before the cache can be indexed and tagged?

- A) The cache must index the block using the Virtual Address.
- B) The main memory must send the data directly to the CPU.
- C) The CPU generates a Virtual Address, refers to the TLB, and generates the Physical Address.
- D) The tag is extracted from the Virtual Address before the TLB is accessed. **Answer: C**

4. Under the "Virtually Indexed & Physically Tagged" optimization, where does the cache controller get the information it needs for indexing and tagging?

- A) Both the Index and the Tag are extracted directly from the Virtual Address.
- B) Both the Index and the Tag

1. What is the primary problem that the 6th cache optimization attempts to solve to reduce hit time?

- A) The limited capacity of the L1 cache.
- B) The time delay caused by converting a Virtual Address to a Physical Address before indexing the cache.
- C) The high rate of conflict misses in direct-mapped caches.
- D) The slow speed of the Main Memory compared to the CPU.

Answer: B

2. Which strategy is introduced as a solution to avoid waiting for the Virtual Address (VA) to Physical Address (PA) translation?

- A) Physically Indexed and Physically Tagged (PIPT)
- B) Virtually Indexed and Physically Tagged (VIPT)
- C) Avoiding the use of a Translation Lookaside Buffer (TLB)
- D) Physically Indexed and Virtually Tagged (PIVT)

Answer: B

3. In a standard Physically Indexed and Physically Tagged (PIPT) cache system, what must occur before the cache can be indexed and tagged?

- A) The cache must index the block using the Virtual Address.
- B) The main memory must send the data directly to the CPU.
- C) The CPU generates a Virtual Address, refers to the TLB, and generates the Physical Address.
- D) The tag is extracted from the Virtual Address before the TLB is accessed.

4. Under the "Virtually Indexed & Physically Tagged" optimization, where does the cache controller get the information it needs for indexing and tagging?

- A) Both the Index and the Tag are extracted directly from the Virtual Address.
- B) Both the Index and the Tag are extracted from the Physical Address.
- C) The Index is extracted from the Physical Address, and the Tag is extracted from the Virtual Address.
- D) **The Index is extracted from the Virtual Address, and the Tag is extracted from the Physical Address.**

3. In a standard Physically Indexed and Physically Tagged (PIPT) cache system, what must occur before the cache can be indexed and tagged?

- A) The cache must index the block using the Virtual Address.
- B) The main memory must send the data directly to the CPU.
- C) The CPU generates a Virtual Address, refers to the TLB, and generates the Physical Address.
- D) The tag is extracted from the Virtual Address before the TLB is accessed.

Answer: C

4. Under the "Virtually Indexed & Physically Tagged" optimization, where does the cache controller get the information it needs for indexing and tagging?

- A) Both the Index and the Tag are extracted directly from the Virtual Address.
- B) Both the Index and the Tag are extracted from the Physical Address.
- C) The Index is extracted from the Physical Address, and the Tag is extracted from the Virtual Address.
- D) **The Index is extracted from the Virtual Address, and the Tag is extracted from the Physical Address.**

Answer: D



THANK YOU



Team MPCA

Department of Computer Science and
Engineering